

Project Plan
for
Time Management System

TABLE OF CONTENTS

TABLE OF CONTENTS i

LIST OF TABLES iv

Sidebar Navigation Structure by User Role 1

Member UI 1

Tabs Descriptions 1

Manager UI 4

Tabs Descriptions 4

Super Admin UI 7

Tabs Description 8

Client UI 13

Tabs Descriptions 13

Web and Desktop Agent 16

Core Tracking Components 17

Desktop Agent 17

Browser Extension 17

Backend API Specification 18

Data Model 18

Security Implementation 19

UI Integration Points 20

Deployment Guide 21

Member 22

Time Tracker 22

Live Session 22

Session History 29

My Work 33

Daily Log 33

Weekly Summary 43

Add Time 50

Focus Dashboard 58

Productivity Pulse 58

Break Analytics 66

My Submissions 71

Timesheets 71

Time Off 75

Drafts 80

Preferences 85

Workflow 85

Privacy Guard 90

Notification 95

Manager 101

My Team 101

Timesheets 101

Approvals Hub 107

Activity Monitor 112
Productivity 116
Work Distribution 116
Focus Analysis 122
Team Trends 127
Team 135
Members 135
Assignments 141
Capacity Planner 146
Time Of 156
Requests 156
Balances 163
Super Admin 169
Time Management 169
Timesheet Explorer 169
Edge Case Handling 173
Approvals Hub 174
Audit Trail 180
Reports & Insights 185
Productivity Trends 185

LIST OF TABLES

Table 1. Data Capture 17

Table 2. Backend API Essential Endpoints 18

Table 3. Data Protection 19

Table 4. API Test Matrix ..... **Error! Bookmark not defined.**

Table 5. Component Breakdown and API Mapping... **Error! Bookmark not defined.**

Table 6. API Endpoints..... 29

Table 7. API Tests ..... **Error! Bookmark not defined.**

Table 8. Component-API Mapping ..... **Error! Bookmark not defined.**

Table 9. API Endpoints..... 34

Table 10. API Tests..... 36

Table 11. Component-API Mapping 41

Table 12. API Endpoints..... 44

Table 13. Component-API Mapping 49

Table 14. Test Case..... 49

Table 15. API Endpoints..... 50

Table 16. API Endpoints..... 57

Table 17. API Endpoints..... 59

Table 18. Component-API Mapping 63

Table 19. Test Cases 65
Table 20. API Endpoints of Break Analytics 67
Table 21. Component-API Mapping of Break Analytics 71
Table 22. API Endpoints of Timesheets Tab 72
Table 23. Component-API Mapping of Timesheets tab 74
Table 24. API Endpoints of Time Off tab 76
Table 25. Component-API Mapping of Time Off tab 78
Table 26. Edge Case Handling 79
Table 27. API Endpoints of Draft tab 81
Table 28. Component – API Mapping of Drafts tab 84
Table 29. Edge Case Handling 85
Table 30. API Endpoints of Workflow tab 86
Table 31. Component-API Mapping of Workflow 89
Table 32. Edge Case Handling 90
Table 33. API Endpoint of Privacy Guard 91
Table 34. Component-API Mapping of Privacy Guard 93
Table 35. Edge Case Handling 94
Table 36. API Endpoints of Notification tab 96
Table 37. Component-API Mapping of Notification tab 98
Table 38. Edge Case Handling 99
Table 39. API Endpoints of Timesheets tab 101

Table 40. Component-API Mapping of Time Sheet Tab.....105
Table 41. Testing Strategy.....106
Table 42. API Endpoints of Approvals Hub.....108
Table 43. UI Component Layout Outline of Approvals Hub Tab.....109
Table 44. Component-API Mapping of Approvals Hub tab.....110
Table 45. API Endpoints of Activity Monitor tab.....112
Table 46. Component-API Mapping of Activity Monitor Tab.....114
Table 47. Testing Strategy.....115
Table 48. API endpoints.....117
Table 49. Component-API Mapping of Work Distribution Tab.....119
Table 50. Testing Strategy for Work Distribution Tab.....121
Table 51. API Endpoints of Focus Analysis Tab.....122
Table 52. Component-API Mapping of Focus Analysis Tab.....124
Table 53. Testing Strategy for Focus Analysis Tab.....126
Table 54. API Endpoints of Team Trends Tab.....127
Table 55. Query Parameters of GET /api/team/trends.....128
Table 56. Error Responses.....131
Table 57. Component-API Mapping of Team Trends Tab.....133
Table 58. Testing Strategy for Team Trends Tab.....134
Table 59. API Endpoints of Members Tab.....135
Table 60. Query Parameters for GET /api/team/members:.....136

Table 61. Component- API Mapping of Members Tab.....138

Table 62.API Endpoints of Assignments Tab.....141

Table 63. Query Parameters for GET /api/assignments.....141

Table 64. Component-API Mapping of Assignments Tab 144

Table 65. Testing Strategy for Assignments Tab.....146

Table 66. API Endpoints of Capacity Planner Tab.....147

Table 67. Component-API Mapping of Capacity Planner Tab.....153

Table 68. Testing Strategy for Capacity Planner Tab.....155

Table 69. Edge Case of Capacity Planner 156

Table 70. API Endpoints of Request Tab.....157

Table 71. Testing Strategy for Request Tab.....162

Table 72. Edge Case Handling for Request tab.....163

Table 73. API Endpoints of Balance Tab.....163

Table 74. Edge Case Handling of Balances Tab 168

Table 75. API Endpoints of Timesheet Explorer.js.....169

Table 76. Testing Strategy.....173

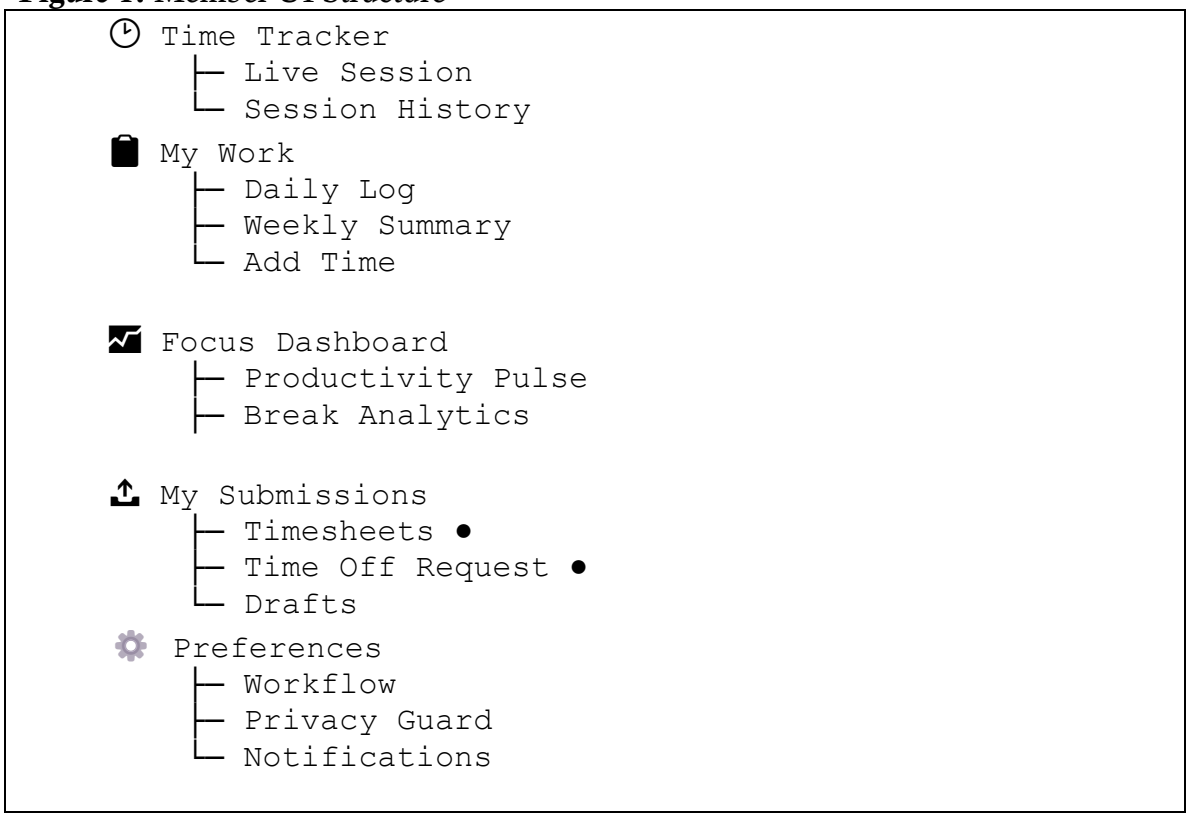
Table 77. Edge Case Handling for Timesheet Explorer Tab.....174

Table 78. API Endpoints for 175

SIDEBAR NAVIGATION STRUCTURE BY USER ROLE

Member UI

Figure 1. Member UI Structure



Tabs Descriptions

Time Tracker

Purpose: Central hub for tracking work sessions with minimal friction.

- **Live Session**
  - One-tap timer: Start, pause, and stop with auto-tagging based on active app.
  - Auto-resume: Recovers sessions after agent restart or via manual resume.

- Visual feedback: Pulsing border (green = tracking, yellow = paused).

- **Session History**
  - Visual timeline with color-coded activity types (green, orange, red).
  - Clickable bars reveal session details.
  - Filter by date, project, or tag.

    Tech: Real-time sync via WebSocket, local storage in SQLite during outages.

📋 My Work

Purpose: Manage and refine daily and weekly time records.

- **Daily Log**
  - Resizable time blocks on a 24h timeline.
  - Inline editing with validation (start < end).
- **Weekly Summary**
  - Flags under-tracked and overworked days.
  - Exports to branded PDF and raw CSV.
- **Add Time**
  - AI-based suggestions from calendar events.
  - Smart defaults based on previous entries.

    Tech: CRUD via `/api/timesheets`, OAuth2.0 for calendar sync.

📈 Focus Dashboard

Purpose: Self-improvement through behavioral insights.

- **Productivity Pulse**
  - Scoring algorithm with peer comparison.
- **Break Analytics**
  - Recommends and reminds ideal break times.

⬆️ My Submissions

Purpose: Submit timesheets and manage time-off requests.

- **Timesheets**
  - Visual approval progress with real-time badge updates.
  - Bulk submission grouped by project.
- **Time Off**
  - PTO calculator and historical usage.
  - Previews company leave policies.
- **Drafts**
  - Auto-saves locally and backs up in cloud.
  - Prompts draft recovery on login.

    Tech: Offline storage via localForage, submission status via PATCH API.

⚙️ Preferences

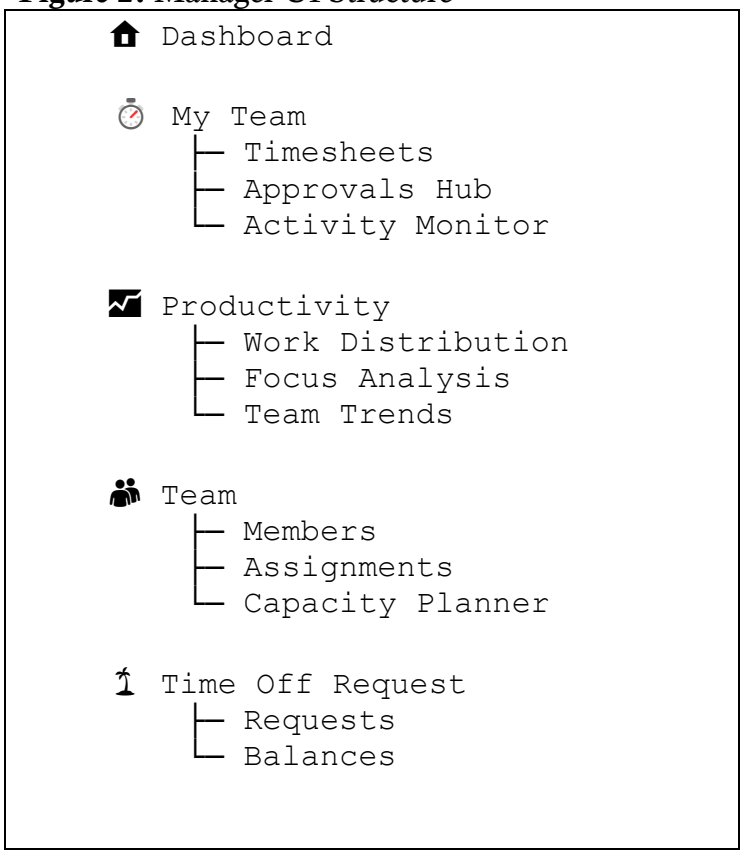
Purpose: Customize workflows, privacy, and notifications.

- **Workflow**
  - Timer presets for common routines.
  - Idle detection with "Ignore in Meetings" option.
- **Privacy Guard**
  - App blocking and blurred screenshots.
  - Full tracking suspension options.
- **Notifications**
  - Schedule-specific alerts with priority settings.
  - Test mode for previewing behavior.

    Tech: Agent-controlled settings synced to user profile.

Manager UI

Figure 2. Manager UI Structure



Tabs Descriptions

Dashboard

Purpose: Real-time overview of team activity and critical KPIs.

- **Live Status Grid**:
  - Color-coded avatars (Green = active, Yellow = idle, Red = offline).
  - Hover reveals active app, task, and hours.
  - Auto-refresh via WebSocket every 60s.
- **Priority KPIs**:
  - Tracked hours, average focus time, overdue approvals.

- **Quick Actions**:
    - Nudge idle team via Teams/Email.
    - One-click daily PDF report.

      Tech: WebSocket live data, role-filtered API.

⏱ My Team

Purpose: Manage team timesheets and monitor productivity.

- **Timesheets**:
    - Flags unusual patterns and missing notes.
    - Inline editing with real-time validation.
- **Approvals Hub**:
    - Groups policy exceptions (weekend, overtime).
    - Bulk feedback with follow-up task scheduling.
- **Activity Monitor**:
    - Color-coded timelines with drill-down.
    - Export as PNG/PDF.

      Tech: Fast access via Redis caching, audit logs in MongoDB.

📈 Productivity

Purpose: Improve output using team-wide behavioral analytics.

- **Work Distribution**:
    - Project health via burn rate and phase tracking.
    - Burnout alerts for excessive focused time.
- **Focus Analysis**:
    - Metrics like switches/hour and deep work scoring.
    - Suggests focus improvement strategies.
- **Team Trends**:
    - Anonymous benchmarks and pattern recognition.

- Highlights habits without naming individuals.

  Tech: Aggregation pipelines, third-party PM tool integration.

🔔 Notifications

Purpose: Alert center for escalations and team updates.

- **Action Center**:
  - Priority tiers (Yellow = reminders, Red = violations).
  - Response templates for fast approval or revision.

    Tech: Priority-based POST API, Teams/email integration via webhooks.

👥 Team

Purpose: Resource planning and skills/capacity tracking.

- **Members**:
  - Heatmap view of workload and time-off trends.
  - Skill tagging via app usage data.
- **Assignments**:
  - Drag-and-drop scheduler with conflict warnings.
- **Capacity Planner**:
  - Models project outcomes with resource changes.
  - Highlights skill gaps.

    Tech: Forecast API, real-time updates via WebSocket.

🏝 Time Off

Purpose: Simplify leave requests while ensuring team coverage.

- **Requests**:
  - Calendar overlay with conflict alerts.

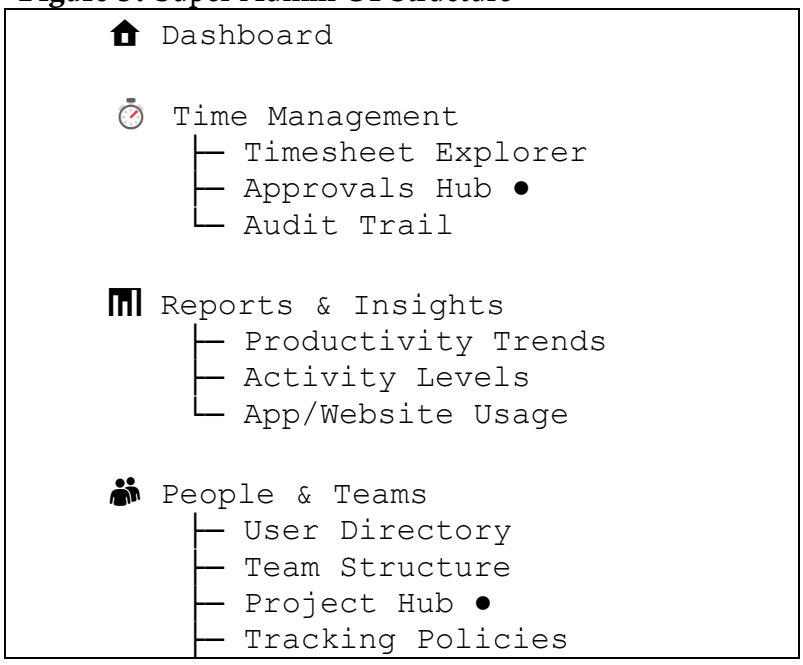
- Suggests backup resources based on skills.

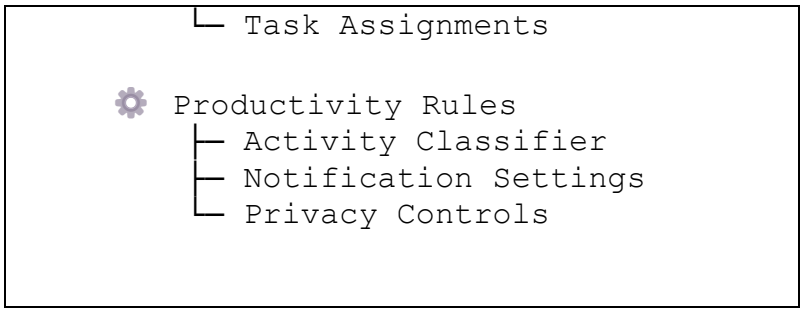
- **Balances**:
  - Forecasts annual usage and tracks accruals.
  - Links to policy guide sections.

    Tech: Balance API, smart assignment logic.

Super Admin UI

Figure 3. Super Admin UI Structure





Tabs Description

Dashboard

Purpose: Centralized oversight of organizational time-tracking health and compliance.

1. Live Activity Feed

- **Real-Time Monitoring**:
  - Team-level tracking across departments (role-filtered).
  - Status indicators: Green = productive, Red = idle >30min.
- **Compliance Alerts**:
  - Overdue timesheets (>5) flagged.
  - Policy violations (e.g., weekend work) detected.

2. Quick Actions

- **Time-Tracking Tasks**:
  - Review idle sessions.
  - Pause tracking system-wide for maintenance.
- **Legacy Tool Integration**:
  - E.g., "Sync BambooHR Data".

Technical Integration:

- WebSocket via `GET /api/org/activity`.
- Role-based access from existing systems.

⏱️ Time Management

Purpose: Govern time data accuracy and accountability.

1. Timesheet Explorer

- **Granular Filters**:
  - By team/project/date.
- **Audit Integration**:
  - Full edit history with user/IP logging.
  - Rollback support for error correction.

2. Approvals Hub

- **Unified Queue**:
  - Includes timesheets, PTO, and exceptions.
  - Priority sorting for deadlines.
- **Decision Tracking**:
  - Logs approvals/rejections to HR (e.g., Workday).

3. Audit Trail

- **Time-Specific Logs**:
  - Tracks who edited what and when.
  - GDPR-compliant exports available.

Technical Integration:

- MongoDB logs with TTL.
- SIEM sync (e.g., Splunk).

📊 Reports & Insights

Purpose: Strategic analysis of organizational productivity.

1. Productivity Trends

- **Cross-Team Comparisons**:
  - Idle rates, app usage, focus time.
- **App/Website Usage**:
  - Rank top distractions.
  - Estimate productivity cost in hours/dollars.

2. Activity Levels

- **Idle Detection**:
  - Heatmaps and correlation to meetings/apps.
- **Attendance Cross-Check**:
  - Compare timesheets to badge swipe data.

Technical Integration:

- Apache Spark for aggregation.
- PowerBI/Tableau export via `GET /api/reports/export`.

👥 People & Teams

Purpose: Apply time policies and manage personnel structures.

1. User Directory

- **Enhanced Profiles**:
  - Status (e.g., "Active Timer").
  - Compliance score (1–5 stars).

- **Bulk Actions**:
    - Apply rules (e.g., blur screenshots).

2. Team Structure

- **Policy Inheritance**:
    - Hierarchical adoption with overrides.
- **Visual Org Chart**:
    - Indicators (e.g., "🔒 Strict Mode").

3. Tracking Policies

- **Screenshot Rules**:
    - Off / Blurred / Full with frequency controls.
    - App-level exclusions.
- **Idle Thresholds**:
    - Dynamic by role.
    - Auto-pause after threshold breach.

💼 Project Hub

Purpose: Manage projects, client connections, and resource planning.

Features & Implementation

- **Add Projects**:
    - Link clients via `POST /api/projects`.
    - Billing type: Fixed / Time & Materials.
- **Team Assignment**:
    - Drag & drop UI (React DnD).
    - Capacity warnings via `GET /api/team/capacity`.

- **Client Access**:
    - Create client portals.
    - Apply watermarks to shared content.

Technical Integration:

- PostgreSQL policy storage with version control.
- Sync via `PUT /api/policies`.

⚙️ Productivity Rules

Purpose: Define activity classification and preserve privacy.

1. Activity Classifier

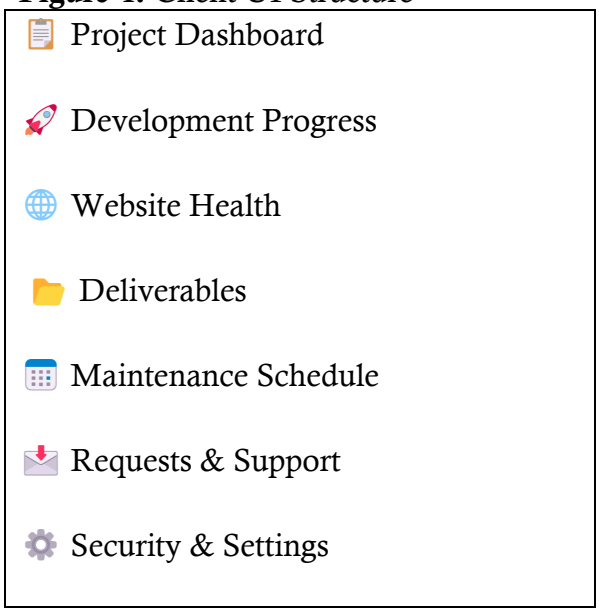
- **Rule Logic**:
    - Wildcard patterns (e.g., *.salesforce.com = productive).
    - Regex matching for complex cases.
- **Override Handling**:
    - Manual overrides take precedence.

2. Privacy Controls

- **User Blacklists**:
    - Employees block personal apps.
    - Admins enforce global exclusions.
- **Data Redaction**:
    - Screenshots blurred for blacklisted content.

Client UI

Figure 4. Client UI Structure



Tabs Descriptions

📋 Project Dashboard

Purpose: Central hub for high-level project oversight

Key Features:

• Real-time phase progress (Design → Development → Testing → Launch)

• Budget tracker (actual vs. estimated)

• Milestone status indicators (✅ On Track / ⚠️ At Risk / ❌ Delayed)

• Team availability calendar for client meetings

🚀 Development Progress

Purpose: Monitor code-level implementation and feature delivery

Key Features:

• Commit timeline (sensitive files excluded)

• Feature heatmap showing % progress by area (frontend/backend/testing)

• Live preview links (Staging/UAT)

• Automated test summaries from CI/CD pipelines

🌐 Website Health

Purpose: Ensure live site reliability and performance

Key Features:

• Uptime monitor with 30-day history and response times

• Weekly Lighthouse performance audits

• SSL certificate expiry alerts

• Traffic analytics (visitors, bandwidth usage)

💼 Deliverables

Purpose: Secure access to project artifacts

Key Features:

• Versioned design exports (Figma/XD)

• Watermarked PDF contracts

• Temporary test environment links with 24h expiry

• Final sign-off approval workflow

📆 Maintenance Schedule

Purpose: Track ongoing support and scheduled updates

Key Features:

• Maintenance calendar (Planned / Security / Emergency)

• Before/after reports with performance comparisons

• Dependency update logs and backup status overview

📨 Requests & Support

Purpose: Streamline client communication and support

Key Features:

• Tiered ticket system with SLA indicators (48h / 24h / 4h)

• Change request pipeline with budget warnings

• Knowledge base of FAQs and past solutions

⚙ Security & Settings

Purpose: Manage client account and security preferences

Key Features:

• Two-factor authentication setup

• Activity log (IP access, file downloads)

• Notification preferences (critical alerts, weekly digests)

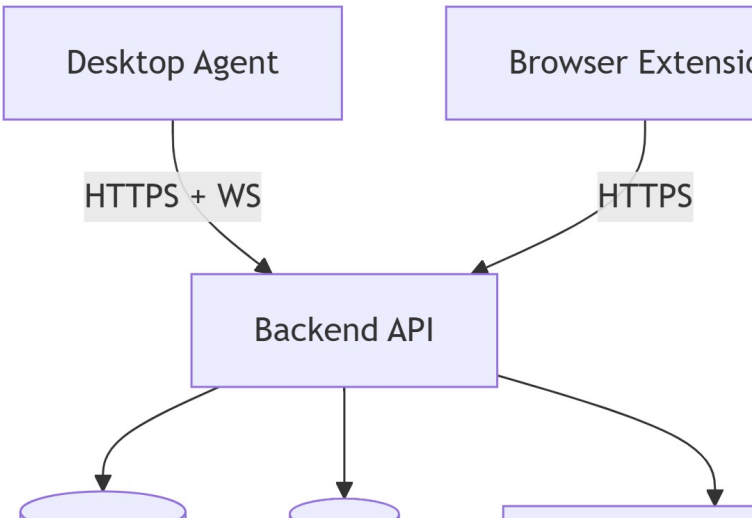
WEB AND DESKTOP AGENT

Objective: Capture comprehensive work activity data across desktop apps and web browsers to power all UI features.

Key Components:

- **Desktop Agent**: Tracks native apps, windows, and user input
- **Browser Extension**: Monitors URL/tab changes and web activity
- **Backend API**: Unifies data from all sources
- **Real-Time Sync**: WebSocket for live UI updates

Figure 5. Architectural Blueprint



Core Tracking Components

Desktop Agent

Tech Stack:

```
framework: Electron
monitoring:
  - active-win (window tracking)
  - iohook (input detection)
storage: SQLite (local buffer)
security: AES-256 encryption
```

Table 1. Data Capture

Event Type	Frequency	Example
Active Window	5s	{app: "VS Code", title: "src/index.js"}
Screenshot	120s	s3://encrypted-screenshots/session123/...
Input State	Realtime	{idle: false, lastActive: "2023-11-20T08:30:00Z"}

Browser Extension

Key Features:

```
{
  "manifest_version": 3,
  "permissions": ["tabs", "webNavigation"],
  "background": {
    "service_worker": "bg.js"
  }
}
```

Tracked Data:

```typescript
interface WebActivity {
  url: string;       // "https://docs.google.com/document/d/123"
  domain: string;    // "docs.google.com"
  tabTitle: string;  // "Project Report Draft"
  activeDuration: number; // Seconds
}
```

Backend API Specification

Table 2. Backend API Essential Endpoints

Endpoint	Method	Purpose
`/api/activity/batch`	POST	Ingest desktop/browser events
`/ws/activity`	WebSocket	Real-time UI updates
`/api/sessions/unresolved`	GET	Crash recovery data

Batch Payload Example:

```json
{
  "userId": "user_123",
  "deviceId": "device_456",
  "events": [
    {
      "type": "app",
      "timestamp": "2023-11-20T08:30:00Z",
      "appName": "Visual Studio Code",
      "windowTitle": "index.js - ProjectX"
    }
  ]
}
```

Data Model

Raw Event Schema

```
{
  userId: ObjectId,
  timestamp: Date,
  type: {
    type: String,
    enum: ['app', 'url', 'input']
  },
  details: {
    appName: String,    // For 'app' type
    windowTitle: String,
    url: String,        // For 'url' type
    idle: Boolean       // For 'input' type
  },
  source: {
    type: String,
    enum: ['desktop', 'browser']
  }
}
```

Processed Session Schema

```
{
  startTime: Date,
  endTime: Date,
  userId: ObjectId,
  dominantApp: String,
  productivityScore: Number,
  webDomains: [String],
  screenshotRefs: [String]
}
```

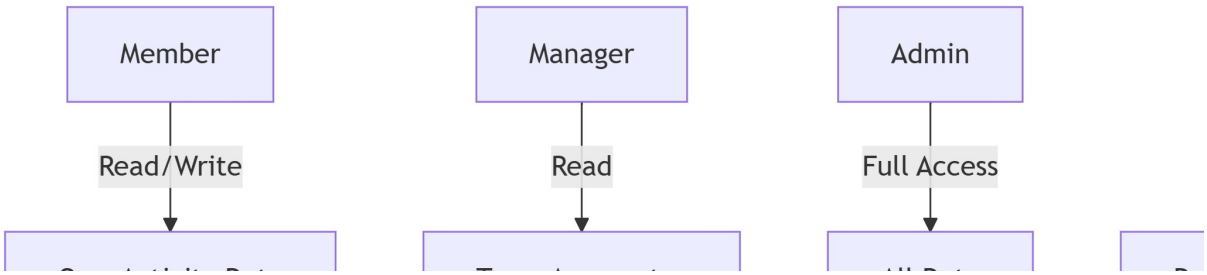
Security Implementation

Table 3. Data Protection

Layer	Security Measure

Layer	Security Measure
Network	TLS 1.3 for all communications
Storage	AES-256 encryption at rest
Access	JWT authentication with AD integration

Figure 6. Permission Model



UI Integration Points

Real-Time Updates:

```
// WebSocket Connection
const socket = new WebSocket('wss://api.example.com/ws/activity');

socket.onmessage = (event) => {
  const data = JSON.parse(event.data);
  updateLiveSessionDisplay(data);
};
```

Historical Data Fetch

```
// Get session history
async function loadHistory(userId, date) {
```

```
  const response = await
fetch(`/api/sessions?userId=${userId}&date=${date}`);
  return response.json();
}
```

Deployment Guide

Desktop Agent Setup

Windows:

Start-Process -FilePath "TimeTrackerAgent.msi" -ArgumentList "/quiet"

macOS:

sudo installer -pkg TimeTracker.pkg -target /

Browser Extension Rollout

```
// Chrome Enterprise Policy
{
  "ExtensionInstallForcelist": [
    "your-extension-id;https://internal-store.example.com"
  ]
}
```

MEMBER

Time Tracker

Live Session

Objective: Enable one-tap time tracking with automated attendance marking and privacy controls.

Key Features

1. **Tracking**
    - Start/stop timer → Auto-mark attendance
    - Aggregate multiple sessions into daily totals
2. **Session Management**
    - Auto-resume crashed sessions
    - Privacy-preserving screenshots (blur/block)
    - Manual time block adjustments (±15min)
3. **Real-Time Feedback**
    - WebSocket-powered progress updates
    - Real-time productivity scoring
4. **Manual Overrides**
    - Edit tags, resume crashed sessions.
    - Classify sessions as productive/distracting

Backend Development

Endpoint	Method	Purpose
POST /api/sessions/start	POST	Start session; returns sessionId, autoTag.
PUT /api/sessions/pause	PUT	Pause session; updates status to paused.
PUT /api/sessions/stop	PUT	Stop session; applies manualTag if provided.
GET /api/sessions/unresolved	GET	Fetch crashed sessions for auto-resume.
GET /api/sessions/current/context	GET	Get live data (screenshot, active apps).
GET /api/attendance/today	GET	Get daily summary Returns { present: bool, totalHours: float }
WebSocket /ws/session-progress	WS	Stream real-time duration and productivity.

Data Models

Session Schema (Mongoose):

```
  userId: ObjectId,
  sessionId: String,
  startTime: Date,      // Server timestamp
  endTime: Date,
  status: "active" | "paused" | "stopped",
  tags: {
    auto: String,       // Generated from active app (e.g., "Development")
    manual: String      // User override (e.g., "Client Meeting")
  },
isProductive: Boolean,
  appContext: {
    activeApp: String,  // Current app name (e.g., "VS Code")
    windowTitle: String,// Window/tab title (e.g., "index.js - ProjectX")
    screenshots: [{
```

```
      url: String,      // S3 path (e.g.,
"s3://screenshots/session123/1.jpg")
    }
  ]}
```

Attendance Schema

```
{
  userId: ObjectId,
  date: Date,            // Day of attendance (indexed)
  clockIn: Date,         // First session start
  clockOut: Date,        // Last session end
  totalHours: Number,    // ∑(session durations)
  sessions: [ObjectId]   // Reference to child sessions
}
```

Frontend Integration

1. Timer Widget

Purpose: Start, pause, or stop tracking sessions with visual feedback.

- **Key Features**:
  - **Start**: POST /api/sessions/start → Starts session, auto-tags activity.
  - **Pause**: PUT /api/sessions/pause → Updates status to paused.
  - **Stop**: PUT /api/sessions/stop → Finalizes duration and tags.=
- **Visual Indicators**:
  - Attendance status badge: 🟢 Present | 6.2h/8h
  - Overtime indicator: ⚠️ 1.5h overtime

2. Auto-Resume Banner

Purpose: Recover crashed/abandoned sessions.

- **Behavior**:
    - Fetches unresolved sessions on mount: GET/api/sessions/unresolved.
    - Displays: *"Resume [X] minutes of [App Name]?"* with **Resume**/**Discard** buttons.
- **API**:
    - Resume: POST /api/sessions/start with existing session data.

3. Tag Editor

Purpose: Manually classify tracked activity.

- **Behavior**:
    - Auto-suggests tags via GET /api/sessions/current/tags (ML + regex).
    - Validates against allowed tags (from GET /api/tags).
    - Saves changes immediately: PUT /api/sessions/current/tag.

4. Activity Timeline

Purpose: Visualize and adjust tracked time blocks.

- **Key Features**:
    - **Color Coding**: Green (productive), Red (distracting).
    - **Drag Handles**: Adjust time blocks in 15-minute increments.
    - **APIs**:
        - Fetch data: GET /api/sessions?date={date}.

- Update time: PATCH /api/sessions/{id}.

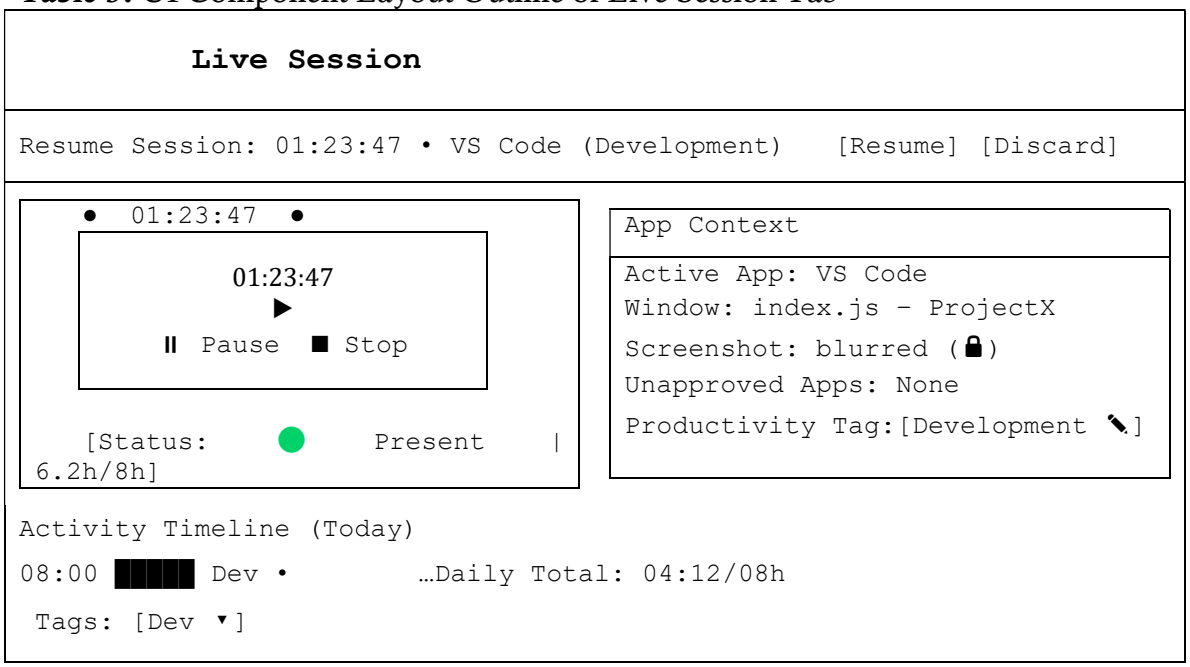
Active Session Panel

Table 4. Sub-Components of Active Session Panel

Component	Description
App Context Card	Displays blurred screenshots (from `GET /api/sessions/current/context`).
Productivity Tag	Editable badge (color matches project taxonomy). Uses `PUT /api/sessions/tag`.
Duration Progress	Real-time bar chart via `WebSocket /ws/session-progress` (fallback to polling).
Security Indicators	🔒 (privacy active) / ⚠️ (unapproved apps) from `GET /api/sessions/privacy-status`.

Visual Layout

Table 5. UI Component Layout Outline of Live Session Tab



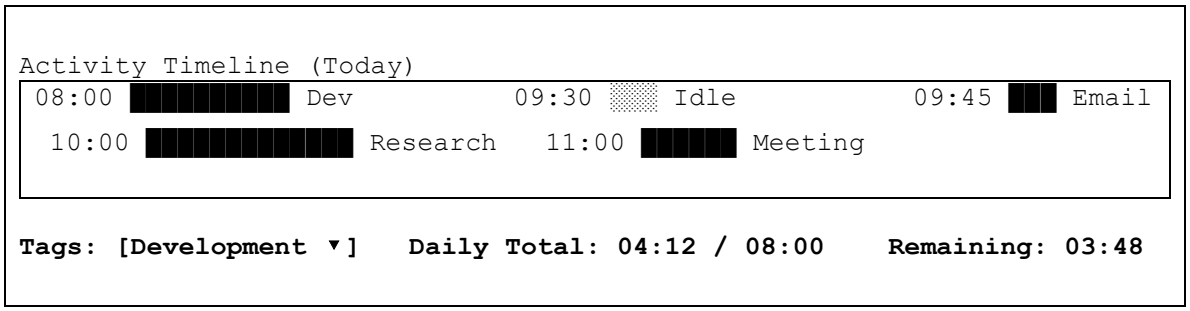


Table 6. Component-API Mapping of Live Session Tab

Component	Description	API Endpoints	Key Props/State
Timer Widget	Start/pause/stop session.	POST /sessions/start PUT /sessions/pause PUT /sessions/stop GET /attendance/today	sessionId, status, activeDuration, present:boolean, totalHours
Auto-Resume Banner	Resume crashed sessions.	GET /sessions/unresolved, POST /start	unresolvedSessions, isVisible
Tag Editor	Edit activity tags.	GET /current/tags, PUT /current/tag	currentTag, tagSuggestions
Activity Timeline	Adjust time blocks.	GET /sessions?date={date}, PATCH /{id}	sessions, selectedDate
App Context	Show live app/screenshot	GET /current/context, GET /screenshot/{id}	screenshotUrl, windowTitle

Component	Description	API Endpoints	Key Props/State
Card	data.		
Productivity Tag	Display/edit activity classification.	GET /current/tags, PUT /current/tag	finalTag, isEditing
Duration Progress	Real-time duration updates.	WS /ws/session-progress, GET /current/progress	activeSeconds, pausedSeconds
Security Indicators	Privacy/security status.	GET /privacy-status	privacyLevel, unapprovedApps

Critical Edge Cases

1. **Network Failure**:
    - Offline queuing with retries.
2. **Invalid Apps**:
    - Fallback to "Uncategorized" tag.
3. **Clock Skew**:
    - Server timestamps override client time.

Visual Workflow

1. **Start Session**:
    - User clicks "Start" → POST /api/sessions/start → Auto-tagging.
2. **Track Progress**:

- WebSocket updates UI → Fallback to GET /api/sessions/current/progress.

3. **Resolve Crashes**:
    - Fetch unresolved sessions → Prompt user to resume/discard.

Session History

Objective: Enable members to review, edit, and export past work sessions with visual timelines and compliance-ready reports.

Key Features

1. **Visual Timeline**: Color-coded sessions by activity type (e.g., green = productive).
2. **Adjustments**: Drag-and-resize time blocks (15-minute increments).
3. **Filters**: By date, project, or tag.
4. **Exports**: CSV/PDF reports with audit trails.

Backend Development

Table 7. API Endpoints of Session History Tab

Endpoint	Method	Purpose	Parameters
GET /api/sessions	GET	Fetch historical sessions.	?start=YYYY-MM-DD&end=YYYY-MM-DD&tag=
PATCH /api/sessions/{id}	PATCH	Edit session times or tags.	{ startTime, endTime }

Endpoint	Method	Purpose	Parameters
`GET /api/sessions/export/csv`	GET	Export sessions to CSV.	`?filter=...`
`GET /api/sessions/export/pdf`	GET	Generate PDF reports.	`?filter=...`

Data Model

```
{
  // Inherits from Live Session schema
  edits: [{
    user: ObjectId,       // Who edited
    timestamp: Date,      // When edited
    changes: {            // What changed
      startTime?: Date,
      endTime?: Date,
      tags?: String[]
    }
  }],
  exportHistory: [{
    format: "csv"|"pdf", // Export type
    requestedAt: Date,   // Timestamp
    recipient: String    // Manager/Client ID
  }]
}
```

Frontend Components

1. Activity Timeline

Purpose: Visualize and adjust past sessions.

- **Key Features**:
    - **Color Coding**: Matches tags (e.g., Development: #4CAF50).
    - **Drag & Resize**: Adjust session times (±15min).
    - **Tooltips**: Show app usage, duration, and notes on hover.
- **APIs**:
    - Fetch data: GET /api/sessions.
    - Update times: PATCH /api/sessions/{id}.

2. Export Panel

Purpose: Generate CSV/PDF reports.

- **Behavior**:
    - Applies current filters (date/tag) to exports.
    - Handles large datasets with pagination (?page=2&limit=50).
- **Edge Cases**:
    - Shows warning for exports >30 days.

3. Filters & Date Navigation

Purpose: Refine session data.

- **Components**:
    - **Date Picker**: Range or single-day selection.

- **Tag Dropdown**: Multi-select with search.
- **Quick Navigation**: Prev/Next buttons.

- **APIs**:
  - Auto-refresh timeline on filter change.

Visual Layout

```
       Session History
Filters: [Date Range ▾: 2025-05-01 – 2025-05-07]  [Tags ▾: Development]
         [← Prev] [Next →]            [Export CSV] [Export PDF]
Activity Timeline
 05/01 ▓▓▓▓▓▓▓▓▓▓▓ Dev          09:30 ──▌ Idle          10:00 ▒
 05/01 ▒▒▒▒▒▒▒▒▒▒▒ Idle         11:15 ▓▓▓▓▓ Meeting     12:00 ▒
 05/02 ▓▓▓▓▓▓▓▓▓▓ Research      14:00 ▓▓▓▓▓▓▓ Email     14:30 ▓
 …
* Drag handles to resize (±15 min) • Click block for details/edit
Edit Panel (on block click)
• Start Time: [09:00]   End Time: [11:30]  [Save] [Cancel]
• Tags: [ Development ▾ ]
• Last Edited: 05/02 15:00 by You
Export Panel
• Current Filters: May 1–7, Tags: Development
• [Generate CSV] [Generate PDF]
• Warning: "Exports >30 days may be delayed."
```

Table 8. UI Component Layout Outline of Session History Tab

Table 9. Component-API Mapping of Session History Tab

Component	Description	API Endpoints	Key Props/State
Activity Timeline	Visualize/edit sessions.	`GET /sessions`, `PATCH /sessions/{id}`	`filteredSessions`, `selectedDate`
Export Panel	Generate reports.	`GET /sessions/export/csv/pdf`	`exportFormat`, `exportProgress`
Filters	Filter by date/tag.	`GET /sessions`	`activeTags`, `dateRange`

Error Handling

Table 10. Error Handling for Session History Tab

Error	Response
`OVERLAPPING_SESSION`	409 Conflict with conflicting session ID.
`EXPORT_LIMIT`	429 Too Many Requests (max 30 days).

My Work

Daily Log

Objective: Enable members to manage their daily time entries with:

- Visual timeline of tracked hours
- Drag-and-resize adjustments
- Calendar event synchronization
- Validation to prevent conflicts

Table 11. API Endpoints

Endpoint	Method	Purpose	Parameters
GET /api/time-entries	GET	Fetch daily entries	?date=YYYY-MM-DD
POST /api/time-entries	POST	Create new entry	{startTime, endTime, tag}
PUT /api/time-entries/{id}	PUT	Update entry	{startTime, endTime}
DELETE /api/time-entries/{id}	DELETE	Remove entry	-
GET /api/calendar-events	GET	Sync external calendar	?source=google/outlook

Request/Response Examples

Fetch Entries:

```
GET /api/time-entries?date=2023-11-20
Authorization: Bearer <JWT>

Response:
[{
  "entryId": "65b...",
  "startTime": "2023-11-20T09:00:00Z",
  "endTime": "2023-11-20T11:30:00Z",
  "tag": "Development",
  "linkedTask": "PROJX-123" // From Jira/Asana
}]
```

Sync Calendar:

```
GET /api/calendar-events?source=google
Authorization: Bearer <JWT>

Response:
[{
  "title": "Sprint Planning",
  "start": "2023-11-20T14:00:00Z",
  "end": "2023-11-20T15:30:00Z"
}]
```

Data Model

TimeEntry Schema:

```
{
  userId: { type: ObjectId, required: true },
  startTime: { type: Date, required: true },
  endTime: { type: Date, required: true },
  tag: {
    auto: String,
    manual: String
  },
  linkedTask: String, // External task ID
  calendarEventId: String, // For sync
  source: { type: String, enum: ['manual', 'calendar', 'auto']
}
}
```

Validation Rules:

1. startTime < endTime
2. No overlapping entries for the same user
3. Calendar events can't be modified directly

Testing Plan

Table 12. API Tests

Test Case	Validation
Create Entry	Returns 201 with accurate duration
Overlapping Entries	Returns 409 Conflict
Calendar Sync	Matches Google/Outlook events

Edge Cases

- Midnight-spanning entries (e.g., 23:30-00:45)
- Sub-minute adjustments (e.g., 09:00 → 09:01)

Frontend Integration

1. Header Controls

1.1 Date Picker

- **Purpose**: Select the "day" to view/edit.
- **UI**: Compact calendar icon + date (e.g. Nov 20, 2023) in a dropdown.
- **Behavior**:
    - Click opens month grid.
    - Selecting a date reloads the timeline via GET /api/time-entries?date=….
- **Props/State**:
    - selectedDate: Date
    - onDateChange(date: Date)

1.2 Sync Button

- **Purpose**: Pull in external calendar events (Google/Outlook)
- **UI**: Button with sync icon ("⟳ Sync Calendar").
- **Behavior**:
    - Disabled while syncing.
    - On click: calls GET /api/calendar-events?source={currentSource}, shows spinner.
    - On success: merges returned events into timeline, marking them "synced."
- **Props/State**:
    - isSyncing: boolean
    - onSync(): Promise

2. Daily Timeline

2.1 DailyTimeline Component

- **Purpose**: Visualize each time-entry block across the 24-hour day.
- **Props**:
    - entries: Array<{ entryId, startTime, endTime, tag, source }>
    - colors: Record<string, string> (map tags to hex colors)
    - Callbacks:
        - onResize(id, newStart, newEnd)

        - onDrag(id, newStart)
        - onBlockClick(id)

- **Layout**:
    - Left margin: hourly labels (00:00, 01:00, … 23:00).
    - Main grid: each hour is equal width; entries drawn as bars.
    - Bars:
        - **Manual** entries: solid fill.
        - **Synced** entries (calendar): semi-transparent with dashed outline.
- **Interactions**:
    - **Drag handles** at ends to stretch/shrink (snaps to nearest 5-min).
    - **Drag entire block** to new time.
    - **Click** opens the **Entry Editor**.
- **Validation**: Prevents overlap by visually "bumping" adjacent blocks and disallowing invalid drags.

3. Entry Editor

3.1 Inline or Modal Editor

- **Purpose**: Fine-tune a time entry's details.
- **UI Elements**:
    - **Time Inputs**: Start & End (<TimePicker> controls).

- **Tag Selector**: <Autocomplete> with existing tags + free text.
- **Linked Task**: <Autocomplete> for project management IDs.
- **Source Badge**: “Manual” / “Calendar” read-only.
- **Save** and **Delete** buttons.

- **Behavior**:
  - **Save** disabled until changes pass validation (no overlaps, start<end).
  - **Delete** shows confirmation (“Remove this entry?”).
  - On **Save**, calls PUT /api/time-entries/{id}; updates timeline.
  - On **Delete**, calls DELETE /api/time-entries/{id}; removes block.

4. Empty-Space “Add Entry” Control

4.1 Quick Add Button

- **Purpose**: Insert a new entry at any time slot.
- **UI**: A “+” icon that appears when hovering over empty timeline space.
- **Behavior**:
  - Clicking opens a minimal popup with default start/end (e.g. 30-minute block).
  - User adjusts times and tag; then “Create” triggers POST /api/time-entries.
- **Props/State**:
  - onAddClick(time: Date)

5. Footer Actions

5.1 Save Draft

- **Purpose**: Persist any unsaved manual edits locally (before bulk submission).
- **UI**: Secondary button "Save Draft."
- **Behavior**: Stores current timeline state in local storage.

5.2 Export Day (Optional)

- **Purpose**: Produce a CSV or PDF for this day.
- **UI**: Dropdown button labeled "Export" with options **CSV** and **PDF**.
- **Behavior**:
    - Selecting CSV calls GET /api/time-entries/export/csv?date=….
    - Selecting PDF calls GET /api/time-entries/export/pdf?date=….
    - Shows download progress; disables until complete.

6. Validation & Error Display

6.1 Inline Error Messages

- **Placement**: Under time inputs in editor.
- **Examples**:
    - "End time must be after start time."
    - "This entry overlaps with another."

6.2 Global Toasts

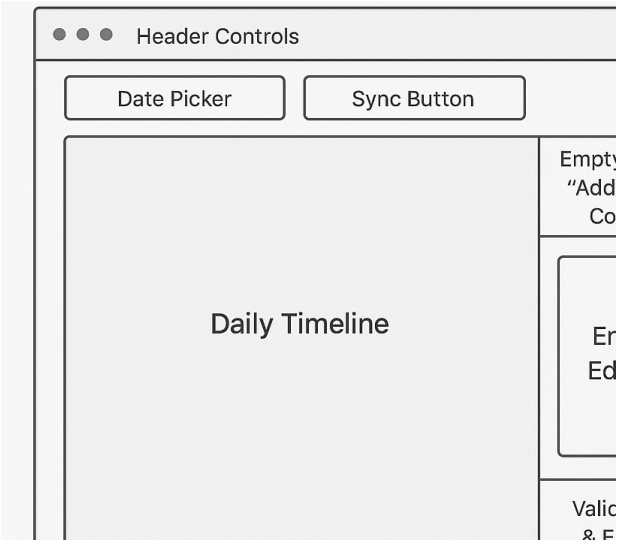
- **Purpose**: Surface save/delete errors (network or conflict).
- **UI**: Small notification at bottom-right, auto-dismiss after 3s.

Table 13. Component-API Mapping

Component	API Endpoints	Key State / Props
Header Controls		
└ Date Picker	GET /api/time-entries?date=...	selectedDate, onDateChange(date)
└ Sync Button	GET /api/calendar-events?source={source}	isSyncing, onSync()
Daily Timeline	(Implicit via GET /api/time-entries)	entries, colors
Quick Add Control	POST /api/time-entries	onAddClick(time)
Footer Actions		
└ Save Draft	(LocalStorage, no API)	-
└ Export Day	GET /api/time-entries/export/csv?date=... GET /api/time-entries/export/pdf?date=...	Export progress state
Validation & Errors	(Handled locally or from failed API calls)	validationErrors, toast state

Visual Layout

Figure 7. Visual Layout



Security & Validation

Access Rules

```
// Middleware
router.use('/time-entries', verifyOwnership);
```

Input Validation

```
{
  "$jsonSchema": {
    "properties": {
      "startTime": { "bsonType": "date" },
      "endTime": {
        "bsonType": "date",
        "minimum": { "$expression": { "$gt": "$startTime" } }
      }
    }
  }
}
```

Error Handling

Common Errors

```
{
  "error": "TIME_TRAVEL",
  "code": 400,
  "message": "Cannot create entries in the future"
}

{
  "error": "CALENDAR_READ_FAILED",
  "code": 503,
  "message": "Failed to sync with Google Calendar"
}
```

Performance Optimization

Indexing

```
db.timeEntries.createIndex({
  userId: 1,
  startTime: -1
});
```

Caching

- Calendar events cached for 15 minutes
- Frequent date queries use Redis

Weekly Summary

Objective: Provide users with a consolidated view of their weekly tracked time, highlighting under-tracked/overworked days and enabling compliance-ready report generation.

Key Features:

- **Date Validation**: Ensures startDate is always a Monday, and endDate is the following Sunday.
- **Dynamic Aggregation**: Pulls data from Daily Log entries to calculate totals and averages.
- **Flagging Logic**: Days with <4h tracked are flagged as "under-tracked"; days with >8h are "overworked".

Backend Development

Table 14. API Endpoints

Endpoint	Method	Purpose
GET /api/weekly-summary	GET	Fetch aggregated weekly data (total hours, daily averages, flagged days).
GET /api/weekly-summary/export	GET	Generate CSV/PDF reports for the selected week.

Request/Response Examples:

Fetch Weekly Data:

```
GET /api/weekly-summary?start=2023-11-20&end=2023-11-26
Authorization: Bearer <JWT>
```

Response:

```
{
  "weekStart": "2023-11-20",
  "weekEnd": "2023-11-26",
  "totalHours": 38.5,
  "dailyAverages": {
    "productive": 6.2,
    "distracting": 0.8
  },
  "flaggedDays": [
    {
      "date": "2023-11-22",
      "status": "under-tracked",
      "trackedHours": 3.1
    },
    {
      "date": "2023-11-24",
      "status": "overworked",
      "trackedHours": 9.8
    }
  ]
}
```

Export Weekly Report:

```
GET /api/weekly-summary/export/csv?start=2023-11-20&end=2023-11-26
Authorization: Bearer <JWT>
```

Response:

- Triggers a CSV/PDF download with columns: Date, Tracked Hours, Tags, Notes.

Data Model

WeeklySummary Schema (MongoDB):

```
{
  userId: { type: ObjectId, required: true },
  startDate: { type: Date, required: true },
  endDate: { type: Date, required: true },
  totalHours: { type: Number, default: 0 },
  dailyEntries: [{
    date: Date,
    trackedHours: Number,
    tags: [String],
    status: { type: String, enum: ["under-tracked", "overworked",
"normal"] }
  }],
  exports: [{
    format: { type: String, enum: ["csv", "pdf"] },
    requestedAt: Date
  }]
}
```

Validation Rules:

- startDate must be a Monday; endDate must be the following Sunday.
- trackedHours per day is derived from DailyLog entries.

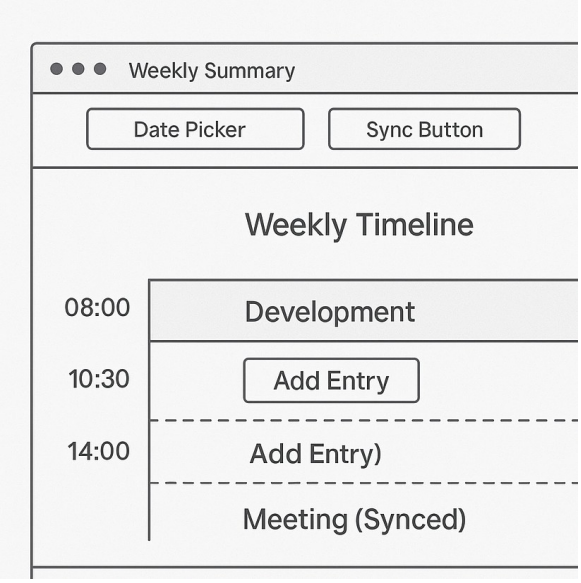
Frontend Integration

UI Components

1. **Header Controls**
    - **Date Picker**:
        - Lets users select a week (defaults to current week).

        - Updates the timeline and triggers a GET /api/weekly-summary call.
    - **Sync Button**:
        - Fetches calendar events from Google/Outlook via GET /api/calendar-events.
        - Disabled during sync; shows a spinner.
2. **Weekly Timeline**
    - **Visualization**:
        - Hourly grid (00:00–23:00) with color-coded bars for entries.
        - **Manual Entries**: Solid bars (e.g., [redacted] Development).
        - **Synced Entries**: Dashed bars (e.g., -----[redacted] Meeting (Synced)).
    - **Interaction**:
        - Drag handles to resize entries (snaps to 5-minute increments).
        - Click an entry to open the **Entry Editor**.
3. **Footer Actions**
    - **Save Draft**: Stores unsaved changes in localStorage.
    - **Export Day**: Generates CSV/PDF reports via GET /api/time-entries/export.
4. **Tooltips**: Hover over progress bars to see productivity score breakdown (e.g., "90%: 6h coding in VS Code").

Visual Layout



Breakdown:

- **Header**: Contains the week picker and sync button.
- **Main Area**:
    - **Timeline**: Shows entries as horizontal bars. Empty slots display an **Add Entry** button.
    - **Synced Entries**: Highlighted with dashed borders and "(Synced)" labels.
- **Footer**: Actions to save drafts or export data.
- **Tooltips:** Hover over progress bars to see productivity score breakdown (e.g., "90%: 6h coding in VS Code").

Table 15. Component-API Mapping

Component	API Endpoint	Purpose
Date Picker	`GET /api/weekly-summary`	Fetch data for the selected week.
Sync Button	`GET /api/calendar-events`	Pull external calendar events.
Timeline Entries	`GET /api/time-entries`	Load daily entries for the week.
Export Day Button	`GET /api/time-entries/export/*`	Generate CSV/PDF reports.

Testing Strategy

Table 16. Test Case

Scenario	Validation
Week spanning two months	Ensure data aggregation handles month transitions.
Export with no tracked hours	Return empty CSV/PDF with headers.
Overworked threshold (>8h/day)	Verify days with >8h are flagged.

Edge Cases

- User edits a Daily Log entry after generating a weekly report (revalidate data).
- Timezone mismatches between client and server.

Security & Permissions

- **Role-Based Access**: Only the entry owner can edit/delete.

- **Data Encryption**: Calendar sync tokens stored encrypted (AES-256).
- **Input Sanitization**: Reject future-dated entries.

Performance Optimization

- **Caching**: Frequent date queries cached in Redis (TTL: 15 mins).
- **Lazy Loading**: Entries load in chunks for weeks with >100 entries.
- **Indexes**:

```
db.timeEntries.createIndex({ userId: 1, startTime: -1 });
```

Add Time

Objective: Enable manual time entry with AI-driven suggestions from calendar events and smart defaults, while ensuring validation to prevent conflicts.

Key Features:

- **AI Suggestions**: Matches calendar event titles to projects/tags (e.g., "Sprint Planning" → "Development").
- **Conflict Prevention**: Blocks overlapping/future-dated entries.

Backend Development

Table 17. API Endpoints

Endpoint	Method	Purpose
POST /api/timesheets/manual	POST	Submit manual time entries.
GET /api/calendar-events	GET	Fetch calendar events for AI suggestions.

Request/Response Examples

Submit Manual Entry:

```
POST /api/timesheets/manual
Authorization: Bearer <JWT>
{
  "startTime": "2023-11-20T09:00:00Z",
  "endTime": "2023-11-20T11:00:00Z",
  "project": "ProjectX",
  "tags": ["Development"]
}
```

Response:

```
{
  "entryId": "65b...",
  "conflicts": [],
  "suggestions": ["ProjectX", "Client Meeting"]
}
```

Fetch Calendar Events:

```
GET /api/calendar-events?date=2023-11-20&provider=google
Authorization: Bearer <JWT>
```

Response:

```
[{
  "title": "Sprint Planning",
  "start": "2023-11-20T14:00:00Z",
  "end": "2023-11-20T15:00:00Z"
}]
```

Data Models

AddTime Schema

```
{
  userId: { type: ObjectId, required: true },
  startTime: { type: Date, required: true },
  endTime: { type: Date, required: true },
  project: String,
  tags: [String],
  source: { type: String, enum: ["manual"], default: "manual"
},
  status: { type: String, enum: ["draft", "submitted"],
default: "draft" }
}
```

Frontend Integration

I. UI Structure & Layout

The **Add Time** tab is designed as a form-based interface with the following key sections:

A. **Time Entry Form**:

- **Time Pickers**:
  - Two MUI DateTimePicker components for selecting **Start Time** and **End Time**, configured with 15-minute increments.
  - Real-time validation ensures endTime is always after startTime.
- **Project/Tag Inputs**:

        - MUI Autocomplete dropdowns for selecting projects and tags, populated from the user's historical data.
        - AI suggestions auto-fill these fields when calendar events are selected.

B. **AI Suggestions Panel**:
    - Displays calendar events fetched from integrated providers (Google/Microsoft).
    - Events are matched to projects/tags using keyword logic (e.g., "Sprint Planning" → "Development").
    - Clicking a suggestion auto-fills the form with event times and suggested project/tags.

C. **Recurring Entries Section** (Optional):
    - **Recurrence Toggle**: An MUI Switch to enable recurrence.
    - **Rule Configuration**:
        - MUI Select for frequency (daily/weekly/monthly).
        - Weekly frequency includes a multi-select for days of the week.
        - Optional end date using an MUI DatePicker.
    - **Preview Grid**: Shows a scrollable list of upcoming instances (e.g., "Every Monday at 9 AM for 5 weeks").

D. **Validation & Feedback**:

- Inline error messages for invalid times (e.g., overlapping entries, future dates).
- **Conflict Modal**: A dialog that lists overlapping entries and offers actions (delete/modify).

E. **Action Buttons**:

- **Save Draft**: Temporarily stores unsaved entries in localStorage.
- **Submit**: Persists the entry to the backend and navigates to the **Daily Log**.

II. Key User Interactions

A. **Manual Time Entry**:

- Users select start/end times and assign projects/tags.
- The form auto-suggests projects/tags based on historical data.

B. **Calendar Event Autofill**:

- Users click a calendar event from the **AI Suggestions Panel**, populating the form with event details.
- The system suggests a project based on event title keywords (e.g., "Demo" → "Client Meeting").

C. **Recurring Entries**:

- Users toggle recurrence, configure rules, and preview future instances.
- The preview grid updates dynamically as rules change.

D. **Conflict Resolution**:

- Overlapping entries trigger a modal showing conflicting sessions.
- Users can delete/modify conflicts directly from the modal.

III. Validation & Error Handling

A. **Input Validation**:

- **Time Order**: endTime must be after startTime.
- **Overlap Prevention**: Backend checks for overlapping entries; conflicts block submission.
- **Future Dates**: Entries with future timestamps are rejected.

B. **Error Display**:

- **Inline Errors**: Highlight invalid fields with MUI FormHelperText.
- **Conflict Modal**: Lists overlapping entries with timestamps and projects.

C. **Recurrence Rules**:

- Weekly entries require at least one selected day.
- Monthly entries disallow day-of-week constraints.

IV. Integration with Backend

A. **Data Flow**:

- **Fetch Calendar Events**: On form load, the frontend calls GET /api/calendar-events to populate the AI panel.
- **Submit Entry**: Manual/recurring entries are sent to POST /api/timesheets/manual or POST /api/timesheets/recurring.

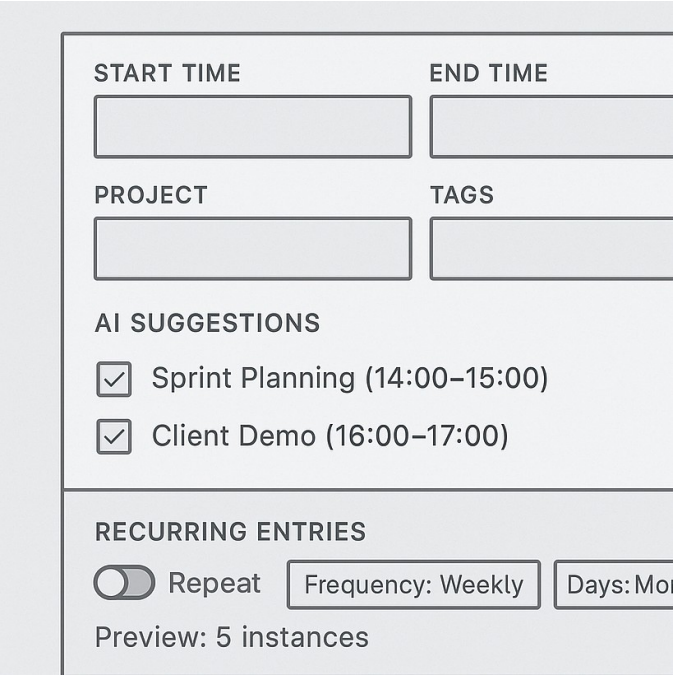
B. **AI Suggestion Logic**:

- Calendar event titles are parsed for keywords (e.g., "Code Review" → "Development").
- Default projects/tags prioritize the user's most frequent recent choices.

C. **Security**:

- OAuth2 tokens for calendar providers are encrypted (AES-256) before storage.
- Users can only access their own calendar events and historical data.

Visual Layout



Recurring Entries

Objective: Allow users to create time entries that repeat daily/weekly/monthly.

Backend Development

Table 18. API Endpoints

Endpoint	Method	Purpose
`POST /api/timesheets/recurring`	POST	Create recurring entries.
`PUT /api/timesheets/recurring/:id`	PUT	Update recurrence rules (future instances).

Request Example:

```
POST /api/timesheets/recurring
{
  "startTime": "2023-11-20T09:00:00Z",
  "endTime": "2023-11-20T11:00:00Z",
  "recurrence": {
    "frequency": "weekly",
    "daysOfWeek": ["monday", "wednesday"],
    "endDate": "2023-12-31"
  }
}
```

Data Models

RecurringEntry Schema:

```
{
  userId: { type: ObjectId, required: true },
  recurrence: {
    frequency: { type: String, enum: ["daily", "weekly", "monthly"]
},
    interval: Number,
    daysOfWeek: [String],
    endDate: Date
  },
  instances: [{ type: ObjectId, ref: 'TimeEntry' }]
}
```

Focus Dashboard

Productivity Pulse

Objective: Provide users with a personalized productivity score, historical trends, and anonymized peer benchmarking to encourage focus improvement.

Key Features:

- **Score Calculation**:
  - Formula: Score = (Productive Minutes + (Neutral Minutes * 0.5)) / Total Tracked Minutes * 100.
  - Example: 320 productive + 45 neutral minutes → (320 + 22.5) / 365 * 100 ≈ 78.5%.
- **Peer Benchmarking**:
  - Compares user scores against anonymized team averages (e.g., "Top 15%").

Backend Development

Table 19. API Endpoints

Endpoint	Method	Purpose
GET /api/productivity-score	GET	Fetch productivity metrics and peer data.
GET /api/activity/rules	GET	Retrieve app/website classification rules.

Request/Response Examples

```
GET /api/productivity-score?range=week
Authorization: Bearer <JWT>
```

Response:

```
{
  "score": 78.5,
```

```
  "productiveMinutes": 320,
  "neutralMinutes": 45,
  "totalTracked": 365,
  "trend": [
    { "date": "2023-10-01", "score": 72 },
    { "date": "2023-10-02", "score": 81 }
  ],
  "peerBenchmark": {
    "teamAverage": 65.2,
    "percentile": 85
  }
}
```

Fetch Classification Rules:

```
GET /api/activity/rules
Authorization: Bearer <JWT>
```

Response:

```
[{
  "pattern": "*.github.com",
  "type": "domain",
  "category": "productive",
  "weight": 1.0
}]
```

Data Models

ProductivityScore Schema

```
{
  userId: ObjectId,
  date: Date,        // Start of day/week/month
  productiveMinutes: Number,
  neutralMinutes: Number,
  totalTracked: Number,
  score: Number,
```

```
  peerBenchmark: {
    teamAverage: Number,
    percentile: Number
  }
}
```

ClassificationRule Schema

```
{
  pattern: String,    // e.g., "*.github.com"
  type: String,       // "domain" or "app"
  category: String,   // "productive", "neutral", "unproductive"
  weight: Number      // 0.0–1.0 (influence on score)
}
```

Frontend Integration

UI Components

1. **Score Card**:
    - Displays the current productivity score (0–100%) as a large numeric value.
    - Includes a progress bar visually representing the score (e.g., 78% → bar fills 78% of its width).
2. **Trend Chart**:
    - **React-Vis Integration**: A bar chart showing daily/weekly/monthly score trends.
    - X-axis: Dates. Y-axis: Productivity score (0–100%).

- Hover interaction reveals exact scores for specific dates.

3. **Peer Benchmarking Panel**:
    - Shows the team's average score and the user's percentile rank (e.g., "Top 15%").
    - Data is anonymized; no individual team member details are exposed.
4. **Classification Rules Transparency**:
    - A collapsible section listing how apps/websites are categorized (e.g., *.github.com = Productive).

Visual Layout

Figure 8. Outlined Position of the Components

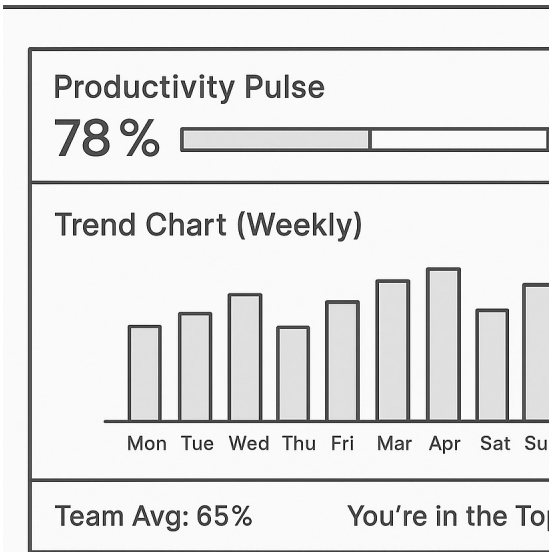


Figure 11 shows the Outlined Position of each UI component.

Table 20. Component-API Mapping

Component	API Endpoint	Purpose
Score Card	GET /api/productivity-score	Fetch the user's current productivity score, total tracked time, and peer benchmarks.
Trend Chart	GET /api/productivity-score	Retrieve historical productivity scores for rendering trends (daily/weekly/monthly).
Peer Benchmarking Panel	GET /api/productivity-score	Display anonymized team averages and percentile rankings.
Classification Rules	GET /api/activity/rules	Show transparency into how apps/websites are categorized (productive/neutral).
Real-Time Updates	WebSocket /ws/activity	Listen for live activity updates (e.g., timer start/stop) to refresh scores.

User Interactions

Breakdown of Interactions

1. **Score Card & Trend Chart**:
    - **Trigger**: On component mount or date range change (e.g., switching from "week" to "month").
    - **Data Flow**:
        - Frontend calls GET /api/productivity-score?range=week.
        - Backend returns aggregated scores and trends.
        - MUI components (Card, LinearProgress) render the data.
2. **Peer Benchmarking Panel**:
    - **Data Source**: Embedded in the GET /api/productivity-score response.
    - **UI**: MUI Typography and Chip components display "Top 15%" or similar.
3. **Classification Rules Transparency**:
    - **Trigger**: User clicks "How is my score calculated?"
    - **Data Flow**:
        - Frontend calls GET /api/activity/rules.
        - Backend returns regex-based rules (e.g., *.github.com = Productive).
        - Rules are displayed in an MUI Accordion.
4. **Real-Time Updates**:
    - **Integration**:

- WebSocket connection established on dashboard load.
- On activity_update events (e.g., new session tracked), frontend re-fetches GET /api/productivity-score.

Security & Privacy

- **Data Aggregation**: Peer benchmarking uses anonymized averages; no individual data is exposed.
- **Rule Access**: Members can view classification rules; only Super Admins can edit them.
- **Input Sanitization**: Block SQL injection in API queries (e.g., GET /activity/rules).

Testing Strategy

Table 21. Test Cases

Scenario	Validation
Full Productivity (100%)	8h tracked as "productive" → score = 100.
Zero Tracked Time	Score = 0; show "Insufficient Data" placeholder.
Rule Updates	Modified rules reflect in next score calculation.

Edge Cases

- Timezone discrepancies in trend data (e.g., UTC vs. local).

- Users with no team assignments (hide peer benchmarking).

Error Handling

1. **Frontend**:
    - Show a placeholder card if score data is unavailable.
    - Auto-retry failed WebSocket connections every 30 seconds.
2. **Backend**:
    - Return 503 Service Unavailable during rule engine maintenance.
    - Cache scores for 1 hour to reduce database load.

Break Analytics

Objective: Help users optimize work-break rhythms through data-driven recommendations and real-time nudges.

Key Features:

- **Break Detection**: Identifies breaks as gaps ≥5 minutes between tracked sessions.
- **Recommendation Engine**: Generates ideal break intervals based on user settings.

Backend Development

Table 22. API Endpoints of Break Analytics

Endpoint	Method	Purpose
GET /api/break-analytics	GET	Fetch break statistics and recommendations.
POST /api/break-settings	POST	Update user's break preferences.
GET /api/activity/status	GET	Check real-time break status for nudges.

Request/Response Examples

Fetch Break Analytics:

```
GET /api/break-analytics?range=day&tz=America/New_York
Authorization: Bearer <JWT>
```

Response:

```
{
  "actualBreaks": [
    { "start": "2023-10-01T10:00:00Z", "end": "2023-10-01T10:05:00Z" }
  ],
  "recommendedBreaks": [
    { "start": "2023-10-01T09:45:00Z", "end": "2023-10-01T09:50:00Z" }
  ],
  "compliance": 75
}
```

Update Break Settings:

```
POST /api/break-settings
Authorization: Bearer <JWT>
{
  "interval": 90,
```

```
  "duration": 5,
  "pauseType": "strict"
}
```

Data Models

Break Schema

```
{
  userId: ObjectId,
  start: Date,
  end: Date,
  type: { type: String, enum: ["actual", "recommended"] }
}
```

UserSettings Schema

```
{
  breakSettings: {
    interval: Number,  // Minutes between breaks (default: 90)
    duration: Number,  // Break length in minutes (default: 5)
    pauseType: String  // "strict" or "flexible" (default: "flexible")
  }
}
```

Frontend Integration

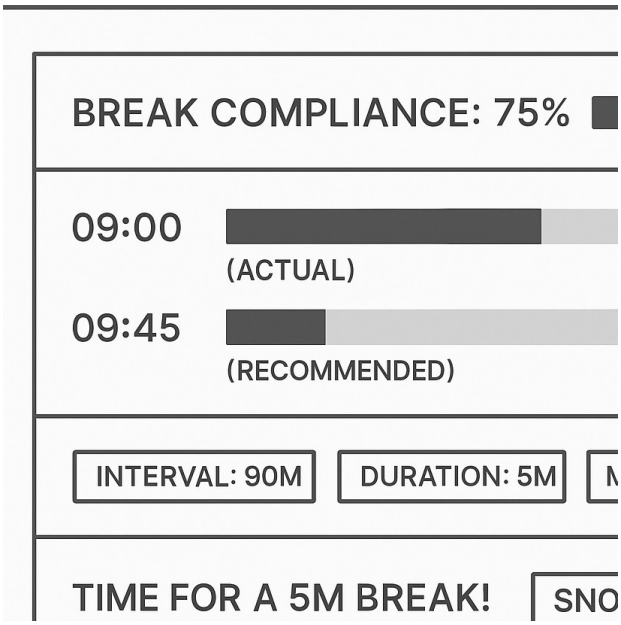
UI Components

1. **Break Timeline**:
    - **Visualization**:
        - **Actual Breaks**: Solid green bars (▓▓▓▓).
        - **Recommended Breaks**: Semi-transparent yellow bars (▒▒▒▒).

- **Interactions**: Hover to view break duration and compliance status.

2. **Settings Form**:
    - **Controls**:
        - Sliders for break interval (30–120 minutes) and duration (1–15 minutes).
        - Toggle between "strict" (blocks work during breaks) and "flexible" modes.
3. **Nudge Banner**:
    - **Triggers**:
        - Desktop agent calls GET /api/activity/status every minute.
        - Displays "Time for a break!" when status: "due".
    - **Actions**: "Snooze" button delays the nudge for 5 minutes.

Visual Layout



User Interactions

1. **Break Timeline**:
    - Users compare actual vs. recommended breaks over a day/week.
    - Compliance percentage is shown in a header badge (e.g., "75% Compliant").
2. **Real-Time Nudges**:
    - WebSocket listeners trigger the nudge banner when breaks are due.
    - In "strict" mode, work apps are temporarily blocked during breaks.

Table 23. Component-API Mapping of Break Analytics

Component	API Endpoint	Purpose
Break Timeline	GET /api/break-analytics	Fetch actual/recommended breaks for visualization.
Settings Form	POST /api/break-settings	Update user's break interval, duration, and pause type.
Nudge Banner	GET /api/activity/status	Check real-time break status and trigger nudges.

My Submissions

Timesheets

Objective: Implement a streamlined timesheet submission system with bulk submission capabilities, real-time approval tracking, and conflict resolution for developers.

Key Features:

- **Batching**: Group up to 100 entries per submission.
- **Optimistic Locking**: Version checks prevent stale updates.
- **Audit Trail**: Logs all status changes with user/timestamp.

Core Workflow

1. **Submission**:
    - Users select time entries → group by project → submit via POST /submissions.

2. **Approval**:
    - Managers review submissions → approve/reject via PATCH /submissions/{id}.
3. **Tracking**:
    - Real-time status updates via WebSocket (e.g., "Under Review" → "Approved").

Backend Development

Table 24. API Endpoints of Timesheets Tab

Endpoint	Method	Purpose
POST /submissions	POST	Submit batch entries (grouped by project).
PATCH /submissions/{id}	PATCH	Approve/reject submissions with comments.
GET /submissions/status	GET	Track progress (pending/approved/rejected).

Data Model

Submission Schema

```
{
  user: ObjectId,
  entries: [{
    entryId: ObjectId,  // Linked to TimeEntry
    project: String,
    hours: Number
  }],
  status: {
    type: String,
    enum: ['draft', 'submitted', 'under_review', 'approved', 'rejected'],
    default: 'draft'
  },
```

```
  approvalChain: [{
    manager: ObjectId,
    decision: String,    // "approve" or "reject"
    timestamp: Date
  }],
  version: Number        // For conflict resolution
}
```

Frontend Integration

UI Components

1. **Submission Timeline**:
    - **Visual Pipeline**: Color-coded stages (gray = draft, blue = under review, green = approved, red = rejected).
    - **Tooltips**: Show manager comments on hover.
2. **Bulk Select Interface**:
    - **Grouping**: Toggle between project-based or chronological views.
    - **Validation Warnings**: Highlight overlaps or missing project tags.
3. **Badge Counter**:
    - **Real-Time Updates**: Animated badge showing pending submissions (e.g., "3 Pending").
    - **Click Action**: Filters submissions by status.

Visual Layout

Figure 9. UI Component Layout Outline

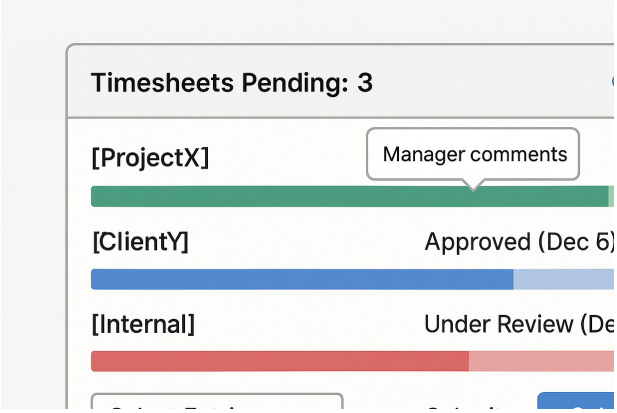


Table 25. Component-API Mapping of Timesheets tab

Component	API Endpoint	Purpose
Bulk Select Interface	POST /submissions	Submit batched entries.
Submission Timeline	GET /submissions/status	Fetch and display submission states.
Badge Counter	WebSocket /submissions	Real-time status updates.

Key Technical Considerations

1. **Conflict Resolution**:
    - **Optimistic Locking**: Reject updates if version doesn't match.
    - **Auto-Merge**: Apply non-conflicting draft changes automatically.
2. **Offline Support**:
    - Drafts saved to IndexedDB (localForage).
    - Sync queue resumes when online.
3. **Real-Time Updates**:

- WebSocket events (status_update, new_comment) refresh UI instantly.

Security & Validation

- **Access Control**:
  - Users can only modify their own submissions.
  - Managers only view submissions from their team.
- **Input Rules**:
  - Reject entries with future dates.
  - Max 100 entries per batch.

Performance Optimization

- **Bulk Operations**: MongoDB bulkWrite for batch updates.
- **Caching**:
  - Frequent GET /submissions/status queries cached for 5 minutes.
  - Stale-while-revalidate for manager views.

Time Off Request

Objective: Enable employees to request and track time off with policy visibility, balance tracking, and streamlined approval workflows.

Key Features:

- **Policy Engine**:

- Rules stored in PostgreSQL (e.g., "3-day notice required").
- Accrual formula: Available Days = (Monthly Rate × Months Employed) - Used Days.

- **Balance Calculator**:
  - Nightly cron job updates balances using employment dates and PTO history.

Backend Development

Table 26. API Endpoints of Time Off tab

Endpoint	Method	Purpose
GET /time-off/policies	GET	Fetch PTO rules (accrual rates, blackout dates).
GET /time-off/balance	GET	Calculate available days based on tenure and usage.
POST /time-off	POST	Submit time-off requests with dates and type.
PATCH /time-off/:id	PATCH	Approve/reject requests (manager action).

Data Models

TimeOffRequest Schema

```
{
  userId: ObjectId,
  dates: [Date],          // Requested dates
  type: String,           // "Vacation", "Sick", "Emergency"
  status: String,         // "submitted", "approved", "rejected"
  policyId: ObjectId,     // Linked to PTO rules
  balanceUsed: Number,    // Days deducted
  approvalChain: [{
    manager: ObjectId,
    decision: String,     // "approve" or "reject"
```

```
    comment: String
  }]
}
```

Policy Schema

```
{
  accrualRate: Number,     // Days per month
  maxDays: Number,         // Annual cap
  blackoutDates: [Date],   // Company-wide restricted dates
  noticePeriod: Number     // Minimum advance notice (days)
}
```

Frontend Integration

UI Components

1. **Time Off Request Interface**:
    - **Calendar Picker**:
        - Blocks dates with existing PTO requests or company holidays (fetched from holidays table).
        - Supports range selection (e.g., July 1–5).
    - **Policy Preview**:
        - Overlays rules from GET /time-off/policies (e.g., "Max 10 days per request").
        - Highlights conflicts (e.g., insufficient notice).
    - **Balance Card**:

        - Displays accrued/used/remaining days (e.g., "Accrued: 15, Used: 5, Remaining: 10").
        - Warns if the request exceeds the balance.

2. **Approval Tracking**:
    - **Status Badges**:
        - Color-coded badges: Submitted (), Approved (), Rejected ().
        - Real-time updates via WebSocket (/notifications channel).
    - **History Timeline**:
        - Shows past requests with dates, types, and decisions.
        - Clickable entries expand to show manager comments.

Visual Layout

Figure 10. UI Component Layout Outline of Time Off tab

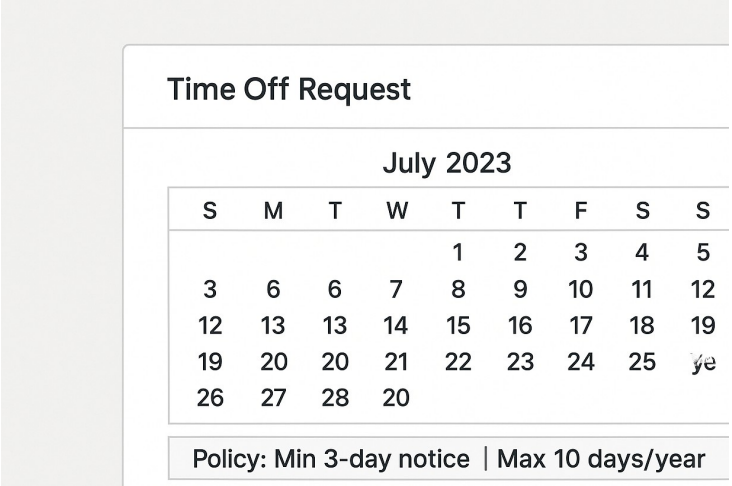


Table 27. Component-API Mapping of Time Off tab

Component	API Endpoint	Purpose
Calendar Picker	`GET /time-off/policies`	Fetch blackout dates and policy rules.
Balance Card	`GET /time-off/balance`	Display accrued/remaining days.
Submit Request	`POST /time-off`	Create a new time-off request.
Status Badges	WebSocket `/notifications`	Real-time approval updates.

Key User Journeys

1. **Standard Request**:
    - User selects dates → system checks balance/policies → submits → manager approves → balance updates.
2. **Emergency Leave**:
    - Bypasses notice period → auto-approves if <3 days → notifies team via email.

Security & Validation

- **Data Privacy**: Sensitive notes (e.g., medical leave) encrypted with AES-256.
- **Role Access**: Managers only see requests from their team.
- **Conflict Checks**: Blocks overlapping requests and blackout dates.

Edge Case Handling

Table 28. Edge Case Handling

Scenario	Resolution

Scenario	Resolution
Retroactive Edit	Requires admin approval.
Balance Overdraft	Partial approval allowed (e.g., 3 of 5 days).
Policy Conflict	Flags for HR review.
Manager Absence	Escalates to backup approver.

Performance Optimization

- **Caching**: Redis caches policy rules (15-minute TTL).
- **Batching**: Nightly balance recalculations for all users.
- **Indexing**: PostgreSQL indexes on user_id and dates.

Cross-Module Dependencies

- **User Directory**: Validates manager-team relationships.
- **Notifications**: Sends WebSocket/email alerts for status changes.
- **Activity Logs**: Tracks approval/rejection history in MongoDB.

Drafts

Objective: Enable users to save, recover, and manage incomplete submissions across sessions, with seamless offline/online transitions and conflict resolution.

Key Features:

- **Auto-Save Engine**:
    - **Local Persistence**: Drafts saved to IndexedDB every 15 minutes.

    - **Cloud Sync**: Background upload on connectivity restore; resolves conflicts via "last edit wins" with version tracking.
- **Validation**: Pre-submission checks for data integrity (e.g., required fields, valid dates).

Implementation Guidance

1. **Reuse Patterns**:
    - Offline logic from **Time Tracker** sessions.
    - Conflict resolution from **Daily Log** merging.
    - WebSocket updates from **Timesheet Approvals**.
2. **Development Sequence**:
    - Implement IndexedDB schema for local drafts.
    - Build auto-save lifecycle hooks in React.
    - Add draft recovery middleware for cloud sync.

Backend Development

Table 29. API Endpoints of Draft tab

API Endpoints

Endpoint	Method	Purpose
`GET /drafts`	GET	List all user drafts (local and cloud).

Endpoint	Method	Purpose
`PUT /drafts/:id`	PUT	Update a draft's cloud copy.
`POST /drafts/sync`	POST	Force manual sync of local drafts to cloud.

Data Models

Draft Schema

```
{
  userId: ObjectId,
  type: String,           // "Timesheet", "TimeOff", etc.
  data: Object,           // Submission-specific payload
  metadata: {
    lastSaved: Date,
    version: Number,      // For conflict resolution
    size: Number,         // Draft size in KB
    deviceId: String      // Origin device for local drafts
  },
  cloudCopy: Object       // Optional cloud backup
}
```

Frontend Integration

UI Components

1. **Draft List Panel**:
    - **Sync Status**: Icons indicate draft state (✓ Synced, ⚠ Local Only).
    - **Progress Bars**: Visualize completion for multi-step submissions (e.g., 75% for partially filled timesheets).

- **Filters/Sort**: Group by type (Timesheet/Time Off) or sort by last modified.

2. **Auto-Save Indicator**:
    - **Status Messages**: Animated "Saving..." → "Saved" → "Offline" based on connectivity.
    - **Navigation Warnings**: Alerts users about unsaved changes during page transitions.
3. **Recovery Modal**:
    - **Version Comparison**: Side-by-side diff for conflicting drafts (local vs. cloud).
    - **Actions**: "Restore from Cloud" or "Keep Local Version".

Visual Layout

Figure 11. UI Component Layout Outline of Drafts tab

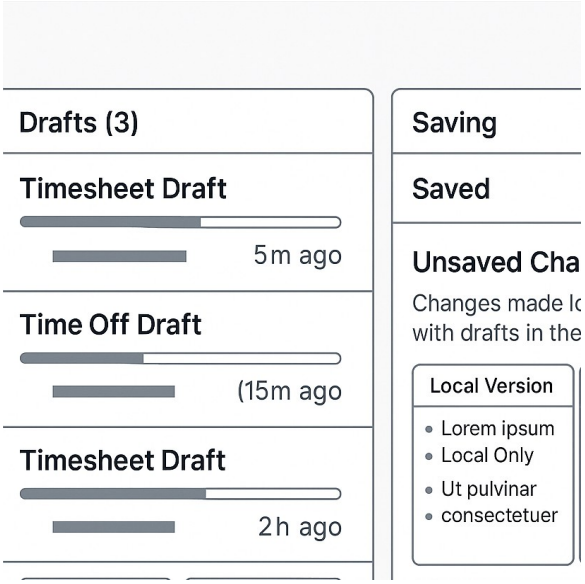


Table 30. Component – API Mapping of Drafts tab

Component	API Endpoint	Purpose
Draft List Panel	`GET /drafts`	Fetch and display local/cloud drafts.
Auto-Save Indicator	`PUT /drafts/:id`	Update cloud copy on save.
Recovery Modal	`POST /drafts/sync`	Resolve conflicts and force sync.

User Workflows

Scenario: Member drafts a timesheet offline.

1. **Auto-Save**:
    - Draft saved to IndexedDB at 0%, 15%, 30% intervals.
    - Status icon shows "Offline Draft".
2. **Reconnect**:
    - System detects network → auto-uploads to MongoDB.
    - Badge updates to "Cloud Backup Completed".
3. **Resume**:
    - User selects draft from list → continues editing.

Security & Validation

- **Data Isolation**: User-specific drafts encrypted (AES-256).
- **Access Control**: Drafts retrieved via JWT-bound API calls.
- **Size Limits**: Max 5MB per draft to prevent storage bloat.

Edge Case Handling

Table 31. Edge Case Handling

Scenario	Resolution
Device Loss	Restore drafts from cloud backup.
Draft Corruption	Fallback to last stable version.
Storage Quota Exceeded	Prioritize newest/most complete drafts.
Cross-Device Edits	Lock drafts during active editing.

Performance Optimization

- **Debounced Saving**: Reduces writes during rapid input (e.g., typing).
- **Selective Sync**: Only upload changed fields to minimize bandwidth.
- **Cache Warming**: Preload frequently accessed drafts on user login.

Cross-Feature Integration

- **Timesheets/Time Off**: Reuse validation logic for draft integrity checks.
- **Notifications**: Alert users about stale drafts (>48 hours old).
- **Preferences**: Allow users to customize auto-save frequency.

Preferences

Workflow

Objective: Allow users to customize time-tracking behavior, inactivity handling, and timer presets to align with their personal work rhythms.

Key Features:

- **Timer Presets**: Pre-configured workflows (e.g., "Deep Work: 90m, auto-tag=Development").
- **Idle Rules**: Auto-pause thresholds, calendar-aware exceptions, and app whitelists.

Implementation Guidance

1. **Reuse Patterns**:
    - **Settings UI**: Adopt styles from Privacy Guard components.
    - **Timer Logic**: Integrate preset countdowns from Live Session.
    - **Calendar Sync**: Leverage existing OAuth2 flows from Add Time.
2. **Development Sequence**:
    - Build preset editor form with MUI components.
    - Implement idle detection hooks for desktop agent.
    - Add version stamps to resolve multi-device conflicts.

Backend Development

Table 32. API Endpoints of Workflow tab

Endpoint	Method	Purpose
`GET /user/settings`	GET	Fetch current workflow rules (timer presets, idle settings).
`PATCH /user/settings/workflow`	PATCH	Update timer presets or idle detection rules.

Data Models

User Settings Schema Additions

```
{
  workflow: {
    timerPresets: [{
      name: String,
      duration: Number,
      autoTag: String,
      projectLink: String
    }],
    idleRules: {
      threshold: Number,        // 15, 30, or 60 minutes
      ignoreDuringMeetings: Boolean,
      whitelistedApps: [String] // e.g., "VS Code", "Figma"
    }
  }
}
```

Frontend Integration

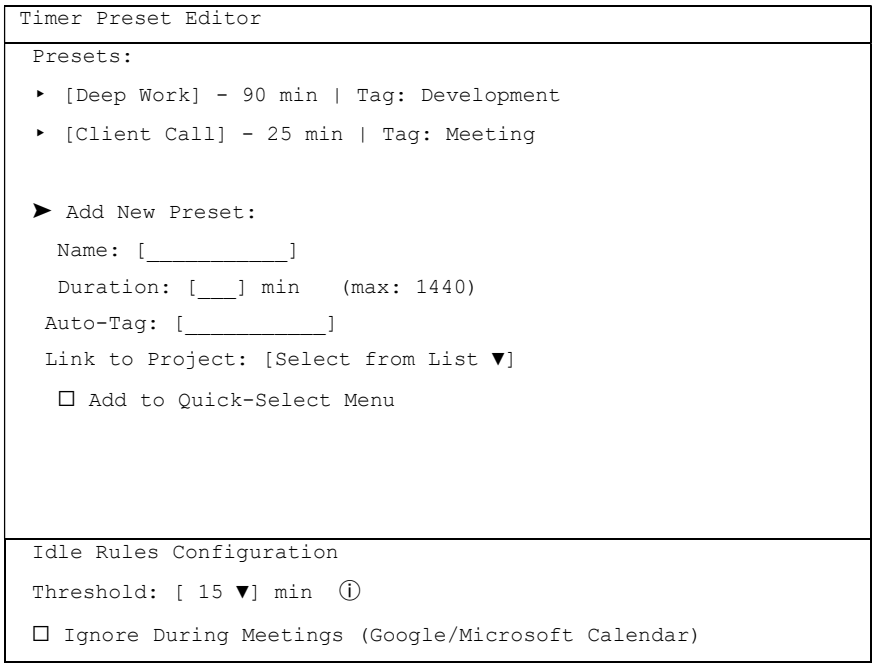
1. **Timer Preset Editor**:
    - **Form Fields**: Name, duration (minutes), auto-tag, optional project link.
    - **Quick-Select Menu**: Presets appear in the **Live Session** tab's dropdown.
2. **Idle Rules Configuration**:
    - **Threshold Slider**: 15/30/60-minute auto-pause triggers.
    - **Context Toggles**:

- "Ignore during Meetings" (syncs with Google/Microsoft Calendar).
- Whitelist apps that bypass idle detection (e.g., VS Code in fullscreen).

3. **Save & Sync Status**:
    - **Indicator**: Shows "Saved" or "Syncing..." based on desktop agent status.
    - **Validation**: Blocks invalid inputs (e.g., durations >24h).

Visual Layout

Figure 12. UI Component Layout Outline of Workflow tab



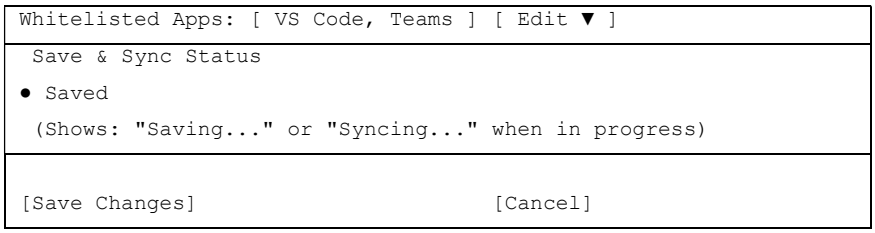


Table 33. Component-API Mapping of Workflow

Component	API Endpoint	Purpose
Timer Preset Editor	PATCH /user/settings/workflow	Save/update presets.
Idle Rules Panel	PATCH /user/settings/workflow	Update auto-pause thresholds and whitelists.
Settings Sync Status	GET /user/settings	Fetch current rules for display.

Key User Workflows

1. Create Timer Preset:
    - User navigates to Workflow → Timer Presets → Defines name/duration → Saves.
    - Preset becomes available in the Live Session tab.
2. Configure Idle Detection:
    - Set auto-pause threshold to 30 minutes → enable "Ignore during Meetings" → whitelist "Teams".

Security & Validation

- **Encryption**: User settings stored with AES-256 encryption.

- **Rate Limiting**: Max 10 preset updates/hour to prevent abuse.
- **Access Control**: Users can only modify their own settings.

Table 34. Edge Case Handling

Scenario	Resolution
Preset Edited Mid-Session	Changes apply on next timer start.
Conflicting Idle Rules	Whitelisted apps override system detection.
Calendar Sync Failure	Fallback to default idle thresholds.

Performance Optimization

- **Local Cache**: Timer presets stored in IndexedDB for instant access.
- **Debounced Saves**: Batch frequent idle rule adjustments to reduce API calls.
- **Desktop Agent Priority**: Critical settings (e.g., auto-pause) sync first.

Cross-Feature Integration

- **Live Session:** Applies timer presets on launch.
- **Calendar Sync**: Checks meeting status to disable idle detection.
- **Notifications:** Alerts users when nearing idle thresholds.

Privacy Guard

Objective: Empower users to control privacy-sensitive tracking elements through app/website blocking, screenshot protection, and temporary tracking suspension.

Key Features:

- **Blocking Modes**:
    - **Full Block**: Prevents tracking entirely (e.g., WhatsApp).
    - **Partial Block**: Tracks time but hides details (e.g., app name).
- **Screenshot Protection**: Auto-blurs sensitive apps with configurable opacity.

Implementation Guidance

1. **Reuse Patterns**:
    - **Settings UI**: Adopt MUI components from Workflow Preferences.
    - **Process Monitoring**: Leverage logic from Activity Classifier.
2. **Development Sequence**:
    - Build blacklist manager with search and mode toggles.
    - Integrate GPU-accelerated blur into desktop agent.
    - Implement gap-filling prompts post-tracking suspension.

Backend Development

Table 35. API Endpoint of Privacy Guard

Endpoint	Method	Purpose
`GET /privacy/blacklist`	GET	Fetch blocked apps/websites and modes.
`PATCH /privacy/blacklist`	PATCH	Update blocking rules (full/partial).
`POST /privacy/screenshots`	POST	Configure screenshot blur/watermark rules.

Data Models

Privacy Settings Schema

```
{
  blacklist: {
    apps: [String],        // ["whatsapp.exe", "spotify.exe"]
    websites: [String],    // ["facebook.com"]
    mode: "block" | "obscure"
  },
  screenshotRules: {
    blur: "auto" | "manual",
    watermark: Boolean
  },
  trackingSuspension: {
    until: Date,           // Optional end time
    fillGaps: Boolean      // Prompt to fill gaps post-pause
  }
}
```

Frontend Integration

UI Components

1. **Blacklist Manager**:
    - **App/Website Search**: Add entries by name or domain (e.g., facebook.com).
    - **Blocking Mode Toggle**: Switch between "Full Block" and "Partial Block".
2. **Screenshot Protection Panel**:
    - **Blur Controls**: Adjust intensity (1–5) and enable auto-blur for detected apps.
    - **Watermarking**: Optional text overlay (e.g., "Confidential").

3. **Tracking Suspension**:
    - **Temporary Pause**: Toggle to stop tracking for 1 hour or until tomorrow.
    - **Stealth Mode**: Disables desktop agent indicators (no glowing borders).

Visual Layout

Figure 13. UI Component Layout Outline of Privacy Guard tab

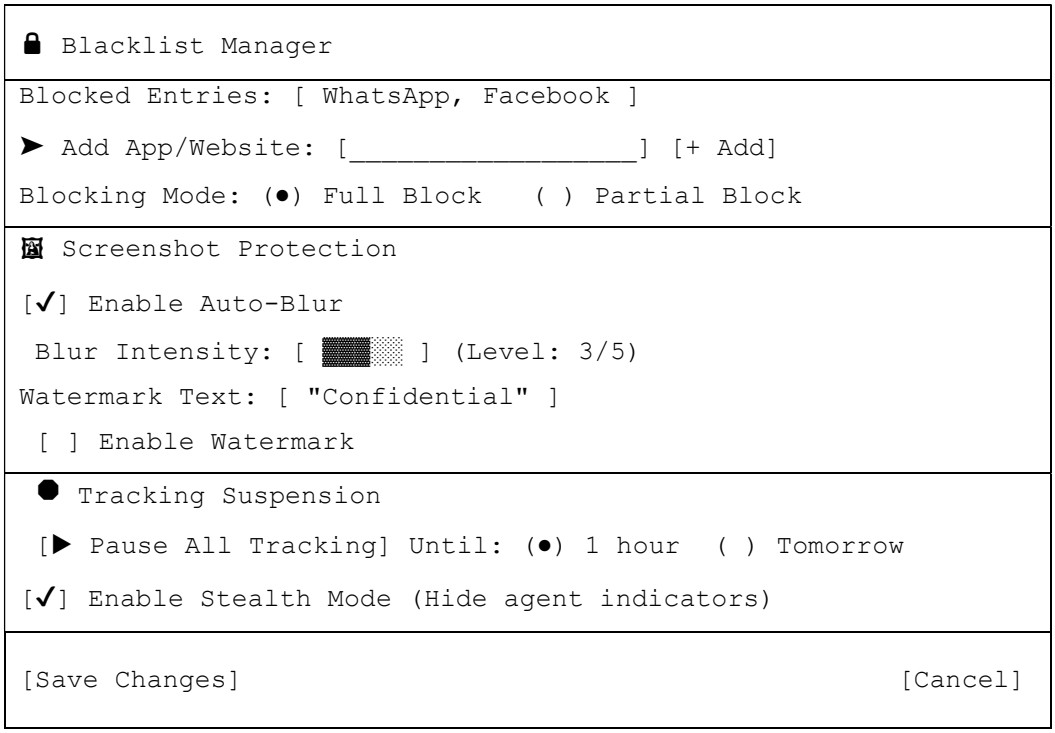


Table 36. Component-API Mapping of Privacy Guard

Component	API Endpoint	Purpose
Blacklist Manager	PATCH /privacy/blacklist	Update blocking rules.
Screenshot Protection	POST /privacy/screenshots	Configure blur/watermark settings.

Component	API Endpoint	Purpose
Tracking Suspension	`PATCH /privacy/blacklist`	Toggle tracking pauses.

User Workflows

1. **Block Personal Apps**:
    - User adds "WhatsApp" to blacklist → app is blurred in screenshots and excluded from productivity scores.
2. **Temporary Tracking Pause**:
    - Enable "Stop All Tracking" → desktop agent suspends monitoring → post-pause, prompt to log time manually.

Security & Validation

- **Encryption**: Blacklist and screenshot rules stored with AES-256.
- **Process Validation**: Desktop agent verifies app signatures to prevent spoofing.
- **Audit Trail**: Logs all privacy config changes in MongoDB.

Table 37. Edge Case Handling

Scenario	Resolution
App Name Collision	Confirm via process path/SHA hash.
Multi-Monitor Capture	Apply blur to all displays.
Rules Sync Failure	Desktop agent uses last-known configuration.

Performance Optimization

- **Local Caching**: Blacklist stored in desktop agent's SQLite for instant access.
- **Selective Monitoring**: Only track running blacklisted apps.
- **Hardware Acceleration**: GPU-accelerated blur for real-time screenshot processing.

Cross-Feature Integration

- **Session Tracking**: Excludes time spent in blacklisted apps from reports.
- **Notifications**: Alerts when stealth mode is activated.
- **Reporting**: Omits blurred screenshots in exports.

Notification

Objective: Allow users to customize notification preferences, including alert types, delivery channels, and scheduling, to stay informed without interruptions.

Key Features:

- **Priority Levels**:
  - **Urgent** (Red): Timesheet deadlines, system outages.
  - **Standard** (Yellow): Approval updates, reminders.
  - **Informational** (Gray): Weekly summaries, tips.
- **Snooze Functionality**: Temporarily mute non-urgent alerts (1h/24h/custom).

Implementation Guidance

1. **Reuse Patterns**:

- **Settings UI**: Adopt MUI components from Privacy Guard/Workflow tabs.
- **WebSocket Logic**: Leverage existing infrastructure from Timesheet approvals.

2. **Development Sequence**:
   - Build the priority matrix with drag-and-drop channel selectors.
   - Implement timezone-aware quiet hours using moment-timezone.
   - Add snooze timers with desktop agent sync.

Backend Development

Table 38. API Endpoints of Notification tab

Endpoint	Method	Purpose
GET /notifications/settings	GET	Fetch notification rules and channels.
PATCH /notifications/settings	PATCH	Update notification preferences.

Data Models

Notification Settings Schema

```
{
  userId: ObjectId,
  rules: {
    timesheets: {
      priority: "urgent" | "standard" | "off",
      channels: ["in-app", "email"]
    },
    timeOff: { ... },
```

```
    system: { ... }
  },
  snoozedUntil: Date,          // Global snooze end time
  quietHours: {
    start: "22:00",            // 10 PM
    end: "07:00"               // 7 AM
  },
  version: Number              // For conflict resolution
}
```

Frontend Integration

UI Components

1. **Priority Matrix**:
    - **Grid Layout**: Organize alerts by type (e.g., Timesheets, Time Off, System).
    - **Toggle Switches**: Enable/disable notifications per category.
    - **Channel Selectors**: Choose delivery methods (In-App, Email, Teams).
2. **Snooze Panel**:
    - **Preset Durations**: 1 hour, 24 hours, Custom.
    - **Active Snoozes**: List ongoing snoozes with countdown timers.
3. **Delivery Schedule**:
    - **Quiet Hours**: Block notifications during specified times (e.g., 10 PM–7 AM).
    - **Timezone Awareness**: Auto-adjust schedules based on user's local time.

Visual Layout

Figure 14. UI Component Layout Outline of Notification tab

Notifications

Priority Matrix

Category	Priority	Channels
Timesheets	(●) Urgent ( ) Standard ( ) Off	In-App Email Teams
Time Off	(●) Urgent ( ) Standard ( ) Off	In-App Email Teams
System	(●) Urgent ( ) Standard ( ) Off	In-App Email Teams

Snooze Panel

Preset Snooze: ( ) 1h ( ) 24h ( ) Custom ▸
Custom Until: [_____ YYYY-MM-DD HH:MM ▾]
Active Snoozes:
• Non-urgent alerts (ends in 52m)

Delivery Schedule

Quiet Hours: [22:00 ▼] ➔ [07:00 ▼] (Local Time)
☐ Auto-Adjust for Timezone Changes

ⓘ You can snooze or disable any category above without affecting others.

ⓘ Changes are saved in real-time (debounced, AES-256 encrypted).

[Save Changes] [Cancel]

Table 39. Component-API Mapping of Notification tab

Component	API Endpoint	Purpose

Component	API Endpoint	Purpose
Priority Matrix	`PATCH /notifications/settings`	Update notification rules and channels.
Snooze Panel	`PATCH /notifications/settings`	Set global/local snooze durations.
Delivery Schedule	`PATCH /notifications/settings`	Configure quiet hours and timezone rules.

User Workflows

1. **Customize Timesheet Alerts**:
    - Set deadline reminders to **Urgent** (Red) with Teams notifications.
    - Snooze non-critical alerts during focus hours.
2. **Enable Quiet Hours**:
    - Block all non-urgent notifications from 10 PM to 7 AM.

Security & Validation

- **Encryption**: Notification settings stored with AES-256.
- **Rate Limiting**: Max 10 notification updates/hour.
- **Access Control**: Users can only modify their own settings.

Table 40. Edge Case Handling

Scenario	Resolution
Timezone Mismatch	Convert quiet hours to user's local time.
Multi-Device Snooze	Sync snooze state via WebSocket.

Scenario	Resolution
Failed Delivery	Retry 3 times → fallback to in-app alert.

Performance Optimization

- **Debounced Updates**: Batch frequent preference changes (e.g., toggling switches).
- **Caching**: Frequent GET `/notifications/settings` responses cached for 5 minutes.
- **WebSocket Efficiency**: Only push updates for high-priority alerts.

Cross-Feature Integration

- **Timesheets**: Trigger deadline alerts based on submission status.
- **Time Off**: Notify managers of pending requests.
- **Activity Logs**: Audit notification delivery attempts.

MANAGER

My Team

Timesheets

Objective: Enable managers to review, approve, and analyze team timesheets with integrated attendance verification and productivity insights.

Key Features:

1. Attendance Validation

- **Auto-flag** entries where logged hours don't match physical attendance.
- **Row Color-Coding**:
    - Attendance Verified
    - Missing or mismatched attendance

2. New Filters

- **Attendance Status** (Present / Partial / Absent)
- **Compliance**: Under / Over / Matching hours vs. attendance

3. Enhanced Security

- **Block** approvals for days lacking verified attendance.

Backend Development

Table 41. API Endpoints of Timesheets tab

Endpoint	Method	Purpose

Endpoint	Method	Purpose
GET /api/team/timesheets	GET	Fetch team timesheets with optional attendance filter
PATCH /api/timesheets/{id}	PATCH	Approve/reject and auto-adjust hours to attendance
GET /api/team/productivity	GET	Load aggregated productivity & attendance metrics

Data Model

Extended Timesheet Schema:

```
// Extended Timesheet Schema
{
  teamId: ObjectId,
  entries: [{
    userId: ObjectId,
    date: Date,
    attendanceStatus: {                 // NEW
      type: String,
      enum: ["present", "partial", "absent"]
    },
    hours: {
      logged: Number,
      attendanceVerified: Number        // NEW
    },
    project: String,
    status: {                           // pending / approved / rejected
      type: String,
      enum: ["pending","approved","rejected"]
    }
  }],
  submittedAt: Date,
  approvedBy: ObjectId,
  auditLog: [{
```

```
    action: String,                    // "approve", "reject", etc.
    user: ObjectId,
    timestamp: Date,
    attendanceImpact: String           // NEW: "adjusted-due-to-attendance"
  }]
}
```

Frontend Integration

UI Components

1. **Team Timesheet Dashboard**:
    - **Grid View**
        - **Columns**:

          Member Name | Project | Logged Hrs | Status | Attendance |

          Match % | Actions
        - **Attendance** column:

          Present / Partial / Absent
        - **Match %** column:

          Present / Partial / Absent
    - **Filters**
        - Date range, project, member, status
        - Attendance group:

          [Attendance]
          ☑ Present Only
          ☑ Show Discrepancies
          ☑ Hide Verified

- **Bulk Actions**
  - Approve All / Reject All (disabled if any selected row is ● Absent)

2. **Timesheet Detail Modal**:
   - **Time Entries**: Daily breakdown with tags/notes
   - **Attendance Verification Section**
   - **Approval History**: Timeline of previous actions (e.g., "Approved by Jane Doe on 2023-11-20").
3. **Productivity Insights Panel**:
   - **Charts**: Weekly hours trend, project distribution.
   - **New Metrics:** Attendance Compliance Rate (%), Hours Logged vs. Attendance Verified
   - **Alerts**: Flags anomalies (e.g., under-logged days, overtime).

Visual Layout

Figure 15. UI Component Layout Outline of Notification tab

Timesheet (Pending: 5) [Export ▾]

Filters: Date: [2025-05-01▾]-[2025-05-07 ▾] Project: [All ▾] Member: [All ▾] Status: [All ▾] Attendance: [Present ▾]
[☑ Show Discrepancies] [☑ Hide Verified]

Member Name	Project	Hours	Status		Actions
John Doe	ProjectX	40h	Pending	Attendance	[Details][✓][✗]
Jane Smith	ClientY	35h	Approved	Present	[Details][✓][✗]
Bob Lee	Internal	42h	Rejected	Partial	[Details][✓][✗]

[Bulk Approve ✓] [Bulk Reject ✗]

Productivity Insights

Weekly Hours Trend Chart (line graph)

Project Distribution Pie

Alerts:
Jane Smith: 20h (below avg)
John Doe: 50h (overtime)

Note: Click "Details" to open modal with daily entries & approval history.

Table 42. Component-API Mapping of Time Sheet Tab

Component	API Endpoint	Purpose
Team Dashboard	GET /api/team/timesheets	Fetch and filter team timesheets.
Approval Modal	PATCH /api/timesheets/{id}	Submit approve/reject decisions.
Productivity Panel	GET /api/team/productivity	Load aggregated metrics.

Security & Validation

- **Role-Based Access**: Managers can only view/act on their team's timesheets.
- **Validation**:
    - Reject timesheets with future dates or unapproved projects.
    - Block approvals when attendanceStatus === "absent".
    - Enforce logged ≤ attendanceVerified or throw error.

Performance Optimization

- **Caching**: Frequent team queries cached for 5 minutes (Redis).
- **Pagination**: Load timesheets in chunks (e.g., 50 per page).
- **WebSocket Updates**: Real-time sync for status changes.

Testing

Table 43. Testing Strategy

Scenario	Validation
Bulk Approval	Verify all selected timesheets update status.
Conflict Handling	Detect and alert if another manager approves first.
Offline Access	Ensure pending actions queue and sync on reconnect.

Cross-Feature Integration

- **Notifications**: New alert for "Attendance mismatch in approved timesheet."

- **Activity Logs**: Include attendance columns in export feeds for payroll.
- **Reporting**: Record any attendance-driven adjustments under attendanceImpact.

Implementation Guidance

1. **Reuse Patterns**:
    - Adapt the Member's **Daily Log** UI for team views.
    - Use **WebSocket** logic from Timesheet approvals for real-time updates.
2. **Key Development Steps**:
    - Build bulk action handlers with checkboxes and confirmation modals.
    - Integrate productivity charts using React-Vis or similar.

Approvals Hub

Objective: Centralize and streamline the approval process for timesheets, time-off requests, and policy exceptions (e.g., overtime, weekend work).

Key Features:

- **Unified Queue**: Combines timesheets, time-off, and exceptions in one list.
- **Priority Sorting**: Orders requests by deadline proximity.
- **Policy Checks**: Automatically flags non-compliant entries (e.g., weekend work without justification).

Backend Development

Table 44. API Endpoints of Approvals Hub

Endpoint	Method	Purpose
`GET /api/approvals/pending`	GET	Fetch all pending requests requiring action.
`PATCH /api/approvals/{id}`	PATCH	Approve/reject a single request.
`POST /api/approvals/bulk`	POST	Process bulk approvals/rejections.

Data Model

Approval Schema:

```
{
  type: { type: String, enum: ["timesheet", "time_off", "exception"] },
  requester: ObjectId,
  details: {
    // Timesheet: { entries: [ObjectId], totalHours: Number },
    // Time Off: { dates: [Date], type: String },
    // Exception: { reason: String, policyViolated: String }
  },
  status: { type: String, enum: ["pending", "approved", "rejected"] },
  deadline: Date,
  comments: [{
    user: ObjectId,
    text: String,
    timestamp: Date
  }],
  version: Number  // For optimistic locking
}
```

Frontend Integration

UI Components

1. **Approvals Queue**:
    - **Table View**:

        - **Columns**: Request Type | Requester | Deadline | Status | Action.
        - **Filters**: Type (timesheet/time-off/exception), urgency (e.g., "Due Today").
    - **Priority Tags**: Highlight overdue requests in red.
2. **Bulk Action Toolbar**:
    - **Checkbox Selection**: Select multiple requests.
    - **Dropdown Actions**: "Approve All," "Reject All," "Add Comment to All."
3. **Request Detail Sidebar**:
    - **Contextual Data**:
        - Timesheet: Daily logs with hours and tags.
        - Time Off: Dates, type (vacation/sick), and balance impact.
        - Exception: Policy violated and justification notes.
    - **Audit Trail**: History of comments and decisions.

Visual Layout

Table 45. UI Component Layout Outline of Approvals Hub Tab

Approvals Hub (Pending:8)[Search▾][Type▾][Urgency▾] [Clear]				
☐Request Type	Requester	Deadline	Status	Action
☐ Timesheet	John Doe	Today	Pending	[View]
☐ Time Off	Jane Smith	Dec 25–30	Pending	[View]
☐ Exception	Bob Lee	Overdue	Rejected	[View]
Bulk Actions: [Approve All] [Reject All] [Add Comment] [Export CSV]				

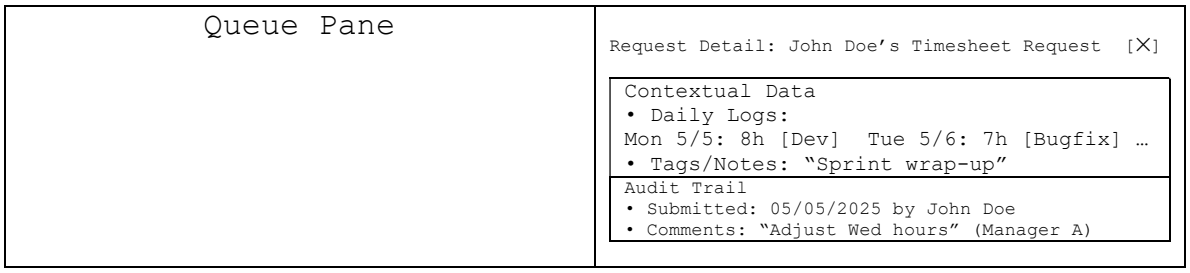


Table 46. Component-API Mapping of Approvals Hub tab

Component	API Endpoint	Purpose
Approvals Queue	GET /api/approvals/pending	Load and filter pending requests.
Bulk Action Toolbar	POST /api/approvals/bulk	Submit bulk decisions.
Request Detail Sidebar	PATCH /api/approvals/{id}	Update individual request status.

Key User Workflows

1. **Single Approval**:
    - Click a request → Review details → Approve/Reject with comment → Submit.
2. **Bulk Approval**:
    - Select 10+ requests → Add bulk comment → "Approve All."
3. **Policy Exception Handling**:
    - Flagged requests show compliance warnings → Require additional justification before approval.

Security & Validation

- **Role-Based Access**: Managers only see requests from their direct reports.
- **Validation**:
    - Prevent approving expired requests (deadline passed).
    - Ensure comments are added for rejections.
- **Audit Logs**: Immutable record of all decisions (stored in MongoDB).

Performance Optimization

- **Caching**: Frequent GET `/api/approvals/pending` responses cached for 2 minutes.
- **WebSocket Updates**: Real-time refresh when new requests arrive or status changes.
- **Lazy Loading**: Details sidebar content loads on demand.

Testing Strategy

Scenario	Validation
Bulk Approval with Comment	Verify comment applies to all selected requests.
Expired Request	Block approval and show "Deadline Passed" alert.
Real-Time Sync	New requests appear without manual refresh.

Cross-Feature Integration

- **Notifications**: Alert managers of high-priority approvals and confirmations.
- **Time Off**: Auto-update balances post-approval.

- **Reporting**: Export approval history for compliance audits.

Activity Monitor

Objective: Provide managers with real-time and historical visibility into team activities, enabling productivity analysis and compliance monitoring through interactive timelines and exports.

Key Features:

- **Real-Time Updates**: WebSocket (/ws/activity) streams live activity (e.g., app switches, idle time).
- **Granular Filters**: Date range, team member, project, or activity type (productive/neutral/non-productive).
- **Audit Logs**: Track access to sensitive activity data.

Backend Development

Table 47. API Endpoints of Activity Monitor tab

Endpoint	Method	Purpose
GET /api/team/activity	GET	Fetch activity data (real-time/historical).
GET /api/team/activity/export	GET	Export activity data as PNG/PDF.

Data Model

Activity Schema:

```
{
  userId: ObjectId,
  timestamp: Date,
  activityType: {
    type: String,
    enum: ["app", "website", "meeting", "idle"]
  },
  details: {
    appName: String,     // e.g., "VS Code"
    domain: String,      // e.g., "github.com"
    duration: Number     // Minutes
  },
  productivityTag: String  // "productive", "neutral", "non-productive"
}
```

Frontend Integration

UI Components

1. **Activity Timeline**:
    - **Color-Coded Bars**:
        - Green = Productive (e.g., coding in VS Code).
        - Yellow = Neutral (e.g., email).
        - Red = Non-productive (e.g., social media).
    - **Drill-Down**: Click a bar to view details (app used, duration, screenshots).
2. **Filters Panel**:
    - **Dynamic Filters**: Select by member, date, project, or activity type.
    - **Presets**: "Today," "Last 7 Days," "Custom Range."
3. **Export Controls**:

- **Formats**: PNG (for quick sharing) and PDF (for formal reports).
- **Customization**: Include/exclude columns (e.g., hide sensitive app names).

Visual Layout

Figure 16. UI Component Layout Outline of Activity Monitor Tab

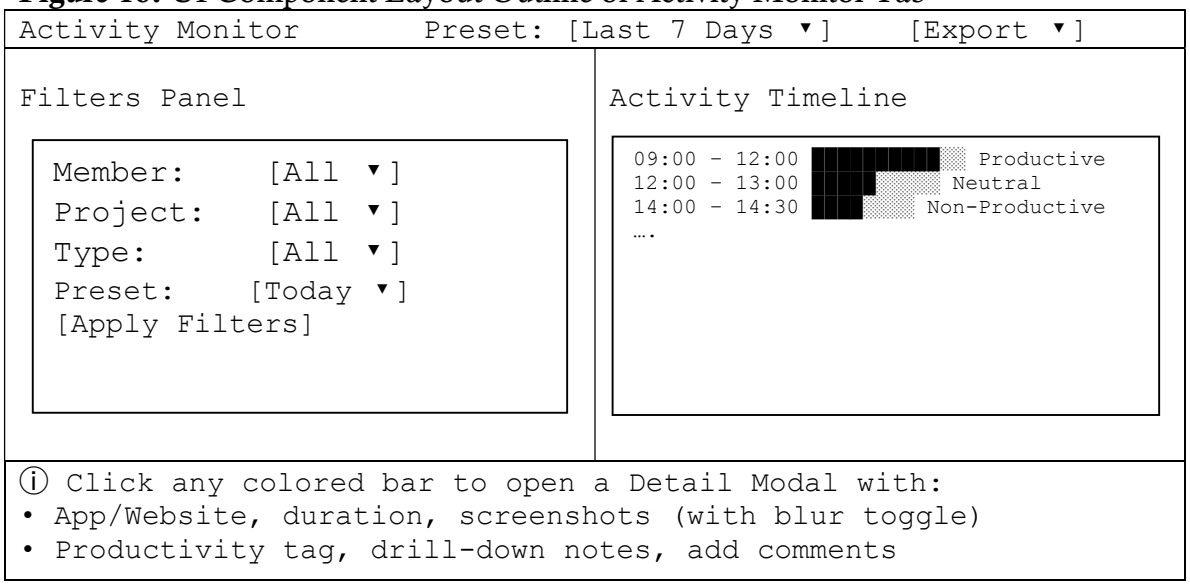


Table 48. Component-API Mapping of Activity Monitor Tab

Component	API Endpoint	Purpose
Activity Timeline	GET /api/team/activity	Load filtered activity data.
Export Controls	GET /api/team/activity/export	Generate PNG/PDF reports.
Real-Time Feed	WebSocket /ws/activity	Stream live activity updates.

Key User Workflows

1. **Real-Time Monitoring:**
    - Watch live activity streams via WebSocket (e.g., "John switched to Teams → yellow bar").
2. **Historical Analysis:**
    - Filter to last month → identify patterns (e.g., frequent idle time after lunch).
3. **Export Findings:**
    - Generate a PDF report for HR meetings → highlight compliance issues.

Security & Validation

- **Access Control**: Restrict data to the manager's team members.
- **Data Redaction**: Blur screenshots for blacklisted apps in exports.
- **Rate Limiting**: Max 10 exports/hour to prevent abuse.

Performance Optimization

- **Caching**: Cache frequent date-range queries in Redis (TTL: 15 minutes).
- **Lazy Loading**: Load timeline data in chunks (e.g., 1 week at a time).
- **Indexing**: MongoDB indexes on userId, timestamp, and productivityTag.

Testing Strategy

Table 49. Testing Strategy

Scenario	Validation
Real-Time Sync	New activities appear without refresh.
Large Dataset Export	10,000+ activities export in <30s.
Drill-Down Permissions	Block access to other teams' activity details.

Cross-Feature Integration

- **Productivity Pulse**: Feed data into team productivity scores.
- **Approvals Hub**: Flagged activities trigger policy exceptions.
- **Audit Logs**: Track manager access to sensitive activity data.

Implementation Guidance

1. **Reuse Patterns**:
    - Adapt the **Timeline UI** from the Member's Session History.
    - Use **WebSocket** logic from Timesheet approvals for live updates.
2. **Key Development Steps**:
    - Build drill-down modal with screenshot previews (blurred if needed).
    - Integrate export libraries (e.g., pdfmake for PDF, html2canvas for PNG).

Productivity

Work Distribution

Objective: Enable managers to analyze, optimize, and assign team workloads using real-time heatmaps, burnout alerts, and conflict-aware scheduling tools.

Key Features:

- **Burnout Detection**: Flags members with >6 hours/day of focused work for >5 consecutive days.
- **Conflict Checks**: Validates assignments against project deadlines, existing tasks, and time-off requests.
- **Forecast Integration**: Models project outcomes based on resource allocation changes.

Backend Development

Table 50. API endpoints

Endpoint	Method	Purpose
`GET /api/work-distribution`	GET	Fetch workload heatmaps and project allocations.
`POST /api/work-distribution`	POST	Assign/reassign tasks with conflict checks.
`GET /api/team/capacity`	GET	Retrieve team members' current capacity.

Data Model

WorkDistribution Schema:

```
{
  projectId: ObjectId,
  assignments: [{
    userId: ObjectId,
```

```
    taskId: ObjectId,
    startDate: Date,
    endDate: Date,
    hoursPerDay: Number
  }],
  capacity: {
    userId: ObjectId,
    maxHours: Number,   // Daily limit (e.g., 8h)
    currentLoad: Number // Current assigned hours/day
  },
  historicalLoad: [{
    date: Date,
    productiveHours: Number,
    meetings: Number
  }]
}
```

Frontend Integration

UI Components

1. **Workload Heatmap**:
    - **Visualization**:
        - Green = Underutilized (<4h/day).
        - Yellow = Balanced (4–6h/day).
        - Red = Overloaded (>6h/day).
    - **Interaction**: Hover to view member details (skills, current tasks).
2. **Drag-and-Drop Scheduler**:
    - **Features**:
        - Drag tasks to assign → auto-checks for conflicts (e.g., double-booking, deadline overlaps).
        - Tooltip warnings for overloads or skill mismatches.

- **Undo/Redo**: Revert accidental assignments.

3. **Burnout Alerts Panel**:
   - **Triggers**: Excessive focused time or consistent overloads.
   - **Actions**: Suggest task redistribution or time-off recommendations.

Visual Layout

Figure 17. UI Component Layout Outline of Work Distribution Tab

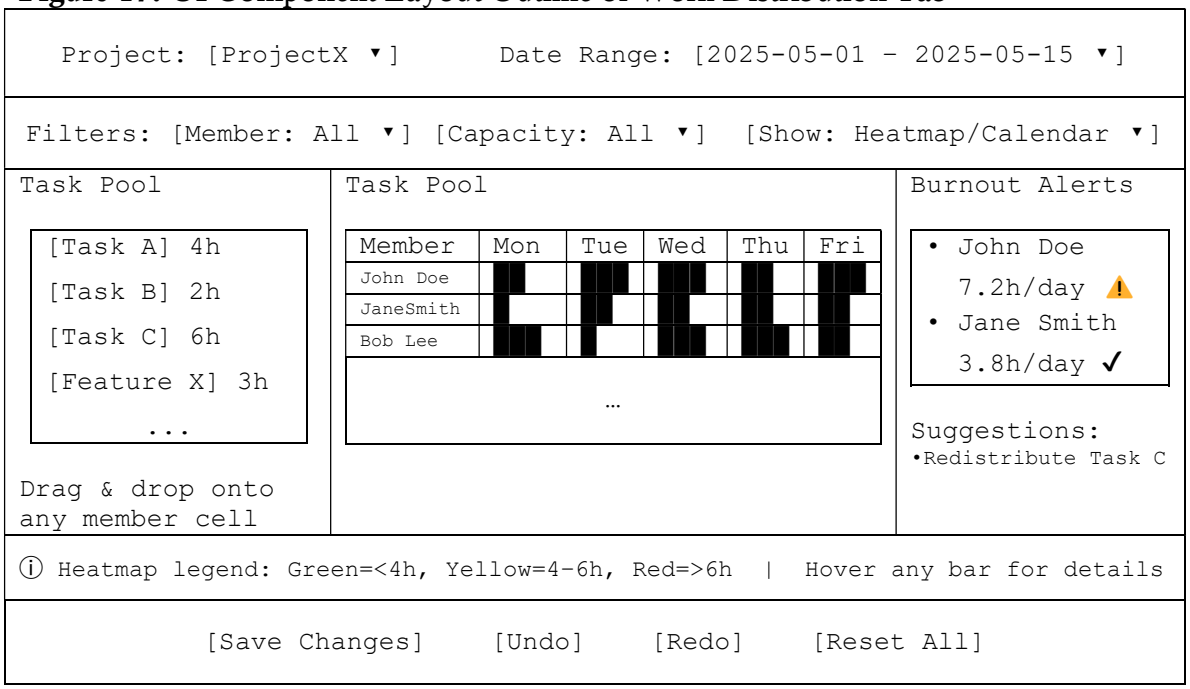


Table 51. Component-API Mapping of Work Distribution Tab

Component	API Endpoint	Purpose
Workload Heatmap	`GET /api/work-distribution`	Load heatmap and capacity data.
Drag-and-Drop Scheduler	`POST /api/work-distribution`	Submit new assignments with validation.
Burnout Alerts	`GET /api/team/capacity`	Fetch capacity/utilization metrics.

Key User Workflows

1. **Assign Tasks**:
    - Drag a task to a team member → system validates capacity → confirm assignment.
2. **Optimize Workload**:
    - Review heatmap → redistribute tasks from red (overloaded) to green (underutilized) zones.
3. **Forecast Outcomes**:
    - Adjust assignments → view projected project timelines and burnout risk.

Security & Validation

- **Role-Based Access**: Managers only view/edit their team's assignments.
- **Validation Rules**:
    - Prevent assignments exceeding daily max hours (e.g., 8h/day).
    - Block assignments during approved time-off periods.

- **Audit Logs**: Track all assignment changes with user/IP stamps.

Performance Optimization

- **Caching**: Heatmap data cached for 10 minutes (Redis).
- **Bulk Operations**: Use MongoDB bulkWrite for batch assignment updates.
- **Indexing**: Indexes on userId, projectId, and endDate for fast conflict checks.

Testing Strategy

Table 52. Testing Strategy for Work Distribution Tab

Scenario	Validation
Conflict Detection	Assign overlapping tasks → trigger warning.
Burnout Alert >6h focused work for 5 days → flag member.	
Undo/Redo Revert assignments → ensure state consistency.	

Cross-Feature Integration

- **Forecast API**: Model project timelines based on assignment changes.
- **Time Off**: Block assignments during approved leave periods.
- **Activity Monitor**: Correlate workload with productivity trends.

Implementation Guidance

1. **Reuse Patterns**:

- Adapt **Timeline UI** from Activity Monitor for the scheduler.

2. **Key Development Steps**:
    - Build drag-and-drop interface with React DnD.
    - Implement conflict detection using MongoDB aggregation pipelines.

Focus Analysis

Objective: Provide actionable insights into team focus patterns, identify distractions, and recommend strategies to improve deep work efficiency.

Key Features:

- **Deep Work Detection**:
    - Sessions >30 minutes without app/website switches classified as "deep work."
- **Distraction Analysis**:
    - Flags frequent context switches (e.g., >5 app switches/hour).
- **Trend Aggregation**: Compares focus metrics across weeks/months.

Backend Development

Table 53. API Endpoints of Focus Analysis Tab

Endpoint	Method	Purpose
GET /api/focus-analysis	GET	Fetch focus metrics (deep work, switches/hour).
GET /api/focus-trends	GET	Retrieve historical focus trends.

Endpoint	Method	Purpose
POST /api/focus-recommendations	POST	Generate personalized improvement strategies.

Data Model

FocusMetric Schema:

```
{
  userId: ObjectId,
  date: Date,
  deepWorkMinutes: Number,
  switchesPerHour: Number,
  topDistractions: [String],  // e.g., "Fb", "Email"
  productivityScore: Number    // 0-100 scale
}
```

Frontend Integration

UI Components

1. **Focus Heatmap**:
    - **Visualization**:
        - **Daily Grid**: Color intensity reflects deep work duration (darker = more focused).
        - **Hover Details**: Shows distractions and switches/hour for each day.
    - **Filters**: Team member, date range, project.
2. **Switches/Hour Chart**:

- **Line Graph**: Tracks context switches over time.
- **Threshold Line**: Red line at 5 switches/hour (industry benchmark).

3. **Recommendations Panel**:
   - **AI-Powered Tips**:
     - "Schedule focus blocks for John (avg. 8 switches/hour)."
     - "Limit Teams notifications during 9–11 AM (peak deep work)."

Visual Layout

Figure 18. UI Component Layout Outline of Focus Analysis Tab

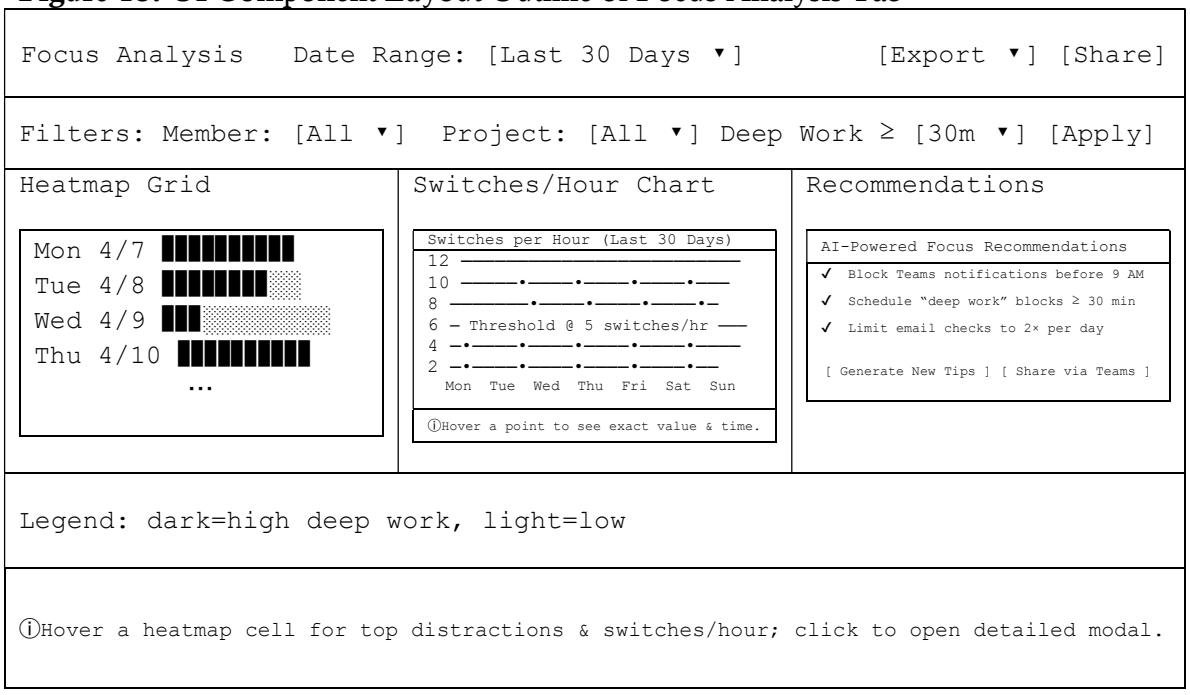


Table 54. Component-API Mapping of Focus Analysis Tab

Component	API Endpoint	Purpose
Focus Heatmap	GET /api/focus-analysis	Load focus metrics and distractions.
Switches/Hour Chart	GET /api/focus-trends	Fetch historical context-switch data.
Recommendations Panel	POST /api/focus-recommendations	Generate and display improvement tips.

Key User Workflows

1. **Identify Distraction Patterns**:
    - Filter to a team member → observe spikes in switches/hour → correlate with apps (e.g., Teams).
2. **Compare Trends**:
    - Overlay focus metrics for two weeks → identify improvement/regression.
3. **Implement Strategies**:
    - Export recommendations → share with team via Teams/email.

Security & Validation

- **Role-Based Access**: Managers only access their team's data.
- **Data Anonymization**: Aggregate metrics hide individual names in team-wide views.
- **Validation**: Reject requests for future dates or non-team members.

Performance Optimization

- **Caching**: Daily focus metrics cached for 24 hours (Redis).
- **Lazy Loading**: Heatmap tiles load on scroll.
- **Indexing**: MongoDB indexes on userId and date.

Testing Strategy

Table 55. Testing Strategy for Focus Analysis Tab

Scenario	Validation
Deep Work Detection	Verify 30m+ uninterrupted sessions are flagged.
Distraction Thresholds >5 switches/hour triggers alerts.	
Cross-Device Sync Ensure metrics sync across desktop/mobile.	

Cross-Feature Integration

- **Activity Monitor**: Feed distraction data into focus metrics.
- **Work Distribution**: Suggest task assignments during peak focus times.
- **Notifications**: Alert managers when switches/hour exceed benchmarks.

Implementation Guidance

1. **Reuse Patterns**:
    - Use **Heatmap UI** from Work Distribution for focus visualization.
    - Integrate **Trend Aggregation** logic from Productivity Pulse.
2. **Key Development Steps**:
    - Build AI recommendation engine using historical data.
    - Implement context-switch tracking using desktop agent event logs.

Team Trends

Objective: Provide managers with a holistic view of team-wide productivity trends, behavioral patterns, and anonymized benchmarks to identify improvement opportunities.

Key Features:

- **Anonymized Benchmarking**: Compare team metrics (e.g., focus time, breaks) without exposing individual data.
- **Pattern Recognition**: Use ML to detect habits (e.g., "Productivity dips after long meetings").
- **Custom Timeframes**: Analyze trends by week/month/quarter.

Backend Development

Table 56. API Endpoints of Team Trends Tab

Endpoint	**Method**	**Purpose**

Endpoint	Method	Purpose
GET /api/team/trends	GET	Fetch aggregated team trends and benchmarks.
GET /api/team/trends/export	GET	Export trend data as CSV/PDF.

API Endpoint: GET /api/team/trends

Fetch aggregated team productivity trends and anonymized benchmarks.

Table 57. Query Parameters of GET /api/team/trends

Parameter	Type	Required	Default	Description
startDate	ISO8601	Yes	-	Start date of analysis (e.g., 2023-01-01T00:00:00Z).
endDate	ISO8601	Yes	-	End date of analysis.
granularity	String	No	weekly	Time grouping: daily, weekly, monthly.
metrics	String[]	No	All	Specific metrics to include (comma-separated): focus, switches, breaks, meetings.
compareWith	String	No	-	Benchmark comparison: company, industry, previous_period.
timezone	String	No	UTC	Timezone for date grouping (e.g., America/New_York).

Response Schema

```
{
  trends: {
    startDate: string;     // ISO8601
    endDate: string;       // ISO8601
    metrics: {
      focus?: {            // Included if `metrics` includes "focus"
        value: number;     // Avg daily focus minutes
        unit: "minutes";
        trend: "up" | "down" | "neutral"; // vs previous period
      };
```

```
      switches?: {        // Included if `metrics` includes "switches"
        value: number;    // Avg switches/hour
        unit: "switches/hour";
      };
      breaks?: {          // Included if `metrics` includes "breaks"
        value: number;    // Avg breaks/day
        unit: "breaks/day";
      };
      meetings?: {        // Included if `metrics` includes "meetings"
        value: number;    // Avg meeting hours/day
        unit: "hours";
      };
    };
    benchmarks?: {        // Included if `compareWith` is provided
      company?: {
        focus: number;    // Company avg focus minutes
        switches: number;
      };
      industry?: {
        focus: number;    // Industry avg focus minutes
        switches: number;
      };
      previous_period?: {
        focus: number;    // Team's focus from prior period
        switches: number;
      };
    };
  }[];
  commonDistractions: string[]; // Top 3 distractions (e.g., ["Teams",
"Email"])
  insights: string[];     // Auto-generated tips (e.g., "Longer meetings
correlate with lower focus")
}
```

Example Request

```
GET /api/team/trends?startDate=2023-10-01T00:00:00Z&endDate=2023-10-
31T23:59:59Z&granularity=weekly&metrics=focus,switches&compareWith=company
,previous_period
```

Example Response


```
{
  "trends": [
    {
      "startDate": "2023-10-01T00:00:00Z",
      "endDate": "2023-10-07T23:59:59Z",
      "metrics": {
        "focus": {
          "value": 210,
          "unit": "minutes",
          "trend": "up"
        },
        "switches": {
          "value": 6.2,
          "unit": "switches/hour"
        }
      },
      "benchmarks": {
        "company": {
          "focus": 190,
          "switches": 7.1
        },
        "previous_period": {
          "focus": 180,
          "switches": 8.0
        }
      }
    }
  ],
  "commonDistractions": ["Teams", "Email", "Jira"],
  "insights": [
    "Focus time increased 16% after reducing meeting frequency."
  ]
}
```

Table 58. Error Responses

Status Code	Body Example	Reason
400	`{ error: "Invalid timezone: Mars/Base" }`	Invalid parameter (e.g., unsupported `timezone`).
403	`{ error: "Access denied" }`	User lacks manager permissions.

Data Model

TeamTrend Schema:

```
{
  teamId: ObjectId,
  dateRange: {
    start: Date,
    end: Date
  },
  metrics: {
    avgFocusTime: Number,      // Minutes/day
    avgSwitchesPerHour: Number,
    commonDistractions: [String],
    meetingImpact: Number      // % productivity change post-meeting
  },
  benchmarks: {
    companyAverage: Number,
    industryAverage: Number
  }
}
```

Frontend Integration

UI Components

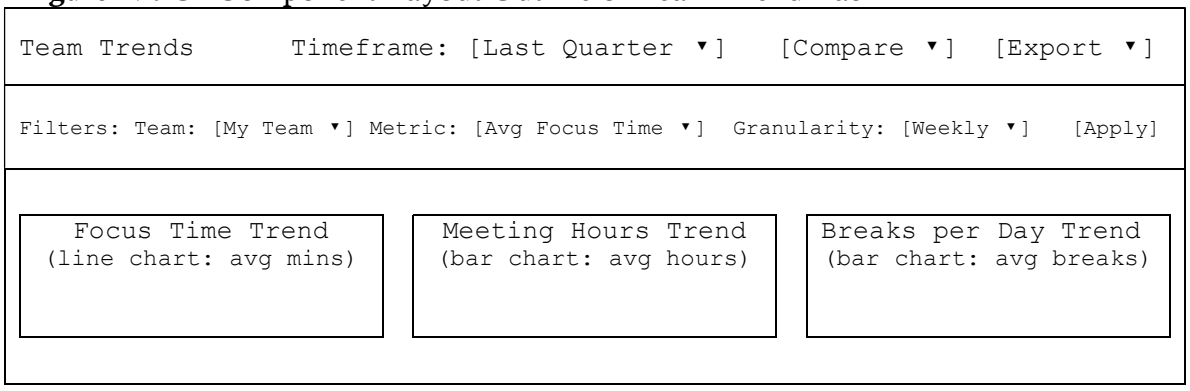
1. **Trends Dashboard**:

- **Line/Bar Charts**: Visualize focus time, meeting hours, and breaks over time.
- **Anonymized Scatter Plots**: Show individual data points without labels (e.g., "Member A: 6h focus").

2. **Pattern Insights Panel**:
   - **Automated Insights**:
     - "Thursdays have 20% lower focus time due to back-to-back meetings."
     - "Teams with 10–15m breaks hourly are 30% more productive."
3. **Benchmark Comparison Tool**:
   - **Dropdowns**: Compare against company/industry averages or past periods.
   - **Delta Indicators**: Show ±% changes (e.g., "+15% vs. Q3").

Visual layout

Figure 19. UI Component Layout Outline of Team Trend Tab



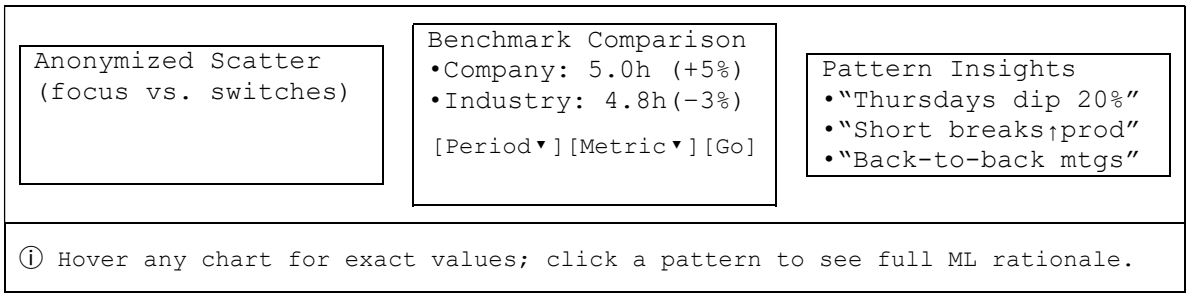


Table 59. Component-API Mapping of Team Trends Tab

Component	API Endpoint	Purpose
Trends Dashboard	GET /api/team/trends	Load aggregated metrics and visualizations.
Benchmark Tool	GET /api/team/trends	Fetch comparison data.
Export Controls	GET /api/team/trends/export	Generate trend reports.

Key User Workflows

1. **Identify Productivity Dips**:
    - Filter to "Last Quarter" → spot correlation between meeting load and focus time.
2. **Benchmark Against Industry**:
    - Compare team's avg. focus time (5.2h) vs. industry (4.8h) → highlight strengths.
3. **Export for Leadership Reviews**:
    - Generate PDF reports with trend summaries and action plans.

Security & Validation

- **Anonymization**: Strip user identifiers before aggregating data.
- **Access Control**: Restrict to managers with team oversight.
- **Validation**: Reject date ranges exceeding 1 year.

Performance Optimization

- **Precomputed Aggregates**: Nightly cron jobs pre-calculate trend data.
- **Lazy Rendering**: Charts load in sections to avoid UI freezes.
- **Indexing**: MongoDB indexes on teamId and dateRange.

Testing Strategy

Table 60. Testing Strategy for Team Trends Tab

Scenario	Validation
Anonymization Check	Ensure no user IDs leak in aggregated data.
ML Pattern Accuracy	Verify detected patterns match manual analysis.
Large-Scale Export	10,000+ data points export in <1 minute.

Cross-Feature Integration

- **Focus Analysis**: Feed focus metrics into trend calculations.
- **Work Distribution**: Correlate workload changes with productivity trends.
- **Activity Monitor**: Use real-time data to update trend dashboards.

Implementation Guidance

1. **Reuse Patterns**:
    - Adopt **Chart Libraries** from Productivity Pulse.

- Use **Aggregation Pipelines** from Work Distribution.

2. **Key Development Steps**:
    - Build ML model for pattern detection (Python/PyTorch).
    - Implement anonymization filters in MongoDB aggregations.

Team

Members

Objective: Enable managers to view, manage, and analyze team members' profiles, roles, and workload distribution with real-time insights and integration with HR systems.

Backend Development

Table 61. API Endpoints of Members Tab

Endpoint	Method	Purpose
`GET /api/team/members`	GET	Fetch team member profiles and metrics.
`POST /api/team/members`	POST	Add a new member to the team.
`PATCH /api/team/members/{id}`	PATCH	Update member role or permissions.
`DELETE /api/team/members/{id}`	DELETE	Remove a member from the team.

Table 62. Query Parameters for GET /api/team/members:

Parameter	Type	Required	Default	Description
include	String	No	basic	Additional data: workload, timeOff, skills.
sortBy	String	No	name	Sort by name, role, joinDate, workload.
status	String	No	active	Filter by active, onLeave, offboarded.

Response Schema

```
{
  members: {
    userId: string;
    name: string;
    role: string;                // e.g., "Developer", "Designer"
    joinDate: string;            // ISO8601
    workload: {
      current: number;           // Hours assigned this week
      capacity: number;          // Max hours/week (e.g., 40)
    };
    timeOff: {
      upcoming: number;          // Days of approved PTO in next 30 days
      used: number;              // Days used this year
    };
    skills: string[];            // e.g., ["React", "UI/UX"]
    contact: {
      email: string;
      Teams: string;
    };
  }[];
  stats: {
    totalMembers: number;
    avgWorkload: number;
    activeProjects: number;
  };
}
```

Example Response:

```
{
  "members": [
    {
      "userId": "user_123",
      "name": "John Doe",
      "role": "Developer",
      "joinDate": "2022-03-15T00:00:00Z",
      "workload": { "current": 35, "capacity": 40 },
      "timeOff": { "upcoming": 5, "used": 10 },
      "skills": ["React", "Node.js"],
      "contact": { "email": "john@company.com", "teams": "@john" }
    }
  ],
  "stats": {
    "totalMembers": 8,
    "avgWorkload": 32,
    "activeProjects": 5
  }
}
```

Frontend Integration

UI Components

1. **Member Grid**:
    - **Columns**: Name | Role | Workload (35/40h) | Skills | Contact.
    - **Sort/Filter**: Toggle between views (e.g., "Show Overloaded Members").
    - **Quick Actions**: "Message on Teams," "View Timesheet," "Edit Role."
2. **Member Profile Modal**:

- **Tabs**:
  - **Overview**: Workload, time-off balance, current projects.
  - **Skills**: Add/remove skills (syncs with HR system).
  - **Activity**: Focus time, distractions, and productivity trends.

3. **Add Member Wizard**:
   - **Steps**: Search by email → Assign role → Confirm permissions.
   - **Integration**: Auto-pull skills from HR system (e.g., Workday).

Visual layout

Figure 20. UI Component Layout Outline of Members Tab

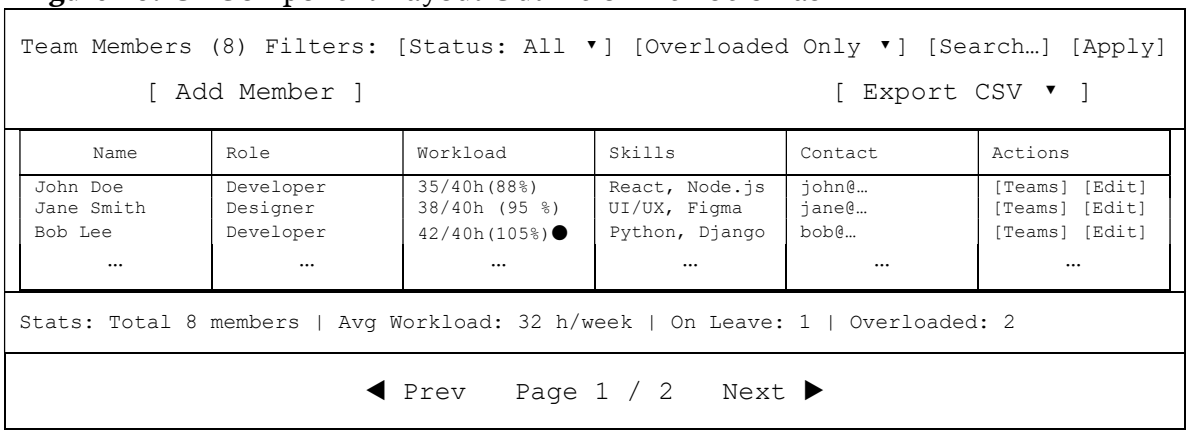


Table 63. Component- API Mapping of Members Tab

Component	API Endpoint	Purpose
Member Grid	GET /api/team/members	Load and filter team members.
Add Member Wizard	POST /api/team/members	Submit new member details.
Member Profile Modal	PATCH /api/team/members/{id}	Update roles/skills.

Key User Workflows

1. **Assign New Role**:
    - Edit role in Member Grid → Select "Tech Lead" → Confirm → Sync to HR system.
2. **Identify Overloaded Members**:
    - Filter by workload > 90% → Redistribute tasks via **Work Distribution** tab.
3. **Offboard Member**:
    - Click "Remove" → Specify offboarding reason → Notify HR and revoke access.

Security & Validation

- **Role-Based Access**:
    - Managers can only edit members within their team.
    - Admins can modify roles/permissions globally.
- **Validation**:
    - Prevent assigning roles beyond the manager's权限 (e.g., can't promote to Admin).
    - Block duplicate member additions.
- **Audit Trail**: Log role changes and offboarding actions.

Performance Optimization

- **Caching**: Member data cached for 15 minutes (Redis).
- **Lazy Loading**: Skills and time-off data fetched on-demand.
- **Indexing**: MongoDB indexes on teamId, role, and workload.

Testing Strategy

Scenario	Validation
Role Permission Escalation	Block managers from assigning admin roles.
Concurrent Edits	Detect version conflicts (e.g., two managers editing the same role).
HR Sync Failure	Retry logic for failed HR system updates.

Cross-Feature Integration

- **Work Distribution**: Auto-fill member skills when assigning tasks.
- **Approvals Hub**: Link to member profiles from timesheet approvals.
- **Activity Monitor**: Drill down into individual productivity metrics.

Implementation Guidance

1. **Reuse Patterns**:
    - Adapt **Grid UI** from Approvals Hub for the Member Grid.
    - Integrate **HR Sync Logic** from Time Off feature.
2. **Key Development Steps**:
    - Build role/permission dropdown with MUI Select.

- Implement real-time sync with HR systems using webhooks.

Assignments

Objective: Enable managers to assign tasks, track progress, and manage deadlines while ensuring balanced workloads and alignment with project goals.

Backend Development

Table 64.API Endpoints of Assignments Tab

Endpoint	Method	Purpose
GET /api/assignments	GET	Fetch all tasks assigned to the team.
POST /api/assignments	POST	Create a new task assignment.
PATCH /api/assignments/{id}	PATCH	Update task details (status, deadline).
DELETE /api/assignments/{id}	DELETE	Remove a task assignment.

Table 65. Query Parameters for GET /api/assignments

Parameter	Type	Required	Default	Description
projectId	String	No	-	Filter by project.
status	String	No	-	Filter by status: pending, in_progress, completed, overdue.
assignedTo	String	No	-	Filter by team member ID.
sortBy	String	No	dueDate	Sort by dueDate, priority, assignedTo.

Request/Response Schemas

Create/Update Assignment (POST/PATCH):

```
{
  taskId: string;          // Required
```

```
  projectId: string;       // Required
  assignedTo: string;      // User ID (Required)
  dueDate: string;         // ISO8601 (Required)
  priority: "low" | "medium" | "high";
  description?: string;
  dependencies?: string[]; // Linked task IDs
}
```

Response Schema (GET):

```
{
  assignments: {
    id: string;
    taskId: string;
    projectId: string;
    assignedTo: {
      userId: string;
      name: string;
      avatar?: string;
    };
    dueDate: string;
    status: "pending" | "in_progress" | "completed" | "overdue";
    priority: "low" | "medium" | "high";
    progress: number;       // 0-100%
    lastUpdated: string;    // ISO8601
  }[];
  stats: {
    totalTasks: number;
    overdue: number;
    avgCompletionTime: number; // Days
  };
}
```

Frontend Integration

UI Components

1. **Assignments Grid**:

- **Columns**: Task | Assigned To | Due Date | Priority | Progress | Status.
- **Filters/Sort**: By project, member, priority, or status.
- **Quick Actions**: Edit, mark as complete, or reassign.

2. **Assignment Detail Modal**:
   - **Tabs**:
     - **Overview**: Description, deadline, dependencies.
     - **Activity Log**: Updates, comments, and file attachments.
     - **Progress Chart**: Visualize completion over time.
3. **Assignment Creator**:
   - **Form Fields**: Task name, assignee, project link, deadline, priority.
   - **Conflict Checks**: Warn if assignee is overloaded or dependencies are incomplete.

Visual Layout

Figure 21. UI Component Layout Outline of Assignment Tab

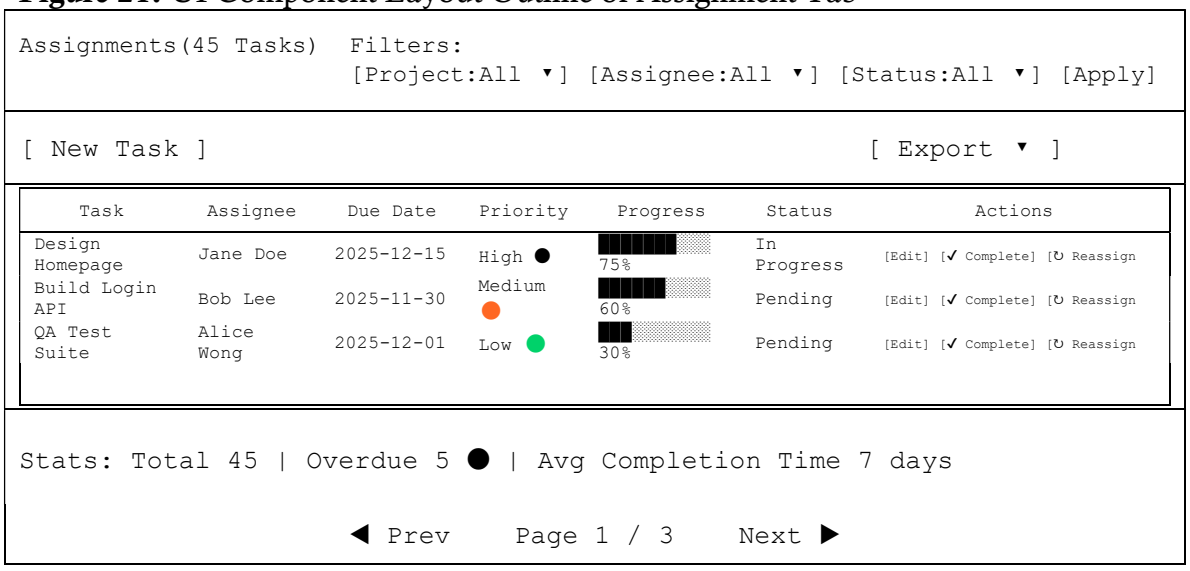


Table 66. Component-API Mapping of Assignments Tab

Component	API Endpoint	Purpose
Assignments Grid	GET /api/assignments	Load and filter tasks.
Assignment Creator	POST /api/assignments	Submit new assignments.
Progress Chart	PATCH /api/assignments/{id}	Update task progress.

Key User Workflows

1. **Assign a Task**:
    - Fill form → Validate assignee's workload → Confirm → Sync to Work Distribution.
2. **Track Progress**:

    - Filter by "in_progress" → Update progress percentage → Add comments.
3. **Handle Overdue Tasks**:
    - Filter by "overdue" → Reassign or adjust deadlines → Notify member via Slack.

Security & Validation

- **Access Control**:
    - Only managers can create/delete assignments.
    - Members can only update their own task progress.
- **Validation**:
    - Block assignments to users with >90% workload (configurable threshold).
    - Reject past due dates or circular dependencies.
- **Encryption**: Task descriptions and files encrypted (AES-256).

Performance Optimization

- **Caching**: Task lists cached for 10 minutes (Redis).
- **Indexing**: MongoDB indexes on projectId, assignedTo, and dueDate.
- **Lazy Loading**: Activity logs load on tab click.

Testing Strategy

Table 67. Testing Strategy for Assignments Tab

Scenario	Validation
Workload Validation	Block assignment if user workload exceeds 90%.
Dependency Chain	Prevent circular task dependencies.
Permission Checks	Members cannot edit others' tasks.

Cross-Feature Integration

- **Work Distribution**: Sync assignments to balance workloads.
- **Activity Monitor**: Track time spent on tasks for productivity insights.
- **Approvals Hub**: Flag overdue tasks for escalation.

Implementation Guidance

1. **Reuse Patterns**:
    - Use **Grid UI** from the Members tab for assignments listing.
    - Integrate **Conflict Checks** logic from Work Distribution.
2. **Key Development Steps**:
    - Build dependency visualization using React Flow or similar.
    - Add real-time progress tracking via WebSocket (/ws/assignments).

Capacity Planner

Objective

Enable managers to allocate tasks efficiently, forecast project timelines, and balance team workloads by:

- Visualizing member capacity vs. allocated hours.
- Identifying skill gaps and recommending solutions (training/hiring).
- Simulating "what-if" scenarios for resource allocation.

Backend Development

Table 68. API Endpoints of Capacity Planner Tab

Endpoint	Method	Parameters	Purpose
/api/team/capacity	GET	projectId, timeframe=weekly	Fetch team capacity, skills, and workload.
/api/assignments/optimize	POST	projectId, strategy=balance	Auto-reassign tasks to resolve overloads/skill gaps.
/api/skill-gaps	GET	projectId	Identify missing skills required for a project.
/api/capacity/scenarios	POST	scenarioName, assignments[]	Save/compare hypothetical allocation scenarios.
/api/assignments/{taskId}	PATCH	newUserId	Reassign a task to another member (real-time update).

Data Models

CapacityPlan Schema

```
{
  projectId: { type: Schema.Types.ObjectId, ref: 'Project' },
  startDate: Date,
  endDate: Date,
```

```
  members: [{
    userId: { type: Schema.Types.ObjectId, ref: 'User' },
    allocatedHours: Number,
    skills: [String],
    timeOff: [Date]
  }],
  scenarios: [{
    name: String,
    assignments: [{
      taskId: { type: Schema.Types.ObjectId, ref: 'Task' },
      userId: { type: Schema.Types.ObjectId, ref: 'User' },
      requiredSkills: [String]
    }]
  }],
  version: { type: Number, default: 1 }
}
```

SkillGap Schema

```
{
  projectId: { type: Schema.Types.ObjectId, ref: 'Project' },
  missingSkills: [String],
  recommendations: [{
    type: { type: String, enum: ["train", "hire", "contractor"] },
    userId: Schema.Types.ObjectId, // For training
    skill: String,
    priority: { type: String, enum: ["low", "medium", "high"] }
  }]
}
```

Front End Integration

1. **Workload Heatmap**
    - **Material UI:**
        - Grid for layout of member rows × timeframe columns.
        - Paper containers for each cell to give elevation and separation.

        - Tooltip on hover to show:
            - Member name & avatar
            - Weekly capacity vs. allocated hours (e.g., "35/40h")
            - Upcoming time-off dates
            - List of tasks contributing to that week's load
    - **Behavior & Interaction:**
        - Cells are colored—green, yellow, or red—based on workloadStatus.
        - Clicking a cell drills into a side panel with full assignment details.
        - Virtualized rendering for large teams/timeframes to maintain performance.
    - **API:**
        - **GET** /api/team/capacity?projectId={id}&timeframe=weekly
        - Parses members[] array, mapping each to a grid row and using weeklyCapacity/allocatedHours to compute color.
2. **Scenario Simulation Panel**
    - **Material UI & Libraries:**
        - DragDropContext and Droppable (from react-beautiful-dnd) for drag-and-drop task assignment.
        - List / ListItem to show the "Tasks Pool."

        - Card or Paper around each member drop zone.
        - Button controls for switching scenario (Select + MenuItem) and "Optimize" action.
    - **Behavior & Interaction:**
        - Users can drag a task from the pool onto a member's card; on drop, the UI:
            - Immediately checks for conflicts—overload or missing skills—via client-side rules.
            - Visually flags any conflict with a red border or warning icon.
            - Updates a local "draft" scenario state.
        - Changing the scenario selector reloads assignments from that scenario via API.
        - "Optimize" button invokes auto-assignment strategy.
    - **API:**
        - **POST** /api/capacity/scenarios to save or compare the current draft scenario.
        - Payload: { scenarioName, assignments[] }
3. **Skill Gap Alerts**
    - **Material UI:**

        - Alert components to surface top-level severity ("Critical," "High," "Medium").
        - Chip elements for each missing skill, labeled with severity or recommended action type.
        - List + ListItem to display recommended actions under each alert.
    - **Behavior & Interaction:**
        - On page load or scenario change, fetches skill gaps and shows:
            - An Alert strip at the top if any critical gaps exist.
            - A collapsible panel listing each skill chip; clicking a chip reveals associated recommended actions (e.g., training link, hiring form).
        - Allows managers to click "Schedule Training" or "Open Hiring Request" directly from the list items.
    - **API:**
        - **GET** /api/skill-gaps?projectId={id}
        - Uses missingSkills[] and recommendedActions[] to populate chips and list entries.
4. **Scenario Comparison Modal**
    - **Material UI:**
        - Dialog for the modal overlay.

        - DialogTitle, DialogContent, and DialogActions for structure.
        - Button to close or switch between scenarios.
        - Chart components (e.g., a LineChart from Recharts) to plot projected timeline overlays of two scenarios.
    - **Behavior & Interaction:**
        - Opens when the user clicks "Compare" on a saved scenario.
        - Fetches both baseline and target scenario data via API.
        - Renders side-by-side or overlaid line/bar charts showing:
            - Projected completion dates
            - Member utilization trends
        - Provides a dropdown inside the modal to pick which two scenarios to compare.
    - **API:**
        - **GET** /api/capacity/scenarios/{id} (pass two scenario IDs as query parameters if needed)
        - Response includes projected timelines and capacity allocations for each scenario to drive the charts.

Visual Layout

Figure 22. UI Component Layout Outline of Capacity Planner Tab

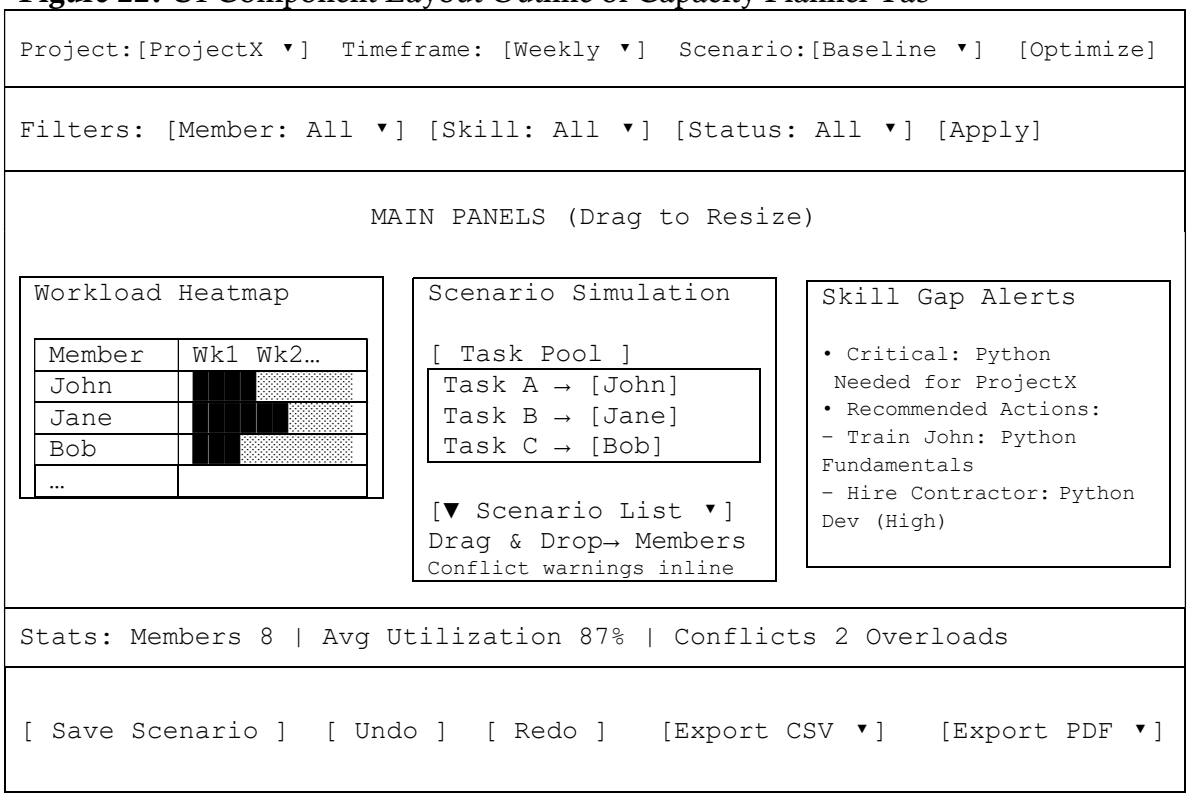


Table 69. Component-API Mapping of Capacity Planner Tab

Component	Material UI Components	API Endpoint	Purpose
Workload Heatmap	Grid, Paper, Tooltip	GET /api/team/capacity	Visualize member workload (green/yellow/red).
Scenario Simulation Panel	DragDropContext, Droppable	POST /api/capacity/scenarios	Drag-and-drop tasks to model hypothetical allocations.
Skill Gap Alerts	Alert, Chip, List	GET /api/skill-gaps	Display missing skills and recommended actions.

Component	Material UI Components	API Endpoint	Purpose
Scenario Comparison Modal	Dialog, Button, LineChart	GET /api/capacity/scenarios/{id}	Compare timelines/outcomes of saved scenarios.

Key User Workflows

1. **Workload Visualization**:
    - Managers view a heatmap of member capacity vs. allocated hours.
    - Overloaded members (>100%) are flagged in red.
    - Clicking a member opens a detail panel with skills and time-off.
2. **Scenario Simulation**:
    - Drag tasks between members to test resource allocation.
    - Auto-calculate projected project deadlines based on new assignments.
    - Save scenarios (e.g., "Hire Python Dev" vs. "Train Existing Team").
3. **Skill Gap Resolution**:
    - System flags missing skills (e.g., "Python").
    - Managers can:
        - Request training for a member.

Testing Strategy

Table 70. Testing Strategy for Capacity Planner Tab

Test Case	Validation
Overload Prevention	Assigning 45h to a member with 40h capacity → API rejects with `409 Conflict`.
Skill-Based Assignment	Assign Python task to a non-Python developer → UI blocks with tooltip.
Scenario Saving	Save a scenario → Verify MongoDB entry and versioning.
Real-Time Sync	Two managers editing the same plan → Optimistic locking prevents overwrites.

Security & Validation

1. **Role-Based Access**:
    - Only users with manager or admin roles can modify assignments.
    - Validated via JWT middleware:

```
router.patch('/assignments/:taskId', authenticate('manager'), (req,
res) => { ... });
```

2. **Input Sanitization**:
    - Reject assignments with invalid userId or taskId.
    - Prevent negative hours or non-existent skills.

Edge Case Handling

Table 71. Edge Case of Capacity Planner

Scenario	Resolution
Member Leaves Mid-Project	Auto-reassign tasks to backups; notify manager.
Skill Gap with No Solutions	Escalate to admin via `POST /api/alerts` with "Urgent Hiring Needed".
Data Corruption	Restore from last validated snapshot stored in AWS S3.

Time Off Request

Requests

Objective

Enable managers to review, approve, or reject time-off requests with:

- Calendar conflict detection (project/task overlaps)
- **Attendance-based conflict detection** (e.g. leave dates where attendance was recorded)
- Bulk processing and delegation
- Full audit logging

Key Features

1. **Attendance Conflict Detection**

- Auto-flag requests on dates marked “Present” or “Partial”
- New status badge: ⚠️ **Attendance Conflict**

2. **Enhanced Backup Suggestions**

- Prioritize users with ≥ 90% attendance rate
- Exclude backups who themselves have pending leave for the same period

3. **Attendance–Audit Integration**

- Upon approval, optionally convert attendance records from "Present" → "On Leave"
- Record override reasons

Backend Development

Table 72. API Endpoints of Request Tab

Endpoint	Method	Parameters	Purpose
GET /api/time-off/requests	GET	status=pending	Fetch filtered requests (date, member, type, attendanceStatus)
GET /api/time-off/requests/{id}	GET	-	Fetch full request details (dates, notes, conflicts).
PATCH /api/time-off/requests/{id}	PATCH	status, comment	Approve/reject (with optional attendanceAdjustment override
POST /api/time-off/requests/bulk	POST	action, requestIds[]	Bulk approve/reject requests.
GET /api/time-	GET	userId, date,	Check coverage & attendance conflicts

Endpoint	Method	Parameters	Purpose
off/conflicts		checkAttendance=true	

Data Models

TimeOffRequest Schema

```
// TimeOffRequest Schema (Mongoose)
{
  userId: ObjectId,
  dates: [Date],
  type: { type: String, enum: ["vacation","sick","unpaid"] },
  status: { type: String, enum: ["pending","approved","rejected"],
default: "pending" },
  approverId: ObjectId,
  backupUserId: ObjectId,          // system-suggested backup
  conflictDetails: {
    isConflict: Boolean,
    conflictingTasks: [String],     // e.g. ["ProjectX Sprint Planning"]
    attendanceConflicts: [          // NEW
      { date: Date, status: String } // "present", "partial", "absent"
    ]
  },
  resolution: {                     // NEW
    attendanceAdjustment: Boolean,  // auto-correct attendance
    overrideReason: String
  }
}
```

Frontend Integration

UI Components

1. **Request List Table**

- **Columns**: Select • Member • Dates • Type • Status (with icons) • Attendance Match • Actions
  - Status icons:
    - ✅ Approved / ❌ Rejected / ⏳ Pending / ⚠️ Attendance Conflict
  - Attendance Match: 🟢 100% / 🟡 50–99% / 🔴 < 50%
- **Filters**:
  - Date Range, Type, Member, Status
  - **Attendance Status**
    - ☑ Show Conflicts ☑ Hide Resolved
- **Behavior**:
  - Sort by any column
  - Expand row → inline detail view

2. **Conflict Calendar Overlay**

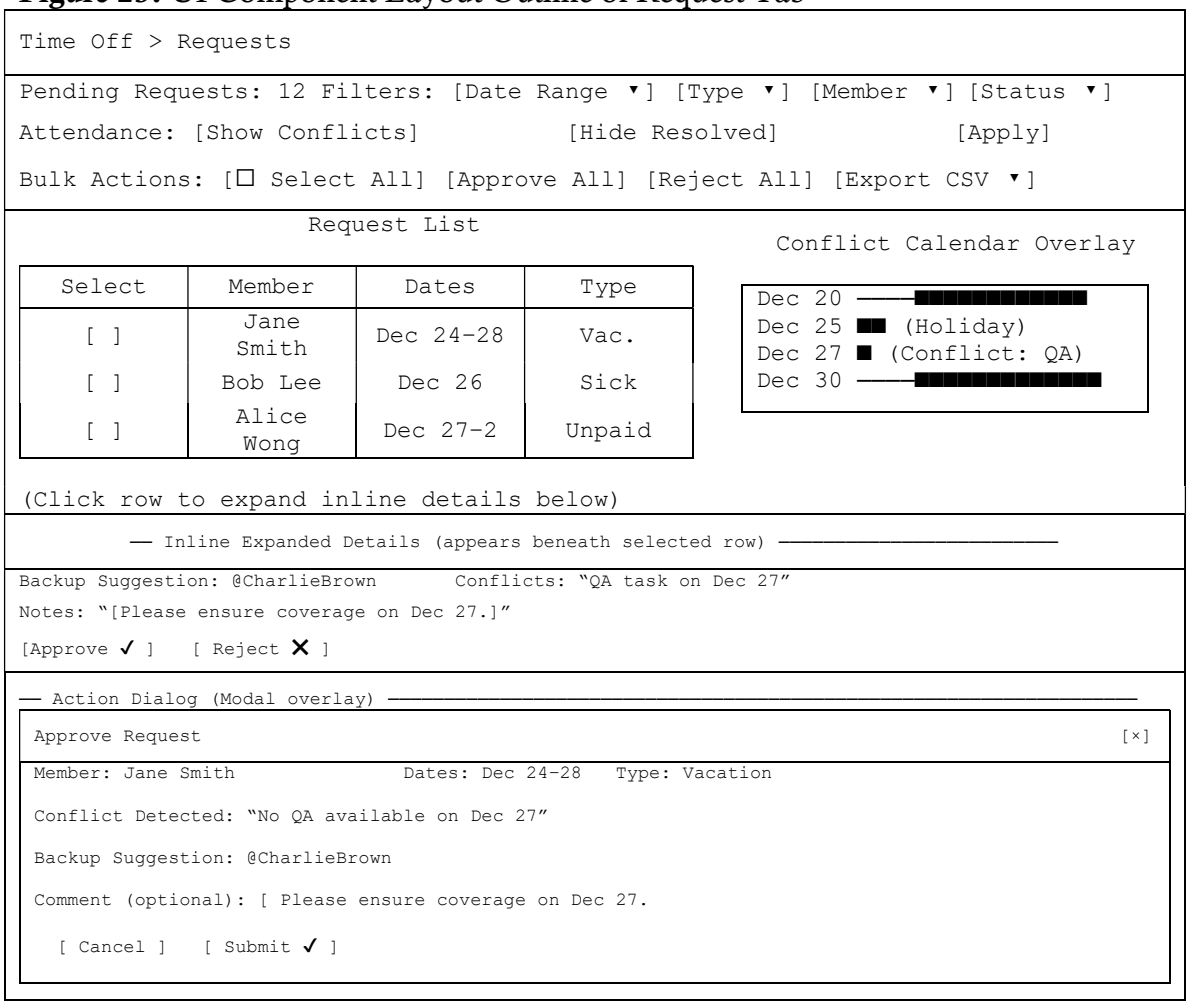
- Overlays leave dates with:
  - Team task deadlines
  - Attendance conflicts flagged in red (e.g. "Dec 27 ■ (Attendance Conflict: Present)")
- API: GET /api/time-off/conflicts?checkAttendance=true

3. **Action Dialog**

- Used for approving/rejecting one request
- **Sections**:
    - Member • Dates • Type
    - Conflict summary (project & attendance)
    - Backup suggestion dropdown (prioritized by attendance rate)
    - **Attendance Adjustment** checkbox (auto-correct attendance)
    - Override Reason text field
    - Comment field
- API: **PATCH** /api/time-off/requests/{id}

Visual layout

Figure 23. UI Component Layout Outline of Request Tab



Key User Workflows

1. **Review Requests**:
    - Filter by "Pending" → View table → Expand rows to check conflicts.
    - Click calendar icon to visualize overlaps with team tasks.
2. **Approve/Reject**:

- Select request → Add comment → Choose backup → Submit (PATCH /api/time-off/requests/{id}).

3. **Bulk Actions**:
    - Select multiple requests → Click "Approve All" → Confirm.
4. **Conflict Resolution**:
    - System flags conflicting dates → Manager assigns backup member from suggestions.

Testing Strategy

Table 73. Testing Strategy for Request Tab

Test Case	Validation
Single Approval	Approve request → Verify status change & audit log entry.
Bulk Rejection	Reject 5 requests → Confirm all statuses update.
Conflict Detection	Submit overlapping leave → Ensure conflict flag triggers.
Backup Assignment	Assign backup → Verify task reallocation in Work Distribution.

Security & Validation

1. **Role-Based Access**:
    - Only manager roles can approve/reject requests.
2. **Input Validation**:
    - Reject invalid dates (e.g., past dates without admin override).
    - Block duplicate requests for the same user/date.

Edge Case Handling

Table 74. Edge Case Handling for Request tab

Scenario	Resolution
Overlapping Requests	Auto-assign backup from suggestions.
Backup Unavailable	Escalate to admin with `POST /api/alerts`.
Retroactive Approval	Require admin override (`forceApprove=true` query param).

Balances

Objective:

Enable managers to:

- View **accrued, used, and remaining time-off balances** for their team members (PTO, sick leave).
- Monitor compliance with company policies (accrual rates, annual caps).
- Identify trends (e.g., frequent absences) and ensure team coverage.

Backend Development

Table 75. API Endpoints of Balance Tab

Endpoint	Method	Parameters	Purpose
`/api/team/time-off/balances`	`GET`	`teamId` (auto)	Fetch balances for the manager's team.
`/api/time-off/policies`	`GET`	-	Retrieve company-wide accrual policies.

Data Models

TeamTimeOffBalance Schema:

```
{
  teamId: { type: Schema.Types.ObjectId, ref: 'Team' },
  members: [{
    userId: { type: Schema.Types.ObjectId, ref: 'User' },
    pto: {
      accrued: Number,
      used: Number,
      remaining: Number,
      annualCap: Number
    },
    sickLeave: {
      accrued: Number,
      used: Number,
      remaining: Number,
      annualCap: Number
    }
  }],
  aggregated: {
    totalUsed: Number,
    totalRemaining: Number
  }
}
```

Frontend Integration

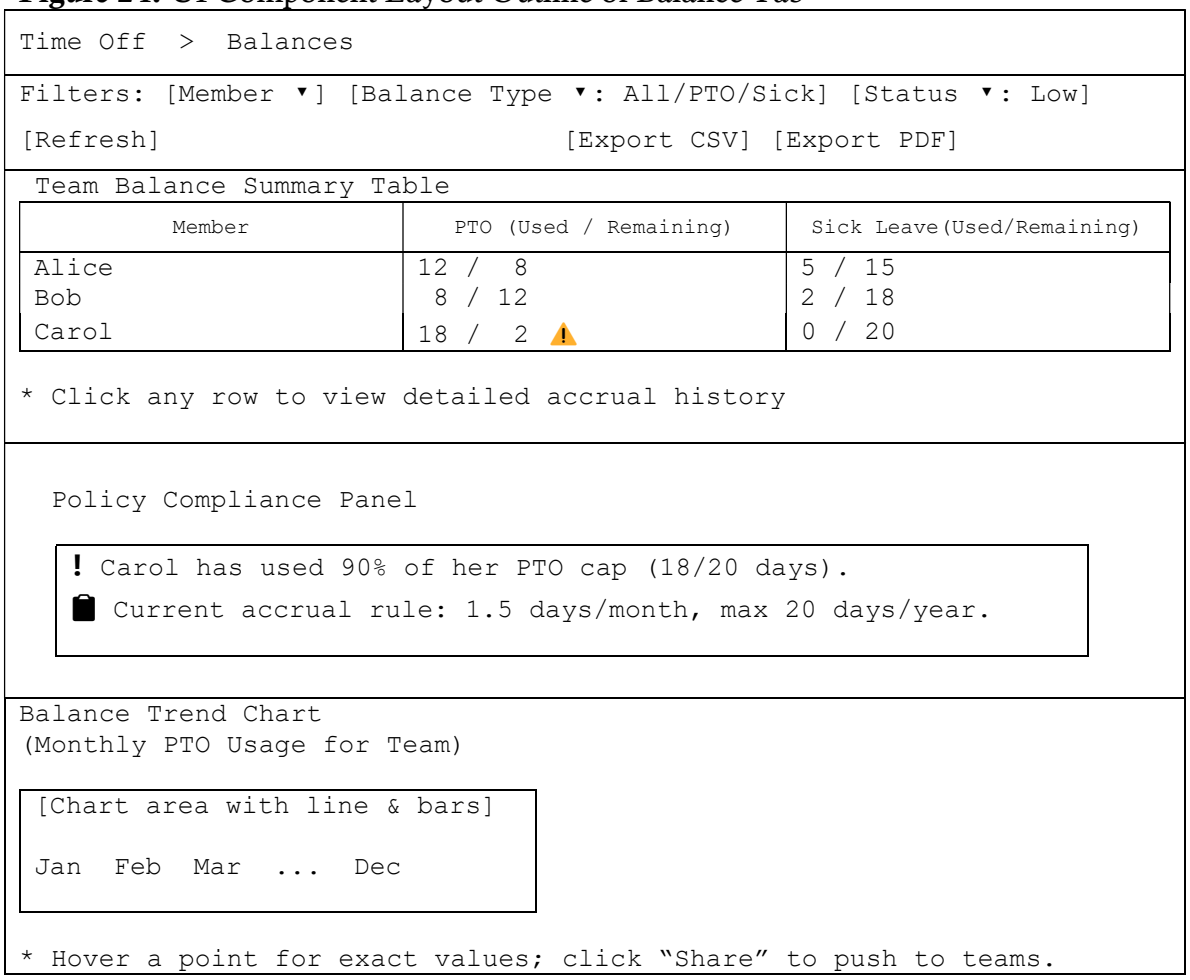
UI Components

1. **Team Balance Summary Table**:
    - **Columns**: Member | PTO (Used/Remaining) | Sick Leave (Used/Remaining) | Status (e.g., "Low Balance").
    - **Filters**: By member, balance type (PTO/sick), or status.
    - **Interactions**:
        - Click a member's row to view detailed accrual history.

        - Export to CSV/PDF for HR reporting.
    - **API**: GET /api/team/time-off/balances.
2. **Policy Compliance Panel**:
    - **Visualization**:
        - Highlight members approaching/annual caps (e.g., "John: 18/20 PTO days used").
        - Warning icons for policy violations (e.g., negative balances).
    - **API**: GET /api/time-off/policies.
3. **Balance Trend Chart**:
    - **Libraries**: Recharts.
    - **Behavior**:
        - Show monthly balance trends for the team (e.g., PTO usage spikes in December).

Visual Layout

Figure 24. UI Component Layout Outline of Balance Tab



Key User Workflows

1. **Monitor Team Balances**:
    - Navigate to **Time Off > Balances** → Filter by "Low Remaining" → Identify members needing coverage planning.
2. **Audit Compliance**:

- Review policy violations (e.g., members exceeding annual caps) → Escalate to Super Admin if needed.

3. **Export Data**:
    - Generate CSV reports for HR meetings or budget planning.

Testing Strategy

Figure 25. Testing Strategy for Balances Tab

Test Case	Validation
Team Filtering	Ensure managers only see their own team's data.
Policy Violation Alerts	Trigger warnings when usage exceeds 90% of cap.
Data Export Integrity	Verify CSV/PDF includes all filtered members.

Security & Validation

- **Access Control**:
    - Managers can **only access their own team's data** (validated via JWT teamId).
    - Balance adjustments restricted to **Super Admin** (not visible in Manager UI).
- **Validation Rules**:
    - Auto-calculate remaining as accrued - used.
    - Block negative remaining values in frontend displays.

Table 76. Edge Case Handling of Balances Tab

Scenario	Resolution
Negative Balances	Display as "0" with red alert for Super Admin.
Conflicting Accrual Rates	Use policy version active during the accrual period.

Cross-Feature Integration

1. **Time Off Requests**:
    - Approved requests auto-update used balances (linked to /api/time-off/requests).
2. **Notifications**:
    - Alert managers when critical thresholds are reached (e.g., "Team PTO usage at 80%").
3. **Super Admin UI**:
    - Escalate policy violations to Super Admin's **Audit Trail**.

SUPER ADMIN

Time Management

Timesheet Explorer

Objective:

Enable Super Admins to:

- Audit and analyze **organization-wide timesheets** for accuracy and compliance.
- Identify anomalies (e.g., overtime without approval, missing entries).
- Enforce policies (e.g., maximum work hours, break rules) across teams.

Backend Development

Table 77. API Endpoints of Timesheet Explorer.js

Endpoint	Method	Parameters	Purpose
/api/timesheets	GET	teamId, startDate, endDate, userId	Fetch timesheets with filters.
/api/timesheets/{id}	PATCH	adjustments, reason	Edit timesheet entries (admin override).
/api/timesheets/export	GET	format=csv/pdf	Export raw or aggregated timesheet data.
/api/timesheets/compliance	POST	policyId	Check timesheets against a specific policy.

Data Models:

***Timesheet Schema (Extended)**:*

```
{
  userId: { type: Schema.Types.ObjectId, ref: 'User' },
  entries: [{
    date: Date,
    hours: Number,
    project: String,
    tags: [String],
    status: {
      type: String,
      enum: ["submitted", "approved", "flagged"]
    }
  }],
  auditTrail: [{
    editedBy: { type: Schema.Types.ObjectId, ref: 'User' },
    timestamp: Date,
    changes: {
      field: String, // E.g., "hours", "project"
      oldValue: mongoose.Schema.Types.Mixed,
      newValue: mongoose.Schema.Types.Mixed
    },
    reason: String
  }],
  policyId: { type: Schema.Types.ObjectId, ref: 'Policy' } // Linked
compliance rule
}
```

Frontend Integration

UI Components:

1. **Advanced Timesheet Grid**:
    - **Columns**: User | Date | Hours | Project | Tags | Status | Last Edited.

- **Features**:
    - Search by user/date/project.
    - Bulk flag entries for review.
    - Sort by compliance status (e.g., "Overtime Violations").
- **API**: GET /api/timesheets.

2. **Audit Trail Panel**:
    - **Behavior**:
        - Click a timesheet entry to view full edit history.
        - Filter edits by date/editor.
    - **API**: Embedded in GET /api/timesheets response.
3. **Compliance Checker**:
    - **Workflow**:
        - Select a policy (e.g., "Max 8h/day") → Scan all timesheets → Flag violations.
    - **API**: POST /api/timesheets/compliance.
4. **Bulk Action Toolbar**:
    - **Actions**:
        - Export selected entries to CSV/PDF.
        - Override status (e.g., force-approve flagged entries).

Visual Layout

Figure 26. UI Component Layout Outline of Balance Tab

Timesheet Explorer

Filters: [Team ▾] [User ▾] [Date Range ▾] [Status ▾: All/Flagged/Approved] [Search]
[Run Compliance Check ▾] [Refresh] [Export CSV] [Export PDF]

🔍 Bulk Action Toolbar

[Flag Selected] [Approve Selected] [Reject Selected] [Override Hours]

📋 Advanced Timesheet Grid

User	Date	Hours	Project	Tags	Status	Last Edited
Alice	2023-11-20	9.5	ProjectX	Dev	Approved	11/21 08:00
Bob	2023-11-20	10.0	ProjectY	QA	Flagged	11/21 09:15
Carol	2023-11-20	12.0	Support	Ops	Flagged	11/21 10:30

* Select rows to enable Bulk Actions

🔎 Audit Trail Panel (side or modal)

• Click an entry to view full history:
  — Bob's 2023-11-20 entry

┬ Edited by Admin1 on 11/21/09:00 – Hours changed from 9.5 → 10.0 (typo)
┴ Edited by Admin2 on 11/21/10:00 – Status changed to Flagged (overtime)

✅ Compliance Results Panel
• Last run: 11/21/2023 10:00
• Policy "Max 8h/day": 5 violations
  – Carol (12h), Dave (9h), …
[View Violations Details] [Re-run Compliance]

Key User Workflows

1. **Audit Timesheets**:
    - Filter by status=flagged → Review anomalies → Approve/reject with comments.
2. **Policy Enforcement**:

- Run compliance check for "Weekend Work Policy" → Export violations for HR.

3. **Data Correction**:
    - Edit incorrect entries (e.g., adjust hours) → Provide audit reason (e.g., "Corrected typo").

Testing Strategy

Table 78. Testing Strategy

Test Case	Validation
Bulk Edit Permissions	Only Super Admins can override timesheets.
Policy Compliance Scan	Flag entries violating max hours/break rules.
Audit Trail Integrity	Ensure all edits log user, timestamp, and reason.

Security & Validation

- **Access Control**:
    - Restricted to Super Admins (validated via JWT role=super_admin).
- **Validation Rules**:
    - Reject edits without a reason field.
    - Block future-dated timesheet entries.

Edge Case Handling

Table 79. Edge Case Handling for Timesheet Explorer Tab

Scenario	Resolution
Conflicting Edits	Optimistic locking via `version` field.
Large Dataset Performance	Pagination + MongoDB aggregation pipelines.
Policy Version Conflicts	Apply rules active during the timesheet period.

Cross-Feature Integration

1. **Audit Trail**:
    - All edits sync to the Super Admin's global audit log.
2. **HR Systems**:
    - Export violations to Workday/BambooHR for disciplinary action.
3. **Notifications**:
    - Alert managers of systemic compliance issues.

Approvals Hub

Objective:

Enable Super Admins to:

- Oversee **all pending approvals** organization-wide (timesheets, time-off requests, policy exceptions).
- Delegate approval tasks, prioritize urgent items, and enforce accountability.
- Audit approval/rejection decisions for compliance and transparency.

Table 80. API Endpoints of Approvals Hub Tab

Endpoint	Method	Parameters	Purpose
/api/approvals	GET	type, status, priority	Fetch approvals (pending/approved/rejected).
/api/approvals/bulk	POST	action, approvalIds[]	Bulk approve/reject/delegate items.
/api/approvals/delegate	POST	approvalId, assigneeId	Reassign approval task to another manager.
/api/approvals/history	GET	userId, date	Audit historical decisions by user/date.

Data Models:

Approval Schema:

```
{
  type: {
    type: String,
    enum: ["timesheet", "time_off", "policy_exception"]
  },
  submissionId: { type: Schema.Types.ObjectId }, // Linked to
Timesheet/TimeOffRequest
  submittedBy: { type: Schema.Types.ObjectId, ref: 'User' },
  assignedTo: { type: Schema.Types.ObjectId, ref: 'User' },
  status: {
    type: String,
    enum: ["pending", "approved", "rejected", "delegated"]
  },
  priority: {
    type: String,
    enum: ["low", "medium", "high"],
    default: "medium"
  },
  deadline: Date,
```

```
  history: [{
    action: String, // E.g., "approved", "delegated"
    user: { type: Schema.Types.ObjectId, ref: 'User' },
    timestamp: Date,
    comment: String
  }]
}
```

Frontend Integration

UI Components:

1. **Unified Approvals Grid**:
    - **Columns**: Type | Submitter | Assigned To | Priority | Deadline | Status.
    - **Filters**: By type, priority, deadline proximity, or assignee.
    - **Interactions**:
        - Click to view submission details (e.g., timesheet hours, PTO dates).
        - Drag-and-drop to reorder by priority.
    - **API**: GET /api/approvals.
2. **Bulk Action Toolbar**:
    - **Actions**:
        - Approve/Reject selected items → Add bulk comments.
        - Delegate tasks to managers → Track reassignment history.
    - **API**: POST /api/approvals/bulk.

3. **Delegation Interface**:
    - **Behavior**:
        - Search/select managers for reassignment.
        - Notify assignees via email/Slack.
    - **API**: POST /api/approvals/delegate.
4. **Audit History Panel**:
    - **Visualization**: Timeline of approval actions (user, decision, timestamp).
    - **API**: GET /api/approvals/history.

Visual Layout

Figure 27. UI Component Layout Outline of Approvals Hub Tab

Approvals Hub

Filters: [Type ▾] [Status ▾] [Priority ▾] [Assignee ▾] [Deadline ▾] [🔍]

[Run Bulk Action ▾] [Refresh] [Export CSV] [Export PDF]

Bulk Actions: [Approve] [Reject] [Delegate] [Add Comment]

Type	Submitter	Assigned To	Priority	Deadline	Status
timesheet	Alice	Bob	High	11/22	Pending
time off	Carol	Dave	Medium	11/25	Approved
policy_exc.	Eve	Frank	Low	11/20	Delegated
…	…	…	…	…	…

Approval Details (on row click)
- Submission ID: 65a…
- Type: Timesheet
- Submitted By: Alice
- Assigned To: Bob
- Priority: High
- Deadline: 11/22
- Comments: "Need more context"

Audit History

- Alice approved by Bob on 11/21 09:00
- Reason: "Overtime without approval"
- Carol delegated to Dave on 11/20 10:00

Key User Workflows

1. **Triage Approvals**:
    - Filter by priority=high → Approve/reject critical items first.
2. **Delegate Overload**:
    - Select 10+ pending items → Reassign to multiple managers → Monitor progress.

3. **Audit Compliance**:
    - Review history for a user → Flag biased decisions (e.g., always approving a specific team).

Testing Strategy

Table 81. Testing Strategy for Approvals Hub Tab

Test Case	Validation
Bulk Delegation	Ensure tasks are reassigned and notifications sent.
Deadline Enforcement	Auto-flag overdue items as "Escalated".
Conflict of Interest	Block self-approval (e.g., manager approving own request).

Security & Validation

- **Access Control**:
    - Only Super Admins can delegate globally or override decisions.
    - Managers can only act on items assigned to them.
- **Validation Rules**:
    - Reject approvals missing comment for rejections.
    - Auto-set deadline based on policy (e.g., 72h for timesheets).

Edge Case Handling

Table 82. Edge Case Handling for Approval Hub Tab

Scenario	Resolution
Assignee Unavailable	Auto-reassign to backup approver defined in policy.
Circular Delegation	Block loops (e.g., User A → User B → User A).
Expired Deadlines	Escalate to Super Admin for manual resolution.

Cross-Feature Integration

1. **Audit Trail**:
    - Log all delegation/approval actions in the global audit log.
2. **Notifications**:
    - Alert assignees of new tasks via WebSocket and email.
3. **Policy Engine**:
    - Auto-prioritize approvals violating SLAs (e.g., pending > 48h).

Audit Trail

Objective:

Enable Super Admins to:

- Track **all system-wide changes** (timesheet edits, policy updates, approvals, user actions).
- Investigate historical data for compliance, security, or debugging purposes.

- Generate legally compliant audit reports (e.g., GDPR, SOC 2).

Backend Development

Table 83. API Endpoints of Audit Trail Tab

Endpoint	Method	Parameters	Purpose
/api/audit-logs	GET	action, userId, startDate, endDate	Fetch filtered audit logs.
/api/audit-logs/export	GET	format=csv/pdf	Export logs in compliance-ready formats.
/api/audit-logs/{id}	GET	-	Retrieve full details of a specific log entry.

Data Models:

AuditLog Schema:

```
{
  type: {
    type: String,
    enum: ["timesheet", "time_off", "policy_exception"]
  },
  submissionId: { type: Schema.Types.ObjectId }, // Linked to
Timesheet/TimeOffRequest
  submittedBy: { type: Schema.Types.ObjectId, ref: 'User' },
  assignedTo: { type: Schema.Types.ObjectId, ref: 'User' },
  status: {
    type: String,
    enum: ["pending", "approved", "rejected", "delegated"]
  },
  priority: {
    type: String,
    enum: ["low", "medium", "high"],
    default: "medium"
  },
  deadline: Date,
  history: [{
```

```
    action: String, // E.g., "approved", "delegated"
    user: { type: Schema.Types.ObjectId, ref: 'User' },
    timestamp: Date,
    comment: String
  }]
}
```

Frontend Integration

UI Components:

1. **Audit Log Viewer**:
    - **Columns**: Timestamp | Action | User | Target | Changes | IP Address.
    - **Filters**: By action type, user, date range, or target entity.
    - **Interactions**:
        - Click a log entry to view pre/post-change snapshots.
        - Export logs for specific users/teams.
    - **API**: GET /api/audit-logs.
2. **Diff Viewer**:
    - **Behavior**:
        - Side-by-side comparison of oldValue vs. newValue (e.g., timesheet hours before/after edit).
        - Highlight text/numeric differences in red/green.
    - **API**: Embedded in GET /api/audit-logs/{id} response.
3. **Anomaly Detection Panel**:

- **Triggers**:
  - Frequent failed login attempts.
  - Mass deletions/edits by a single user.
- **API**: Real-time alerts via WebSocket (/ws/audit-alerts).

Visual layout

Figure 28. UI Component Layout Outline of Audit Hub Tabs

Audit Trail

Filters: [Action ▾] [User ▾] [Target ▾] [Date Range ▾] [🔍 Search]
[Export CSV] [Export PDF]

Audit Log Viewer

Timestamp	Action	User	Target	Changes	IP Address
11/21 09:00	Edt	Alice	Timesheet	hours 6→7	192.168.1.8
11/20 14:30	delete	Bob	Policy	--	192.168.1.8
…	…	…	…	…	…

* Click a row to open Diff Viewer

Diff Viewer (detail pane)

Before	**After**
hours: 6	hours: 7
status: pending	status: approved

Anomaly Detection Panel

- 5 failed login attempts in last hour
- 20 policy deletions by user Carol (↑200%)

Key User Workflows

1. **Investigate Suspicious Activity**:
    - Filter logs by action=delete → Identify bulk deletions → Cross-check with backup systems.
2. **Compliance Reporting**:
    - Export logs for a date range → Submit to auditors as evidence.
3. **Root Cause Analysis**:
    - Trace system errors by correlating logs (e.g., policy change → timesheet errors).

Testing Strategy

Table 84. Testing Strategy for Audit trail

Test Case	Validation
Log Integrity	Ensure edits/approvals trigger accurate logs.
Redacted Data	Mask sensitive fields (e.g., passwords) in exports.
Access Control	Block non-Super Admins from accessing logs.

Security & Validation

- **Access Control**:
    - Restricted to Super Admins (validated via JWT role=super_admin).
- **Validation Rules**:

- Auto-log all actions; no manual deletions allowed.
- Require reason for high-risk actions (e.g., policy overrides).

Edge Case Handling

Table 85. Edge Case Handling of Audit Trail Tab

Scenario	Resolution
Log Storage Limit	Auto-archive old logs to cold storage (AWS S3).
Data Corruption	Rebuild logs from backup snapshots.
High Volume	Paginate results with lazy loading.

Reports & Insights

Productivity Trends

Objective:

Enable Super Admins to analyze organization-wide productivity patterns, correlate behavioral metrics with business outcomes, and export actionable insights for strategic decision-making.

Key Features

1. **Cross-Team Comparisons**
   - Visualize metrics (focus time, idle rates, app usage) across departments/teams.

- Benchmark against company/industry averages.

2. **Cost Analysis**
   - Estimate productivity loss in hours/dollars (e.g., cost = unproductive hours × average hourly wage).
3. **Trend Identification**
   - Detect patterns like weekly productivity dips or app-driven distractions.
4. **Export-Ready Reports**
   - Generate PDF/CSV reports compatible with PowerBI/Tableau.

User Workflows

1. **Analyze Productivity Drop**
   - Filter by date (Q3 2023) → Compare Team A (Engineering) vs. Team B (Sales) → Identify correlation with high Slack usage.
2. **Calculate Financial Impact**
   - Select "Cost Analysis" → Input hourly wage (45)→Exportreportshowing45)→*Exportreportshowing*12K loss from unproductive hours.
3. **Set Benchmarks**
   - Overlay industry standards (e.g., "Tech sector avg. focus time: 5.2h/day") → Flag underperforming teams.

Backend Development

Table 86. API Endpoints of Productivity Trends

Endpoint	Method	Parameters	Purpose
GET /api/reports/productivity-trends	GET	startDate, endDate, teams[], metric=focus|idle|app	Fetch aggregated trends.
POST /api/reports/productivity-cost	POST	hourlyWage, unproductiveHours	Calculate financial impact of productivity loss.
GET /api/reports/export	GET	format=pdf|csv, reportType=trends	Export data to PowerBI/Tableau.

Data Models

ProductivityTrend Schema (MongoDB)

```
{
  orgId: ObjectId,
  date: Date,
  metrics: {
    team: String,
    focusHours: Number,    // From Member's Productivity Pulse
    idleHours: Number,     // From Activity Monitor
    topApps: [String],     // e.g., ["Slack", "VS Code"]
    unproductiveHours: Number
  },
  cost: {
    hourlyWage: Number,    // Configurable via POST
/api/reports/productivity-cost
    loss: Number           // Auto-calculated
  }
}
```

Frontend Integration

UI Components

1. Filters & Navigation

- **Team/Department Selector**

  Dropdown to choose one or more teams.

- **Date Range Picker**

  Select start/end dates (min 1 day, max 1 year).

- **Quick Controls**

  Prev/Next buttons to shift the window.

2. Trend Visualization

- **Line Chart (Focus Time)**

  Plots focus hours over time for each selected team.

- **Heatmap (Hourly Distribution)**

  Grid showing productive vs. idle minutes by hour of day.

- **Top Apps/URLs Widget**

  Lists most-used apps/websites during productive and unproductive periods.

3. Cost Analysis

- **Financial Calculator**

  Input hourly wage → displays total loss = unproductiveHours × wage.

- **Result Display**

  Shows "Total Loss: $XX,XXX" alongside a small bar or KPI card.

4. Export Panel

- **Format Selector**

  Choose PDF (summary) or CSV (raw data).

- **Export Action**

  Button to download current view; shows progress and success/failure toast.

Visual Layout

Figure 29. UI Component Layout Outline of Productivity Trends Tabs

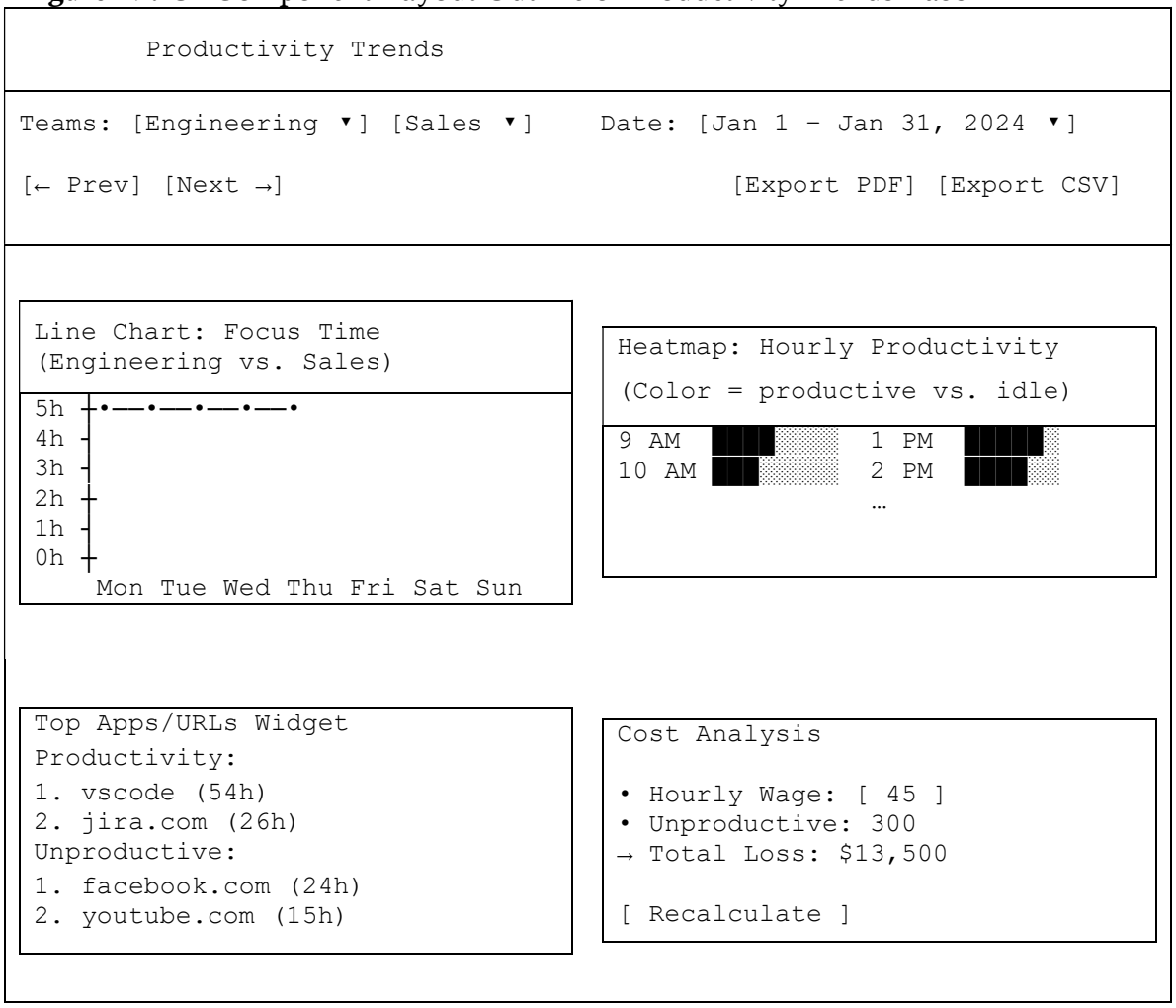


Table 87. Component-API Mapping of Productivity Trends

Component	API Endpoint	Key State/Props
Filters & Navigation	GET /api/teams GET /api/reports/productivity-trends	teams[], startDate, endDate
Line Chart	GET /api/reports/productivity-trends?metric=focus	focusData
Heatmap	GET /api/reports/productivity-trends?metric=hourly	hourlyData
Top Apps/URLs Widget	GET /api/reports/productivity-trends?metric=apps	productiveApps, unproductiveApps
Cost Analysis Calculator	POST /api/reports/productivity-cost	hourlyWage, unproductiveHours, totalLoss
Export Panel	GET /api/reports/export?format={pdf|csv}	exportFormat, exportProgress

Security & Validation

- **Access Control**: Restricted to Super Admins via JWT role check.
- **Validation**:
    - Reject date ranges > 1 year.
    - Sanitize input in cost calculator (e.g., prevent negative wages).
- **Data Privacy**: Aggregate data; exclude individual user details.

Testing Strategy

Table 88. Testing Strategy for Productivity Trends

Scenario	Validation
Cross-Team Comparison	Ensure Engineering data doesn't leak Sales metrics.
Cost Calculation	Verify `$45 × 300h = $13,500` matches API output.
Export Integrity	Confirm CSV headers align with PowerBI expectations.

Cross-Module Integration

1. **Activity Monitor**: Pull idle hours for cost calculations.
2. **Focus Dashboard**: Reuse productivity score formula:

   Score = (Productive Minutes + (Neutral × 0.5)) / Total × 100
3. **Audit Trail**: Log report exports and cost analyses.

Activity Levels

Objective

Enable Super Admins to monitor organization-wide activity patterns, correlate idle time with contextual factors (e.g., meetings, app usage), and validate attendance data against physical records (e.g., badge swipes).

Key Features

1. **Idle Time Heatmaps**
   - Visualize idle periods by time-of-day, department, or individual (threshold: 30+ minutes of inactivity).
2. **Meeting Correlation Analysis**

- Overlay calendar events to identify if idle time aligns with scheduled meetings.

3. **Attendance Cross-Check**
    - Compare digital timesheets with physical badge swipe data to flag discrepancies.
4. **Exportable Compliance Reports**
    - Generate audit-ready reports for HR/legal teams (e.g., GDPR, labor law compliance).

Backend Development

Table 89. API Endpoints of Activity Levels

Endpoint	Method	Parameters	Purpose
GET /api/activity/heatmap	GET	date, granularity=hourly/daily	Fetch idle/active time distribution.
GET /api/activity/correlations	GET	userId, startDate, endDate	Retrieve idle periods correlated with calendar events or app usage.
POST /api/attendance/crosscheck	POST	badgeData, timesheetData	Validate digital vs. physical attendance records.
GET /api/activity/export	GET	format=pdf/csv, type=attendance	Export cross-check results or heatmap data.

Data Models

ActivityLevel Schema (MongoDB)

```
{
  userId: ObjectId,
  date: Date,
  activity: {
    activeMinutes: Number,
    idleMinutes: Number,
    meetings: [{
      title: String,
      start: Date,
      end: Date,
      participants: [String] // Teams/attendees
    }],
    dominantApp: String // Most-used app during idle time
  },
  badgeSwipes: [{
    timestamp: Date,
    location: String // Office zone (e.g., "Floor 3 - North")
  }]
}
```

AttendanceCrossCheck Schema

```
{
  checkId: ObjectId,
  date: Date,
  userId: ObjectId,
  timesheetHours: Number,
  badgeHours: Number,
  discrepancy: Number, // timesheetHours - badgeHours
  status: "resolved" | "pending" | "flagged"
}
```

Frontend Integration

Component Breakdown

1. **Idle Time Heatmap**

Purpose: Visualize idle/active time distribution across teams or individuals.

API: GET /api/activity/heatmap

UI Elements:

- Toggle between team/individual view.
- Tooltip on hover: "9:00 AM: 15 idle users (25% of Engineering team)."

Request Example:

`/api/activity/heatmap?date=2024-03-15&granularity=hourly`
Response:

```
{
  "9:00 AM": { "active": 45, "idle": 15 },
  "10:00 AM": { "active": 32, "idle": 28 },
  // ...24-hour data
}
```

2. **Meeting Correlation Table**

Purpose: Link idle periods to calendar events (e.g., "3:00–4:00 PM idle time → Team Sync Meeting").

API: GET /api/activity/correlations

UI Elements:

- Expandable rows to show meeting details.
- Filter by meeting type (e.g., "All Hands", "1:1").

Request Example:

```
/api/activity/correlations?userId=user_123&startDate
=2024-03-01&endDate=2024-03-07
```

Response:

```
[
  {
    "date": "2024-03-01",
    "idlePeriods": [
      { "start": "14:00", "end": "14:30", "meeting": "Sprint
Retrospective" }
    ],
    "dominantApp": "outlook.exe"
  }
]
```

3. **Attendance Cross-Check Tool**

Purpose: Flag mismatches between timesheet entries and badge swipes.

API: POST /api/attendance/crosscheck

UI Elements:

- Side-by-side comparison table (Timesheet vs. Badge hours).
- "Resolve" button to annotate discrepancies (e.g., "Remote work approved").

Request Example:

```
// Payload
{
  "badgeData": [
    { "userId": "user_123", "date": "2024-03-15", "swipes":
["08:30", "17:45"] }
  ],
  "timesheetData": [
    { "userId": "user_123", "date": "2024-03-15", "hours": 9.5 }
  ]
```

```
}
```

Response:

```
{
  "flaggedUsers": [
    {
      "userId": "user_123",
      "date": "2024-03-15",
      "discrepancy": 2.5, // 9.5h (timesheet) - 7h (badge)
      "status": "flagged"
    }
  ]
}
```

4. **Export Panel**

Purpose: Generate compliance reports.

API: GET /api/activity/export

Response:

- PDF: Summary of discrepancies + resolution status.
- CSV: Raw timestamps (badge swipes vs. timesheet entries).

Request Example:

```
/api/activity/export?format=pdf&type=attendance&start=2024-03-
01&end=2024-03-31
```

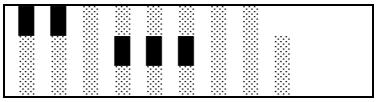
Visual Layout

Figure 30. UI Component Layout Outline of Activity Levels Tabs

Activity Levels

Date: [2024-03-15 ▾] View: [Team ▾] Granularity: [Hourly ▾] [Apply]

Idle Time Heatmap



(25% idle users at 9AM)

Meeting Correlation Table

Date	Idle Periods	Meeting
2024-03-15	14:00–14:30	Retro
2024-03-15	16:00–16:15	Standup
...	...	...

[Filter by Type ▾] [Refresh]

Attendance Cross-Check Tool

Member	Date	Timesheet Hrs	Badge Hrs	Δ Hrs
Alice Wong	2024-03-15	9.5	7.0	+2.5
Alice Wong	2024-03-15	8.0	8.0	0.0
…	…	…	…	…

[Resolve Selected] [Export CSV] [Export PDF]

Security & Validation

- **Data Validation**:
  - Reject cross-check requests with >1000 records (batch processing required).
  - Sanitize date inputs to prevent SQL injection.
- **Privacy**:

- Anonymize badge swipe locations (e.g., "Floor 3 - North" instead of precise coordinates).
- Exclude personal meeting titles (e.g., "1:1 with John" → "Private Meeting").

Cross-Module Integration

1. **Timesheet Explorer**: Pull timesheet data for cross-checking.
2. **Calendar Sync**: Fetch meeting details from Google Workspace/MS Outlook.
3. **Audit Trail**: Log all cross-check actions and report exports.

Productivity Rules

Activity Classifier

Objective: Define global rules to classify apps, websites, and user activity as *productive*, *neutral*, or *unproductive* across the organization. These rules override individual user preferences (e.g., Privacy Guard) to ensure compliance with organizational policies.

Key Features

1. **Rule Creation**:
   - **Wildcard/Regex Matching**: Classify domains/apps using patterns (e.g., *.salesforce.com = productive, regex:.*\.facebook\.com = unproductive).

- **Priority Levels**: Assign weights (1–5) to resolve conflicts between global and user-level rules.

2. **Override Handling**:
   - **Force Compliance**: Super Admins can lock rules to prevent user overrides (e.g., block all social media).
   - **Manual Exceptions**: Allow approved overrides (e.g., "Allow Facebook for Marketing Team").
3. **Integration with Privacy Guard**:
   - If an app/website is **blocked in Privacy Guard**, it is automatically classified as *unproductive* in reports, regardless of Activity Classifier rules.
   - If a Super Admin **classifies an app as productive**, Privacy Guard's "Full Block" mode will still prevent tracking but show a compliance warning.

Backend Development

Table 90. API Endpoints of Activity Classifier Tab

Endpoint		Method	Purpose
GET	/api/activity-classifier	GET	Fetch all classification rules.
POST	/api/activity-classifier	POST	Add a new rule (supports regex, wildcards, and manual overrides).
PUT	/api/activity-	PUT	Update rule priority or compliance mode

Endpoint	Method	Purpose
classifier		(e.g., lock/unlock for overrides).
DELETE /api/activity-classifier/{id}	DELETE	Remove a rule.

Data Model

```
{
    ruleId: String,
    pattern: String,      // e.g., "*.github.com", regex:.*\.slack\.com
    type: "domain" | "app",
    category: "productive" | "neutral" | "unproductive",
    priority: Number,     // 1 (lowest) – 5 (highest)
    isLocked: Boolean,    // If true, users cannot override this rule
    overrides: [{
      userId: ObjectId,   // Allowed exceptions
      category: String    // Override category for specific users/teams
    }]
  }
```

Integration with Privacy Guard (Member UI)

Hierarchy of Rules

1. **Super Admin Rules (Activity Classifier)**:
    - Highest priority if isLocked: true.
    - Example: If *.slack.com is classified as *unproductive* and locked, Members cannot mark it as *productive* in Privacy Guard.
2. **User Rules (Privacy Guard)**:
    - Apply only if no locked global rule exists for the app/website.

- Example: A Member can block spotify.exe locally, but it will still follow Activity Classifier's productivity categorization.

Conflict Resolution Workflow

1. **Check for Locked Global Rules** → Apply if present.
2. **Check User-Level Privacy Guard Rules** → Apply if no conflict.
3. **Log Conflicts**: If a user tries to override a locked rule, log the attempt in the audit trail.

Frontend Integration

1. **Rule Manager**

Purpose: Create, edit, or delete global classification rules.

UI Elements:

- **Rule Pattern Input**
- **Category Dropdown**
- **Priority Slider**
- **Lock Rule Toggle**
- **Exceptions Table**

Table 91. API Endpoint Mapping of Rule Manager Component

Component	API Endpoint	Method	Purpose
Load Rules	/api/activity-classifier	GET	Fetch all rules.
Add Rule	/api/activity-classifier	POST	Create a new rule.
Update Rule	/api/activity-classifier/{ruleId}	PUT	Edit a rule's priority, category,

Component	API Endpoint	Method	Purpose
			or lock status.
Delete Rule	/api/activity-classifier/{ruleId}	DELETE	Remove a rule.
Add Override Exception	/api/activity-classifier/{ruleId}/overrides	POST	Add a user/team exception to a rule.
Delete Override Exception	/api/activity-classifier/{ruleId}/overrides/{overrideId}	DELETE	Remove an exception.

Example Workflow: Adding a Rule

1. **User Action**:
    - Fills in the rule pattern *.slack.com, selects Unproductive, sets priority to 5, and toggles Lock Rule.
2. **API Request (POST** /api/activity-classifier**)**:

```
{
    "pattern": "*.slack.com",
    "type": "domain",
    "category": "unproductive",
    "priority": 5,
    "isLocked": true
  }
```

3. **API Response (201 Created)**:

```
{
    "ruleId": "rule_123",
    "pattern": "*.slack.com",
    "type": "domain",
    "category": "unproductive",
    "priority": 5,
    "isLocked": true,
```

```
  "overrides": []
}
```

4. **UI Update**:
    - The new rule appears in the rules list.
    - If isLocked: true, a padlock icon 🔒 is displayed next to the rule.

2. **Compliance Dashboard**

Purpose: Monitor conflicts between global rules and user overrides (e.g., Privacy Guard blocks).

UI Elements:

- **Conflict Table** (Rule, User, Action, Resolution Status)
- **Resolve Action Dropdown**

Table 92. API Endpoint Mapping of Compliance Dashboard

Component	API Endpoint	Method	Purpose
Load Conflicts	`/api/activity-classifier/conflicts`	GET	Fetch unresolved conflicts.
Resolve Conflict	`/api/activity-classifier/conflicts/{conflictId}`	POST	Mark a conflict as resolved (e.g., force compliance or allow override).

Example Workflow: Resolving a Conflict

1. **User Action**:
    - Selects a conflict: *"User @john blocked 'slack.com' (globally classified as Unproductive)."*
    - Chooses resolution: **"Force compliance: Unblock and apply global rule."**

2. **API Request (POST** /api/activity-classifier/conflicts/conflict_456**)**:

```
{
    "action": "force_compliance",
    "comment": "Slack is organizationally unproductive; override revoked."
  }
```

3. **API Response (200 OK)**:

```
{
    "conflictId": "conflict_456",
    "status": "resolved",
    "updatedRule": "rule_123"
  }
```

4. **UI Update**:

- The conflict is removed from the table.
- User @john's Privacy Guard settings for slack.com are reset to follow the global rule.

Visual Layout

Figure 31. UI Component Layout Outline of Activity Classifier Tab

Add / Edit Classification Rule

Rule Pattern: [*.facebook.com _____] ⓘ

Type:[▾ Domain] Category: [▾ Unproductive]

Priority: [▾ 5 (High)] 🔒 [✓] Lock Rule

Exceptions:

User	Category	Action
@marketing	Productive	Delete

[➕ Add Exception]

User: [▾ Select User] Category: [▾ Productive]

[💾 Save Rule] [❌ Cancel]

📋 Existing Rules

Pattern	Type	Category	Category	Lock	Actions
*.slack.com	Domain	Unproductive	5	🔒	[✏️] [🗑]
regex:.*\.facebook	Domain	Unproductive	4	🔒	[✏️] [🗑]

🛡️ Compliance Dashboard (Conflict Monitor)

Rule Pattern	User	User Action	Resolution
*.slack.com	@john	Blocked (Privacy)	🔴 Unresolved
regex:.*\.facebook	@sarah	Marked Productive	🟡 Pending

🔧 Resolve Selected Conflict:

[▾ Action: Force Compliance / Allow Override]

Comment: [_____]

[✅ Apply Resolution]

🔄 Auto-refresh every 5 min [📡 Enable Live Sync]

Security & Validation

- **Encryption**: Rules encrypted (AES-256) in transit and at rest.

- **Validation**:
    - Reject invalid regex patterns (e.g., .*.salesforce..com).
    - Prevent duplicate rules for the same domain/app.
- **Audit Trail**: Log all rule changes (user, timestamp, old/new values).

Edge Cases

Table 93. Edge Cases of Activity Classifier Tab

Scenario	Resolution
Conflicting Locked Rules	Higher priority wins; log alert for Super Admin review.
User Overrides Locked Rule	Block the action, notify user: "This rule is enforced organization-wide."
Ambiguous Pattern Matching	Flag during rule creation (e.g., `*.com` is too broad; require explicit scope).

Cross-Feature Integration

1. **Privacy Guard (Member UI)**:
    - Display warnings when user actions conflict with locked rules.
    - Example: *"⚠️ This app is organizationally classified as Unproductive. Blocking it is recommended."*
2. **Productivity Reports**:
    - Use Activity Classifier rules to calculate productivity scores, ignoring Privacy Guard's local blocks.
3. **Audit Trail (Super Admin)**:

- Track global rule changes and user override attempts.

CLIENT

Development Progress

Purpose: Provide clients with real-time visibility into code implementation, feature completion, and testing status while excluding sensitive internal data.

Development Progress

Purpose: Provide clients with **real-time visibility** into code implementation, feature completion, and testing status while **excluding sensitive internal data**.

Key Features

- **Commit Timeline**: Filtered GitHub commit history (sensitive paths excluded).
- **Feature Heatmap**: Progress by development area (frontend/backend/testing) using GitHub Issues.
- **Live Previews**: Secure, temporary staging links (24-hour expiry).
- **Test Summaries**: Automated reports from GitHub Actions CI/CD.

Tabs Descriptions

1. **Commit Timeline**

Purpose: Track code commits without exposing sensitive files or developers.

Material-UI Components:

- Timeline (vertical history), Tooltip (message previews), Chip (label filters).

Behavior:

- **GitHub Sync**: Fetches commits via GitHub API.
- **Filtering**: Excludes /config, .env, and other sensitive paths.
- **Anonymization**: Replaces GitHub usernames with "Dev Team A/B/C."
- **Interactions**:
    - Hover to view sanitized messages.
    - Click labels (e.g., bug) to filter commits.

    API Endpoint:
- GET /api/projects/{projectId}/commits

2. Feature Heatmap

Purpose: Visualize progress using GitHub Issues.

Material-UI Components:

- Grid (layout), LinearProgress (progress bars), Paper (category cards).

  Behavior:
- **GitHub Issues**: Pulls issues labeled frontend, backend, or testing.
    - Progress = (Closed Issues / Total Issues) * 100.
- **Drill-Down**: Click a category to view linked GitHub issues.
- **Color Coding**:
    - Green (>80%), Yellow (50-80%), Red (<50%).

    API Endpoint:
- GET /api/projects/{projectId}/issues

3. Live Previews

Purpose: Securely share staging/UAT environments.

Material-UI Components:

- Card (preview links), Button (generate URL), Dialog (security warnings).

  Behavior:

- **GitHub Actions**: Triggers deployment workflow on main branch commits.
- **Security**:
    - **Watermarks**: Added server-side via FFmpeg for videos/images.
    - **Auto-Expiry**: URLs expire in 24 hours (JWT validation).

      API Endpoint:

- POST /api/previews/generate

4. Test Summaries

Purpose: Display test results from GitHub Actions.

Material-UI Components:

- Table (metrics), Accordion (error details), IconButton (exports).

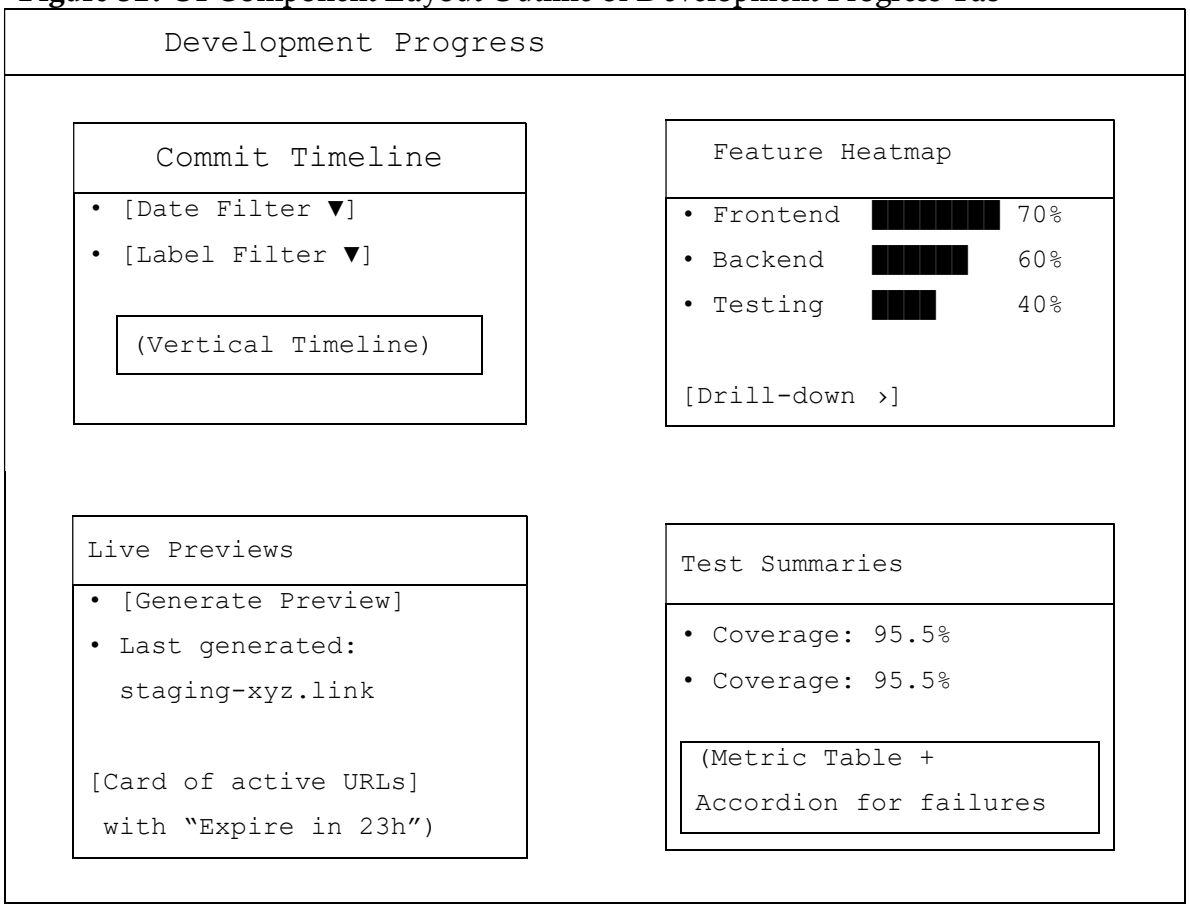
  Behavior:

- **CI/CD Sync**: Pulls results from the latest GitHub Actions run.

API Endpoint: GET /api/projects/{projectId}/tests

Visual Layout

Figure 32. UI Component Layout Outline of Development Progress Tab



Backend Development

Table 94. API Endpoints of Development Progress Tab

Component	Endpoint	Method	Description
Commit	/api/projects/{projectId}/commits	GET	Fetch GitHub commits

Component	Endpoint	Method	Description
Timeline			(filtered).
Feature Heatmap	`/api/projects/{projectId}/issues`	GET	Aggregate GitHub Issues by label.
Live Previews	`/api/previews/generate`	POST	Trigger GitHub Actions deployment.
Test Summaries	`/api/projects/{projectId}/tests`	GET	Retrieve test results from GitHub Actions.

Data Model

Commit Schema

```
{
    projectId: ObjectId,
    commitHash: String,       // "a1b2c3d"
    message: String,          // Sanitized (e.g., "Added login UI")
    date: Date,
    author: String,           // Anonymized (e.g., "Dev Team A")
    files: [String],          // Filtered (exclude .env, secrets/*)
    linkedTask: String        // "PROJ-123"
  }
```

TestSummary Schema

```
{
    projectId: ObjectId,
    coverage: Number,         // 95.5
    passed: Number,           // 148
    failed: Number,           // 2
    details: [{
      testName: String,       // "PaymentServiceTest"
      status: String,         // "passed"/"failed"
      log: String             // Redacted error messages
    }]
```

```
}
```

Frontend Integration

1. **Commit Timeline**:
    - Fetch commits → Filter sensitive paths → Render with Timeline.
2. **Feature Heatmap**:
    - Pull GitHub Issues → Group by label → Update LinearProgress bars.
3. **Live Previews**:
    - User clicks "Generate Preview" → Trigger GitHub Actions → Return URL.
4. **Test Summaries**:
    - Fetch test results → Sanitize logs → Populate Table.

Client Workflow

1. **Track Commits**:
    - Use date/label filters to view progress on specific features or bug fixes.
2. **Generate Previews**:
    - Click "Generate Preview" to get a 24-hour staging link.
3. **Review Tests**:
    - Expand failed tests to view sanitized logs. Export PDF/CSV for audits.

Dependencies

Table 95. Dependencies in Development Progress Tab

Tool/Service	Purpose
GitHub API	Fetch commits, issues, and test results.
GitHub Actions	Deploy previews and run CI/CD pipelines.
Material-UI	UI components (Timeline, Table, etc.).
Microsoft Teams	Notifications for alerts/errors.

Website Health

Integrations: Uptime monitoring (UptimeRobot), Google Lighthouse, SSL checks, and Google Analytics.

Purpose

Provide clients with **real-time insights** into their website's operational health, including uptime, performance, security, and traffic metrics, while **excluding sensitive infrastructure details**.

Key Features

- **Uptime Monitor**: Track website availability with 30-day historical data.
- **Performance Audits**: Weekly Lighthouse scores for SEO, accessibility, and speed.
- **SSL Monitoring**: Alerts for certificate expiration or vulnerabilities.
- **Traffic Analytics**: Visitor counts, bandwidth usage, and geographic distribution.

Tabs Descriptions

1. Uptime Monitor

Purpose: Display real-time and historical uptime/response times.

Material-UI Components:

- LineChart (response time trends), Chip (status indicators: 🟢 Up/🔴 Down).
- Table (30-day uptime history).

Behavior:

- **UptimeRobot Integration**:
  - Fetches uptime data every 5 minutes via UptimeRobot API.
  - Calculates uptime percentage:

Uptime % = (Total Uptime Minutes / Total Tracked Minutes) * 100

- **Interactions**:
  - Hover over the chart to view response times.
  - Filter by date range (last 7/30/90 days).

API Endpoint:

- GET /api/uptime/{projectId}

2. Performance Audits

Purpose: Highlight website performance using Google Lighthouse metrics.

Material-UI Components:

- RadarChart (Lighthouse categories), Chip (scores: ✅ Good/ ⚠️ Needs Improvement).
- Button (re-run audit).

Behavior:

- **Lighthouse Integration**:
    - Weekly automated audits via GitHub Actions.
    - Scores categorized as:
        - Performance (0–100)
        - Accessibility (0–100)
        - Best Practices (0–100)
        - SEO (0–100)
- **Interactions**:
    - Click "Run Audit Now" to trigger a manual scan (limited to 1/hour).
    - Expand categories to view improvement recommendations.

API Endpoint:

- GET /api/performance/{projectId}

3. SSL Monitoring

Purpose: Track SSL/TLS certificate validity and vulnerabilities.

Material-UI Components:

- Alert (expiry warnings), Table (certificate details).

- IconButton (renewal docs link).

Behavior:

- **SSL Labs API Integration**:
    - Daily checks for certificate expiration, weak ciphers, or vulnerabilities.
    - Alerts triggered if expiration < 30 days.
- **Interactions**:
    - Click "Renewal Guide" to open documentation.
    - View certificate issuer, expiry date, and SANs (Subject Alternative Names).

API Endpoint:

- GET /api/ssl/{projectId}

4. Traffic Analytics

Purpose: Show visitor trends and bandwidth usage.

Material-UI Components:

- BarChart (visitors), PieChart (geographic distribution).
- Table (top pages by traffic).

Behavior:

- **Google Analytics Integration**:
    - Fetches data via Google Analytics API.
    - Metrics:

- Unique visitors, page views, bounce rate.
- Bandwidth usage (MB).

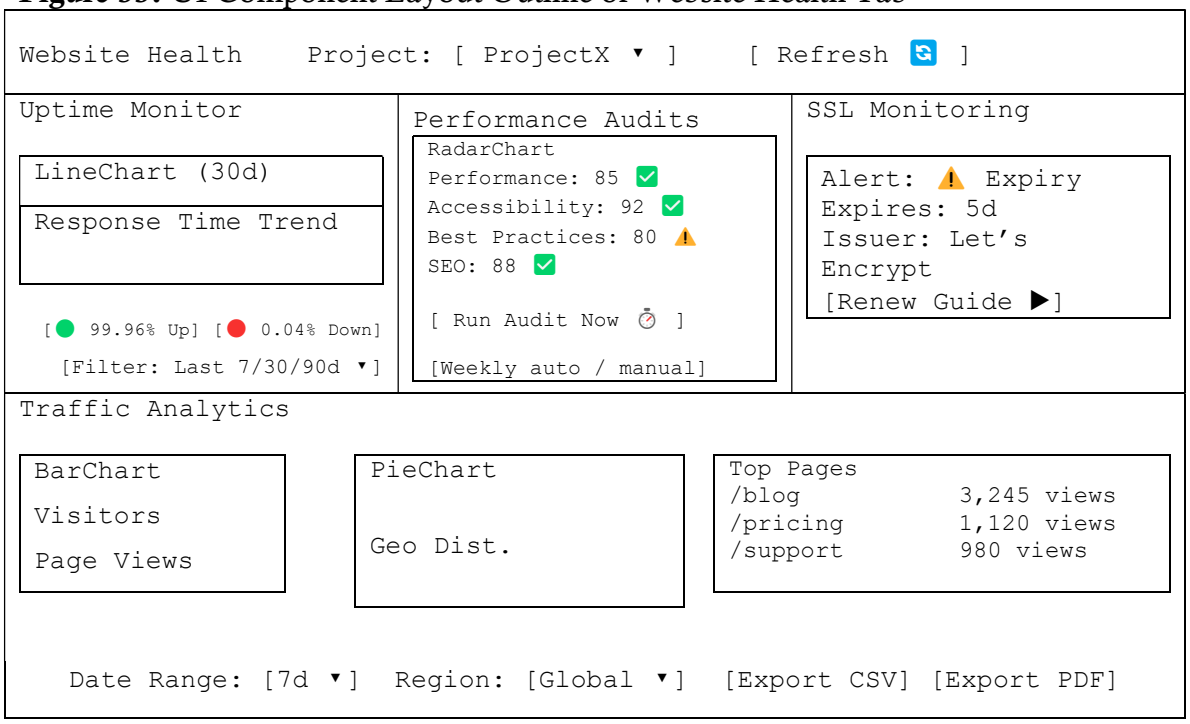
- **Interactions**:
  - Filter by date range or region.
  - Export CSV for custom reporting.

  API Endpoint:

- GET /api/traffic/{projectId}

Visual Layout

Figure 33. UI Component Layout Outline of Website Health Tab



Backend Development

Table 96. API Endpoints of Website Health Tab

Component	Endpoint	Method	Description
Uptime Monitor	`/api/uptime/{projectId}`	GET	Fetch uptime and response time data.
Performance Audits	`/api/performance/{projectId}`	GET	Retrieve Lighthouse scores.
SSL Monitoring	`/api/ssl/{projectId}`	GET	Check SSL certificate health.
Traffic Analytics	`/api/traffic/{projectId}`	GET	Pull visitor and bandwidth metrics.

Data Models

Uptime Schema:

```
{
    projectId: ObjectId,
    date: Date,
    status: "up" | "down",
    responseTime: Number,  // Milliseconds
    outages: [{
      start: Date,
      end: Date,
      cause: String        // "Server error", "Network outage"
    }]
  }
```

Lighthouse Schema:

```
{
    projectId: ObjectId,
    date: Date,
    performance: Number,   // 0-100
    accessibility: Number,
```

```
    bestPractices: Number,
    seo: Number,
    recommendations: [{
      category: String,    // "Performance", "SEO"
      suggestion: String   // "Optimize images", "Minify CSS"
    }]
  }
```

SSL Schema:

```
{
    projectId: ObjectId,
    issuer: String,        // "Let's Encrypt"
    expiry: Date,
    san: [String],         // ["*.example.com", "example.com"]
    vulnerabilities: [{
      name: String,        // "Heartbleed", "POODLE"
      severity: "low" | "medium" | "high"
    }]
  }
```

Traffic Schema:

```
{
    projectId: ObjectId,
    date: Date,
    visitors: Number,
    pageViews: Number,
    bandwidthMB: Number,
    topPages: [{
      path: String,        // "/blog", "/pricing"
      views: Number
    }]
  }
```

Frontend Integration

Data Flow

1. **Uptime Monitor**:
    - Fetch data → Render line chart and status chips.
2. **Performance Audits**:
    - Retrieve Lighthouse scores → Display radar chart and recommendations.
3. **SSL Monitoring**:
    - Check certificate health → Show alerts for near-expiry.
4. **Traffic Analytics**:
    - Pull Google Analytics → Update charts and tables.

Security & Permissions

- **Data Masking**:
    - Hide server IPs and internal hostnames in SSL details.
    - Limit Google Analytics scope to read-only.
- **Access Control**:
    - Clients can only view their project's data (validated via JWT projectId).

Error Handling

Table 97. Error Handling for Website Health Tab

Error	UI Response	Teams Alert
SSL Expiry <7d	Red alert banner	"SSL for ProjectX expires in 5 days."

Error	UI Response	Teams Alert
Lighthouse <50	Yellow warning card	"Performance score dropped to 45."
500 Server Error	Display error toast	"Backend error: Check logs for ProjectX."

User Guide

Client Workflow

1. **Check Uptime**:
    - View real-time status and historical outages.
2. **Audit Performance**:
    - Review Lighthouse scores and implement suggestions.
3. **Renew SSL**:
    - Follow the "Renewal Guide" if expiry is near.
4. **Analyze Traffic**:
    - Identify peak traffic times and popular pages.

Dependencies

Table 98. Dependecies in Website Health Tab

Tool/Service	Purpose
UptimeRobot	Uptime and response time monitoring.
Google Lighthouse	Performance audits.
SSL Labs API	SSL certificate checks.
Google Analytics	Visitor and traffic analytics.

Deliverables Tab

Integrations: Figma/Adobe XD (design files), AWS S3 (file storage), DocuSign (contracts).

Purpose

Provide clients with **secure, version-controlled access** to project artifacts, including design files, contracts, and test environments, while ensuring intellectual property protection via watermarks and access controls.

Key Features

- **Versioned Design Exports**: Auto-sync Figma/Adobe XD files with version history.
- **Watermarked Contracts**: PDF contracts with dynamic client-specific watermarks.
- **Temporary Test Links**: Time-limited access to staging/UAT environments.
- **Approval Workflows**: Digital sign-off for final deliverables.

Tabs Descriptions

1. Versioned Design Exports

Purpose: Share iteratively updated design files (Figma/XD) with clients.

Material-UI Components:

- Accordion (version history), DataGrid (file metadata), Button (download).

Behavior:

- **Auto-Sync**: New Figma/XD versions trigger GitHub Actions to export .fig/.xd files to AWS S3.
- **Version History**:
    - Label versions (e.g., "v1.2 – Dashboard Prototype").
    - Compare changes via visual diffs (image overlays).
- **Interactions**:
    - Click a version to download or view changelog.
    - Restore previous versions (admin-only).

API Endpoint:

- GET /api/deliverables/designs/{projectId}

2. Watermarked Contracts

Purpose: Distribute contracts with tamper-proof client-specific watermarks.

Material-UI Components:

- Card (contract preview), Chip (status: "Draft"/"Signed"), IconButton (download).

Behavior:

- **Dynamic Watermarks**:
    - Inject client name, date, and IP address into PDFs using pdf-lib.
    - Example:

CONFIDENTIAL – {Client Name} – {Date} – {IP}

- **DocuSign Integration**:
    - Click "Sign" to redirect to DocuSign workflow.
    - Update status to "Signed" post-completion.

API Endpoint:

- GET /api/deliverables/contracts/{projectId}

3. Temporary Test Links

Purpose: Provide time-limited access to test environments.

Material-UI Components:

- List (environment links), Chip (status: "Active"/"Expired"), Button (copy link).

Behavior:

- **Auto-Expiry**:
    - Links expire after 24 hours (configurable).
    - Refresh tokens via POST /api/deliverables/refresh-link.
- **Security**:
    - Environment data reset nightly to prevent data leakage.
    - IP whitelisting for sensitive projects.

API Endpoint:

- GET /api/deliverables/environments/{projectId}

4. Approval Workflows

Purpose: Streamline client sign-off for final deliverables.

Material-UI Components:

- Stepper (approval stages), Checkbox (acceptance), Button (submit).

Behavior:

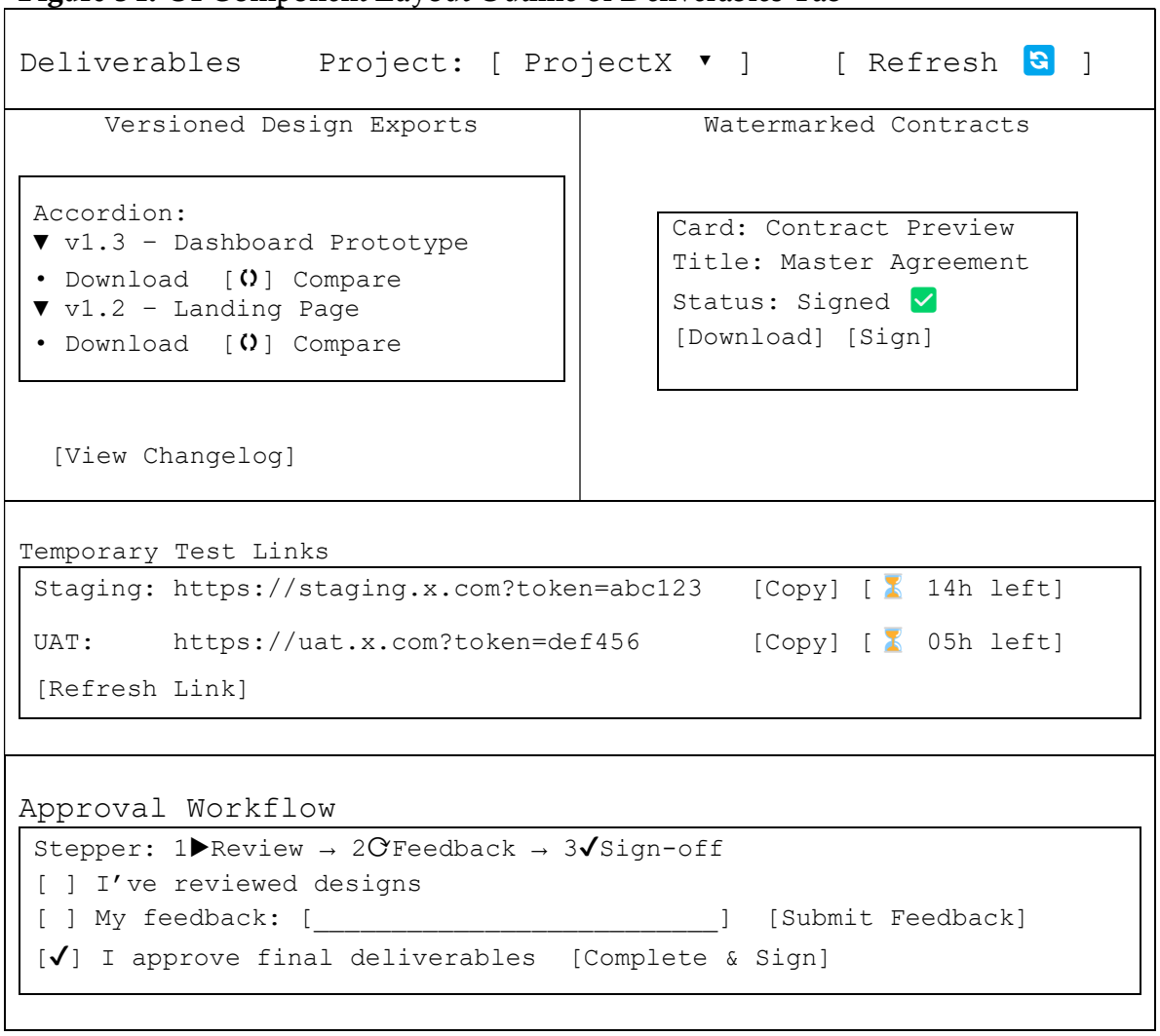
- **Multi-Stage Approval**:
    1. Client reviews deliverables.
    2. Submits feedback or requests changes.
    3. Signs off via DocuSign or in-app checkbox.
- **Audit Trail**: Logs all actions (e.g., "Approved by Client A on 2023-11-20").

API Endpoint:

- POST /api/deliverables/approve/{projectId}

Visual Layout

Figure 34. UI Component Layout Outline of Deliverables Tab



Backend Development

Table 99. API Endpoints of Deliverables Tab

Component	Endpoint	Method	Description

Component	Endpoint	Method	Description
Design Exports	`/api/deliverables/designs/{projectId}`	GET	Fetch versioned design files.
Contracts	`/api/deliverables/contracts/{projectId}`	GET	Retrieve watermarked PDFs.
Test Links	`/api/deliverables/environments/{projectId}`	GET	Generate/copy temporary links.
Approval Workflows	`/api/deliverables/approve/{projectId}`	POST	Submit client sign-off.

Data Models

DesignExport Schema:

```
{
    projectId: ObjectId,
    version: String,          // "v1.2"
    fileUrl: String,          // AWS S3 path
    changelog: String,        // "Added dashboard UI"
    createdAt: Date,
    author: String            // "Design Team"
  }
```

Contract Schema:

```
{
    projectId: ObjectId,
    title: String,            // "ProjectX Master Agreement"
    fileUrl: String,          // Watermarked PDF path
    status: "draft" | "signed",
    signedBy: String,         // Client email
    signedAt: Date
  }
```

TestLink Schema:

```
{
    projectId: ObjectId,
    url: String,              //
"https://staging.projectx.com?token=abc123"
    expiresAt: Date,
    environment: String       // "Staging", "UAT"
  }
```

Approval Schema:

```
{
    projectId: ObjectId,
    step: Number,             // 1 (Review), 2 (Feedback), 3 (Sign-off)
    approvedBy: String,       // Client email
    comments: String,         // "Update footer layout"
    approvedAt: Date
  }
```

Frontend Integration

Data Flow

1. **Design Exports**:
    - Fetch versions → Render Accordion with download links.
2. **Contracts**:
    - Load watermarked PDFs → Preview in <embed> or new tab.
3. **Test Links**:
    - Generate/copy URLs → Show expiry countdown via useCountdown hook.
4. **Approval Workflows**:
    - Track steps with Stepper → Submit sign-off to backend.

Security & Permissions

- **Watermarking**:
    - Inject client-specific metadata into PDFs using pdf-lib.
    - Example: This document is confidential property of {Client Name}.
- **Access Control**:
    - Temporary links use JWT tokens with projectId claim.
    - Design files stored in AWS S3 with read-only presigned URLs.

User Guide

Client Workflow

1. **Download Designs**:
    - Select a version → Click "Download" (watermarked files only).
2. **Sign Contracts**:
    - Review PDF → Click "Sign" to complete DocuSign workflow.
3. **Access Test Environments**:
    - Copy temporary link → Use within 24 hours.
4. **Approve Deliverables**:
    - Follow Stepper steps → Submit feedback or final approval.

Dependencies

Table 100. Dependecies in Deliverables Tab

Error	UI Response	Teams Alert
Expired Link	"Link expired. Request new."	None
Invalid Signature	"Re-sign contract."	"Contract signature failed for ProjectX."
403 Access Denied	Redirect to login	"Unauthorized access to deliverables."

Maintenance Schedule

Integrations: Microsoft Teams (alerts), GitHub Actions (automation), UptimeRobot (dependency monitoring).

Purpose

Provide clients with **visibility into scheduled and historical maintenance activities**, including planned updates, security patches, and emergency fixes, while ensuring transparency into system reliability and third-party dependencies.

Key Features

- **Maintenance Calendar**: Visual timeline of planned, security, and emergency maintenance.
- **Update Logs**: Detailed changelogs with before/after performance metrics.
- **Dependency Status**: Track third-party service health (e.g., AWS, GitHub).
- **Performance Reports**: Compare system performance pre/post-maintenance.

Tabs Descriptions

1. Maintenance Calendar

Purpose: Display upcoming and past maintenance windows.

Material-UI Components:

- FullCalendar (interactive calendar), Chip (type: 🟦 Planned/ 🟨 Security/ 🟥 Emergency).
- Tooltip (hover for details).

Behavior:

- **GitHub Actions Sync**: Auto-add maintenance events from .github/workflows/maintenance.yml.
- **Filtering**: Toggle event types (planned/security/emergency).
- **Interactions**:
    - Click an event to view changelog and downtime duration.
    - Subscribe to iCal feeds for external calendar integration.

API Endpoint:

- GET /api/maintenance/events/{projectId}

2. Update Logs

Purpose: Track maintenance activities and their impact.

Material-UI Components:

- DataGrid (sortable logs), Chip (status: ✅ Complete/ ⚠️ Failed), Button (export CSV).

  Behavior:

- **Changelog Details**:
  - **Before/After Metrics**: CPU load, response time, error rates.
  - **Linked PRs**: GitHub pull requests associated with the update.
- **Search**: Filter logs by date, type, or keyword.

  API Endpoint:

- GET /api/maintenance/logs/{projectId}

3. Dependency Tracking

Purpose: Monitor third-party service status (e.g., AWS, GitHub).

Material-UI Components:

- Card (service status), LinearProgress (uptime), Link (status page).

  Behavior:

- **UptimeRobot Integration**:
  - Poll status pages every 5 minutes.
  - Flag degraded services with 🟠 / 🔴 indicators.
- **Interactions**:
  - Click a service to view incident history.

    API Endpoint:

- GET /api/maintenance/dependencies/{projectId}

4. Performance Reports

Purpose: Quantify maintenance impact on system performance.

Material-UI Components:

- LineChart (response times), BarChart (error rate comparison), IconButton (export PDF).

Behavior:

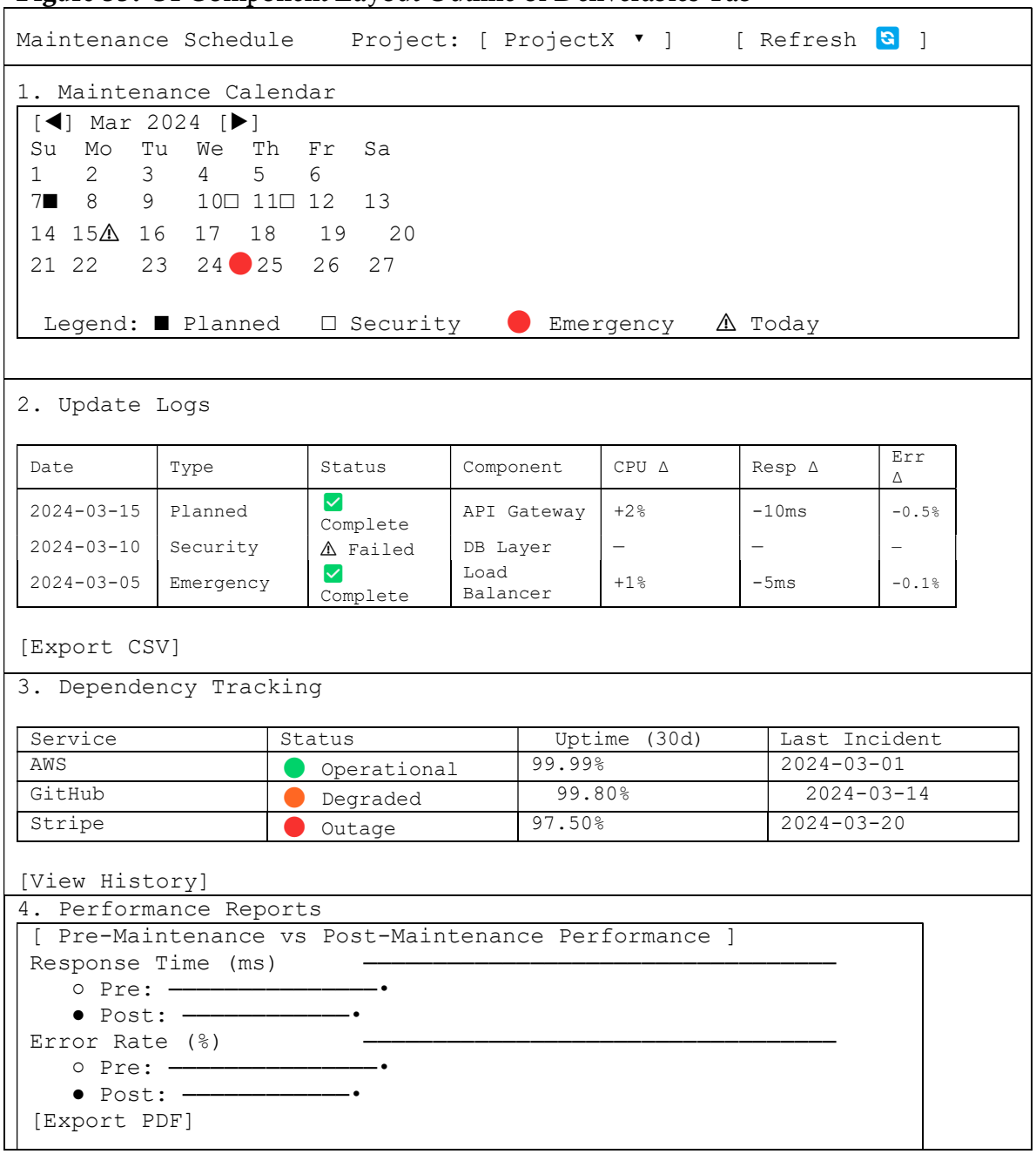
- **Metrics**:
  - **Pre-Maintenance**: Baseline performance (7-day average).
  - **Post-Maintenance**: 24-hour/7-day comparison.
- **Auto-Generated Insights**:
  - Example: "Post-update error rates decreased by 15%."

API Endpoint:

- GET /api/maintenance/performance/{projectId}

Visual Layout

Figure 35. UI Component Layout Outline of Deliverables Tab



Backend Development

Table 101. API Endpoints of Maintainance Schedule Tab

Component	Endpoint	Method	Description
Maintenance Calendar	`/api/maintenance/events/{projectId}`	GET	Fetch maintenance events.
Update Logs	`/api/maintenance/logs/{projectId}`	GET	Retrieve changelogs and metrics.
Dependency Tracking	`/api/maintenance/dependencies/{projectId}`	GET	Poll third-party statuses.
Performance Reports	`/api/maintenance/performance/{projectId}`	GET	Generate pre/post metrics.

Data Models

MaintenanceEvent Schema:

```
{
  projectId: ObjectId,
  start: Date,
  end: Date,
  type: "planned" | "security" | "emergency",
  description: String,      // "Database migration"
  status: "scheduled" | "completed" | "failed",
  downtimeMinutes: Number,  // 30
  linkedPR: String          // "https://github.com/.../pull/123"
}
```

UpdateLog Schema:

```
{
  projectId: ObjectId,
  maintenanceId: ObjectId,  // Linked to MaintenanceEvent
  changes: [{
    component: String,      // "API Gateway"
    action: String          // "Upgraded to v2.5.1"
```

```
  }],
  metrics: {
    cpuLoad: Number,        // Pre/post values
    responseTime: Number,
    errorRate: Number
  }
}
```

DependencyStatus Schema:

```
{
  projectId: ObjectId,
  service: String,          // "AWS", "GitHub"
  status: "operational" | "degraded" | "outage",
  lastIncident: Date,
  uptime30d: Number         // 99.95
}
```

Frontend Integration

Data Flow

1. **Maintenance Calendar**:
    - Fetch events → Render on FullCalendar with color-coded chips.
2. **Update Logs**:
    - Load logs → Populate DataGrid with sortable columns.
3. **Dependency Tracking**:
    - Poll UptimeRobot → Update status cards.
4. **Performance Reports**:
    - Compare pre/post metrics → Generate charts.

Security & Permissions

- **Role-Based Access**:
    - Clients can only **view** maintenance schedules.
    - Admins/DevOps can **create/edit** events (via GitHub Actions).
- **Data Validation**:
    - Reject overlapping maintenance windows.
    - Ensure linkedPR points to valid GitHub pull requests.

Error Handling

Table 102. Error Handling of Maintainance Schedule Tab

Error	UI Response	Teams Alert
Overlap Detected	"Reschedule maintenance."	"Maintenance conflict for ProjectX."
Dependency Outage	Red status card	"AWS outage impacting ProjectX."
403 Access Denied	Redirect to login	"Unauthorized edit attempt."

User Guide

Client Workflow

1. **Review Schedule**:
    - Check the calendar for upcoming maintenance.
    - Subscribe to iCal feeds for reminders.
2. **Analyze Updates**:
    - Search logs to verify bug fixes or performance improvements.
3. **Monitor Dependencies**:

- Track third-party service statuses in real-time.

4. **Export Reports**:
    - Download PDF/CSV for compliance audits.

Dependencies

Table 103. Dependencies in Maintenance Schedule Tab

Tool/Service	Purpose
UptimeRobot	Third-party service monitoring.
GitHub Actions	Automate maintenance event creation.
FullCalendar	Interactive calendar component.
Material-UI Icons	Status indicators (✅, ⚠️, 🔴).

Requests & Support

Purpose: Streamline client communication, manage support requests, and resolve issues efficiently while ensuring transparency and adherence to service-level agreements (SLAs).

Components

1. Ticket Submission

Purpose: Enable clients to submit support requests with context.

- **Material-UI Components**:
    - TextField (title, description), Select (priority, category), Button (submit).
- **Behavior**:

- Auto-saves drafts locally.
- Validates required fields (title, priority).
- Attachments uploaded to AWS S3 (max 5 files, 10MB each).

- **API Endpoint**:
  - POST /api/tickets

2. Active Tickets Grid

Purpose: Track and manage open/closed tickets.

- **Material-UI Components**:
  - DataGrid (columns: ID, Title, Priority, Status), Chip (🟡 Pending/ 🟢 Resolved/ 🔴 Overdue).
- **Behavior**:
  - Real-time updates via WebSocket (/ws/tickets).
  - Filter by status, priority, or keyword.
- **API Endpoint**:
  - GET /api/tickets?projectId={id}

3. Knowledge Base Explorer

Purpose: Provide instant solutions to common issues.

- **Material-UI Components**:
  - Accordion (categories), SearchBar, Typography (article content).
- **Behavior**:
  - AI suggests articles based on ticket content (OpenAI integration).

    - Users rate articles (👍 / 👎) to improve relevance.

- **API Endpoint**:
    - GET /api/knowledge-base?query={searchTerm}

4. Change Request Pipeline

Purpose: Manage client-proposed changes with budget oversight.

- **Material-UI Components**:
    - Stepper (draft → review → approval), Alert (budget warnings).
- **Behavior**:
    - Flags requests exceeding project budget.
    - Requires manager approval for submission.
- **API Endpoint**:
    - POST /api/change-requests

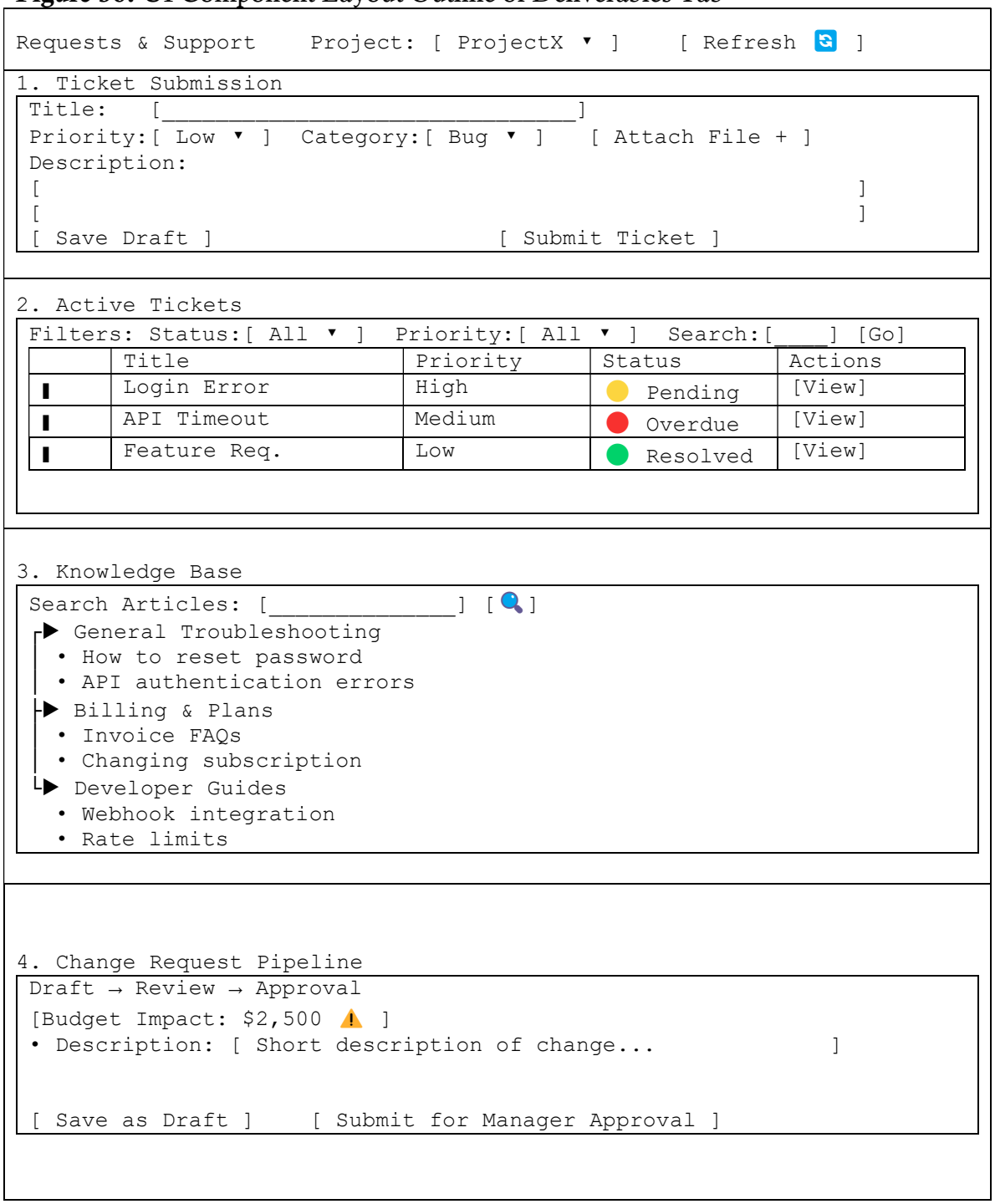
Backend Development

Table 104. API Endpoints in Requests & Support Tab

Component	Endpoint	Method	Purpose
Ticket Submission	`/api/tickets`	POST	Create a new ticket.
Active Tickets Grid	`/api/tickets`	GET	Fetch tickets for a project.
Knowledge Base Explorer	`/api/knowledge-base`	GET	Search self-service articles.
Change Request Pipeline	`/api/change-requests`	POST	Submit change proposals.

Visual Layout

Figure 36. UI Component Layout Outline of Deliverables Tab



Data Models

Ticket Schema

```
interface Ticket {
  ticketId: string;
  projectId: string;
  title: string;
  description: string;
  priority: "low" | "medium" | "high";
  status: "open" | "in_progress" | "resolved";
  attachments: string[]; // AWS S3 paths
  createdBy: string;
  createdAt: Date;
}
```

ChangeRequest Schema

```
interface ChangeRequest {
  requestId: string;
  projectId: string;
  description: string;
  budgetImpact: number;
  status: "draft" | "submitted" | "approved" | "rejected";
}
```

Frontend Integration

1. **Ticket Submission**:
    - Submit form → Validate → Upload attachments → POST /api/tickets.
2. **Active Tickets**:
    - Fetch tickets on mount → Render in DataGrid → Update via WebSocket.

3. **Knowledge Base**:
    - Search triggers GET /api/knowledge-base → Display results in Accordion.
4. **Change Requests**:
    - Draft → Validate budget → Submit POST /api/change-requests.

Security & Permissions

- **Role-Based Access**:
    - **Clients**: Submit tickets, view knowledge base.
    - **Support Agents**: Update ticket status, assign tickets.
    - **Managers**: Approve/reject change requests.
- **Validation**:
    - File type restrictions (PDF, PNG, JPG only).
    - Budget impact cannot exceed 20% of project total.

Error Handling

Error	UI Response	Teams Alert
Invalid File Type	"Unsupported file type."	"Client uploaded .exe file in ProjectX."
Budget Overrun	"Request exceeds budget."	"Change request overrun: $5K."
SLA Breach	Highlight overdue tickets.	"5 tickets missed SLA in ProjectX."

User Guide

1. **Submit a Ticket**:
    - Navigate to Requests & Support → Fill form → Attach files → Submit.
2. **Track Progress**:
    - View "Active Tickets" grid → Filter by priority → Click for details.
3. **Propose Changes**:
    - Draft change request → Check budget impact → Submit for approval.
4. **Search Knowledge Base**:
    - Type keywords → Review AI-suggested articles → Rate solutions.